

UNIT - III

**Non Linear Data Structures – Trees and
Graphs.**

NON – LINEAR DATA STRUCTURES

- A non-linear data structure is another important type in which data elements are not arranged sequentially; mainly, data elements are arranged in random order without forming a linear structure.
- Data elements are present at the multilevel, for example, tree.
- In trees, the data elements are arranged in the hierarchical form, whereas in graphs, the data elements are arranged in random order, using the edges and vertex.
- Multiple runs are required to traverse through all the elements completely. Traversing in a single run is impossible to traverse the whole data structure.
- Each element can have multiple paths to reach another element.

NON – LINEAR DATA STRUCTURES

- The data structure where data items are not organized sequentially is called a non-linear data structure. In other words, data elements of the non-linear data structure could be connected to more than one element to reflect a special relationship among them.

Types:

- 1) Tree**
- 2) Graphs**

NON – LINEAR DATA STRUCTURES PROPERTIES

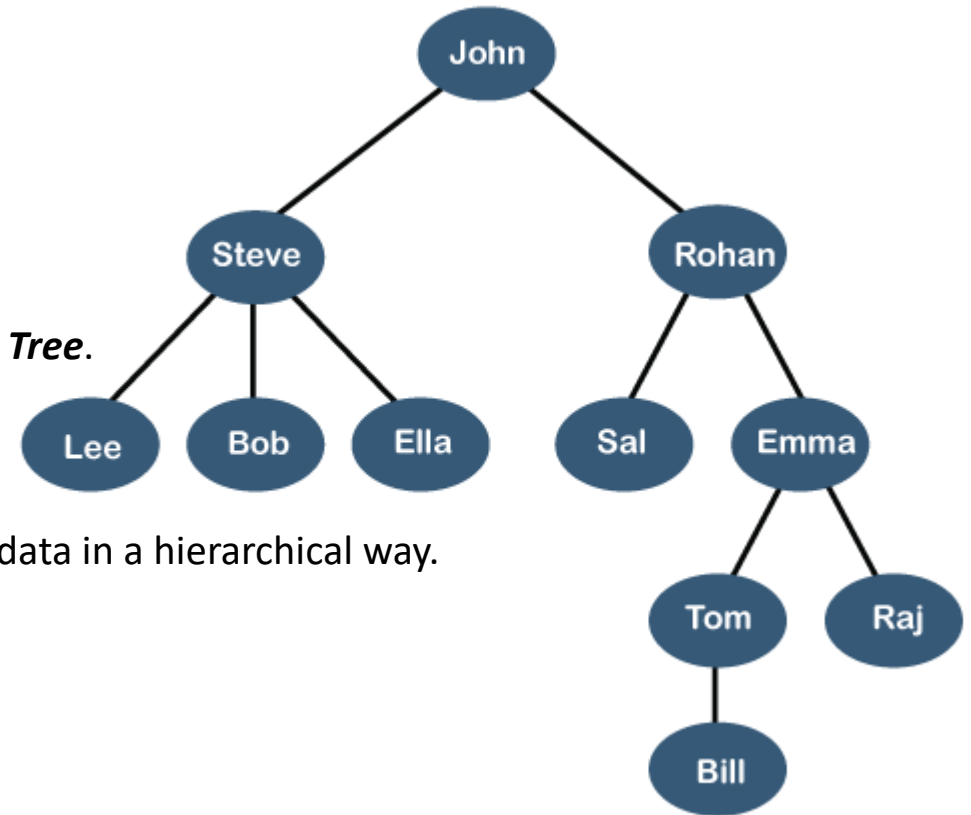
- It is used to store the data elements combined whenever they are not present in the contiguous memory locations.
- It is an efficient way of organizing and properly holding the data.
- It reduces the wastage of memory space by providing sufficient memory to every data element.
- Unlike in an array, we have to define the size of the array, and subsequent memory space is allocated to that array; if we don't want to store the elements till the range of the array, then the remaining memory gets wasted.
- So to overcome this factor, we will use the non-linear data structure and have multiple options to traverse from one node to another.
- Data is stored randomly in memory.
- It is comparatively difficult to implement.
- Multiple levels are involved.
- Memory utilization is effective.

Tree

A **tree** is also one of the data structures that represent hierarchical data. Suppose we want to show the employees and their positions in the hierarchical form then it can be represented as shown below:

The above tree shows the **organization hierarchy** of some company. In the above structure, **john** is the **CEO** of the company, and John has two direct reports named as **Steve** and **Rohan**. Steve has three direct reports named **Lee, Bob, Ella** where **Steve** is a manager. Bob has two direct reports named **Sal** and **Emma**. **Emma** has two direct reports named **Tom** and **Raj**. Tom has one direct report named **Bill**. This particular logical structure is known as a **Tree**. Its structure is similar to the real tree, so it is named a **Tree**. In this structure, the **root** is at the top, and its branches are moving in a downward direction.

Therefore, we can say that the Tree data structure is an efficient way of storing the data in a hierarchical way.

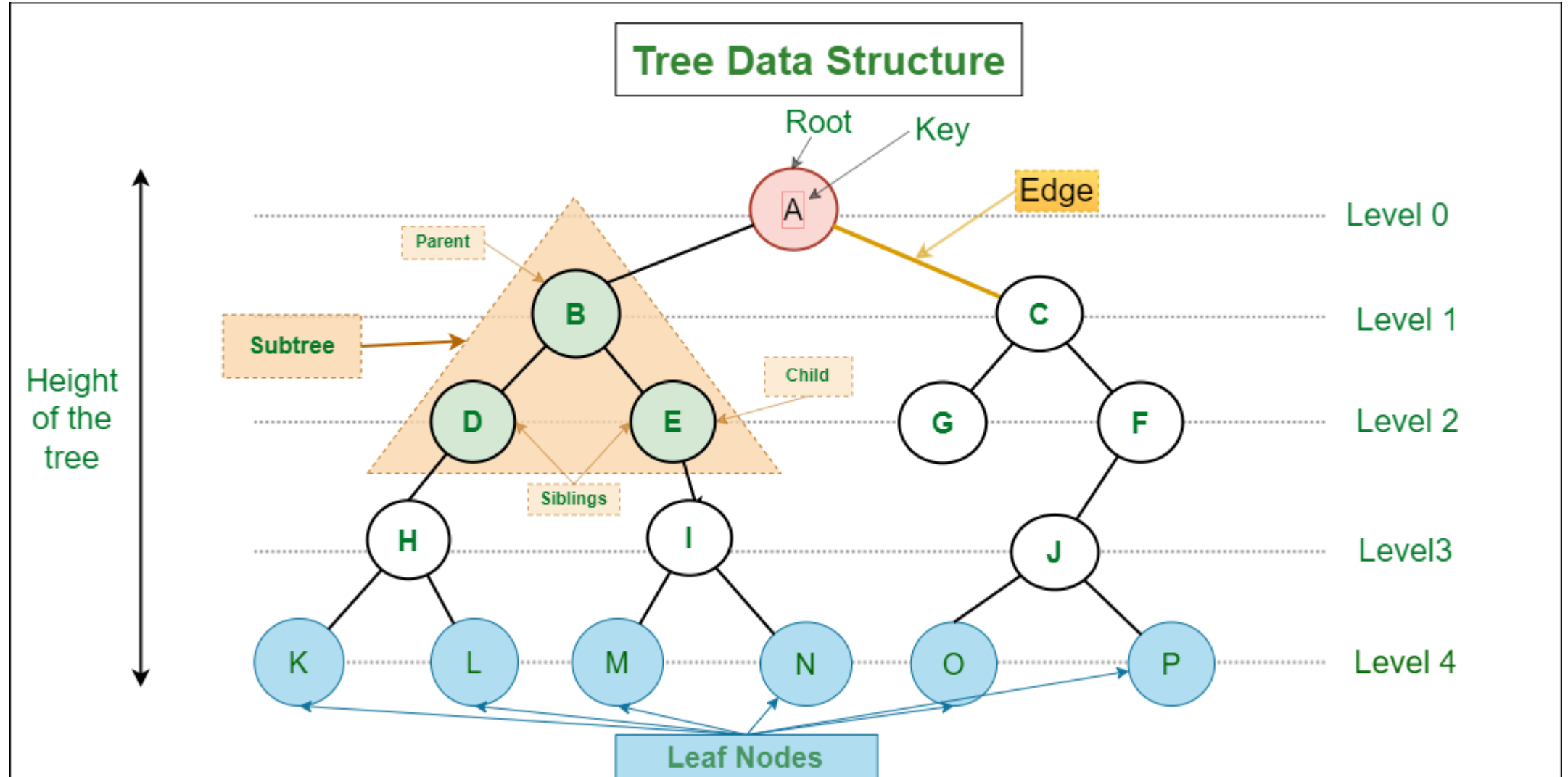


Tree

Let's understand some key points of the Tree data structure.

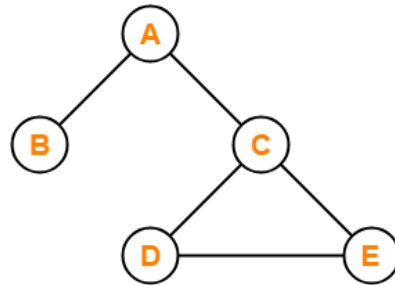
- A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy.
 - A tree data structure is a non-linear data structure because it does not store in a sequential manner.
 - It is a hierarchical structure as elements in a Tree are arranged in multiple levels.
 - In the Tree data structure, the topmost node is known as a root node.
 - Each node contains some data, and data can be of any type.
 - In the above tree structure, the node contains the name of the employee, so the type of data would be a string.
 - Each node contains some data and the link or reference of other nodes that can be called children
-
- **Why Tree is considered a non-linear data structure?**
 - The data in a tree are not stored in a sequential manner i.e, they are not stored linearly. Instead, they are arranged on multiple levels or we can say it is a hierarchical structure. For this reason, the tree is considered to be a non-linear data structure.

Tree



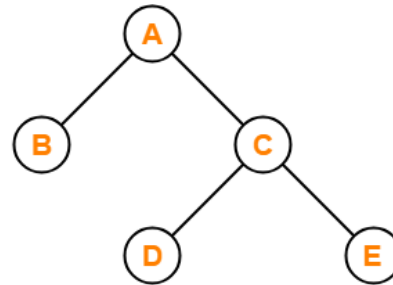
Tree

- **Properties of a Tree:**
- The important properties of tree data structure are-
- There is one and only one path between every pair of vertices in a tree.
- A tree with n vertices has exactly $(n-1)$ edges.
- A graph is a tree if and only if it is minimally connected.
- Any connected graph with n vertices and $(n-1)$ edges is a tree.



X

This graph is not a Tree



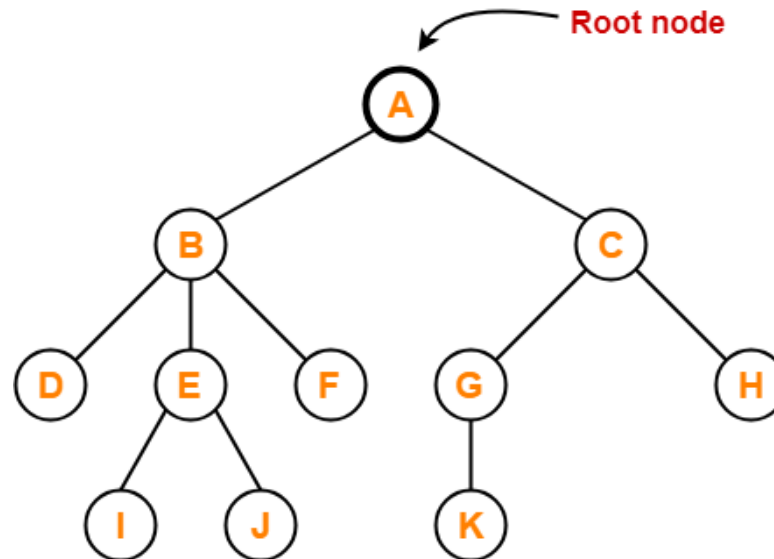
✓

This graph is a Tree

Tree

- **1. Root-**

- The first node from where the tree originates is called as a **root node**.
- In any tree, there must be only one root node.
- We can never have multiple root nodes in a tree data structure.
- Here, node A is the only root node.

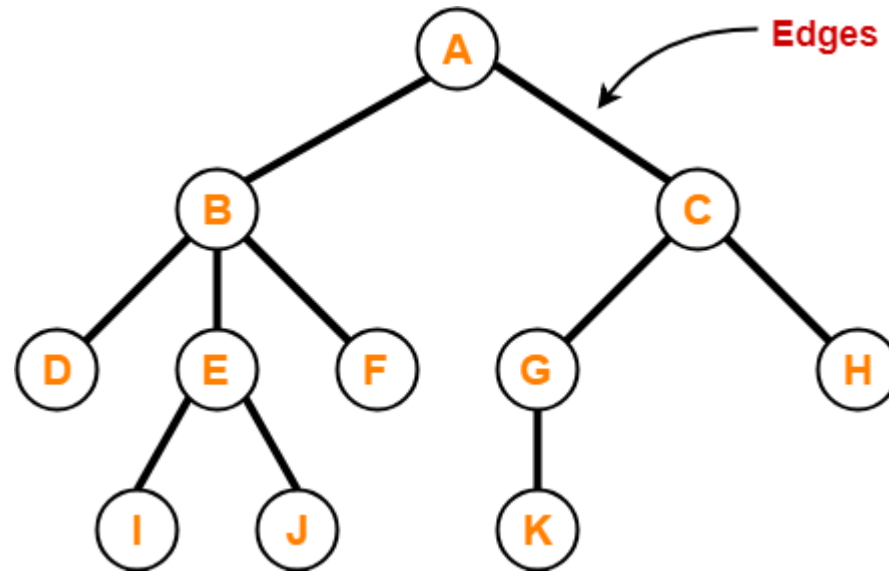


Tree

- 2. Edge-

-

- The connecting link between any two nodes is called as an **edge**.
- In a tree with n number of nodes, there are exactly $(n-1)$ number of edges.

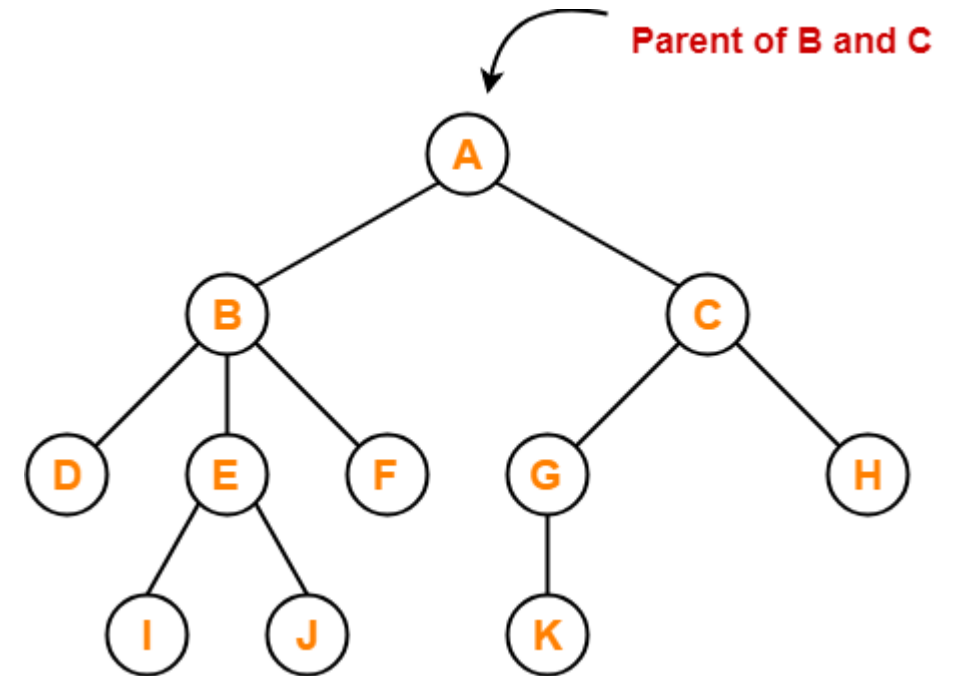


Tree

• 3. Parent-

- The node which has a branch from it to any other node is called as a **parent node**.
- In other words, the node which has one or more children is called as a parent node.
- In a tree, a parent node can have any number of child nodes.

- Here,
- Node A is the parent of nodes B and C
- Node B is the parent of nodes D, E and F
- Node C is the parent of nodes G and H
- Node E is the parent of nodes I and J
- Node G is the parent of node K



Tree

- 4. Child-

-

- The node which is a descendant of some node is called as a **child node**.

- All the nodes except root node are child nodes.

- Here,

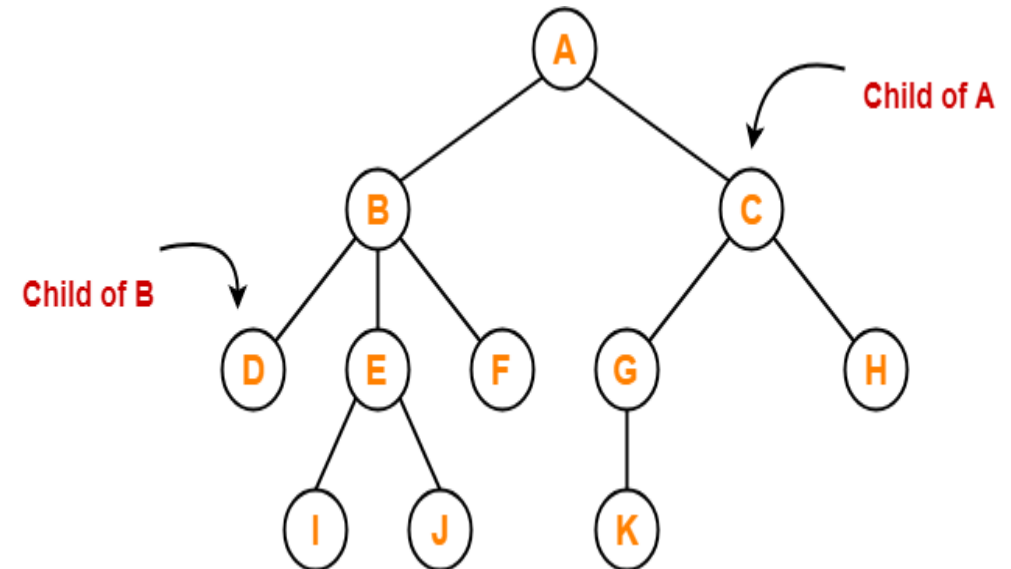
- Nodes B and C are the children of node A

- Nodes D, E and F are the children of node B

- Nodes G and H are the children of node C

- Nodes I and J are the children of node E

- Node K is the child of node G



Tree

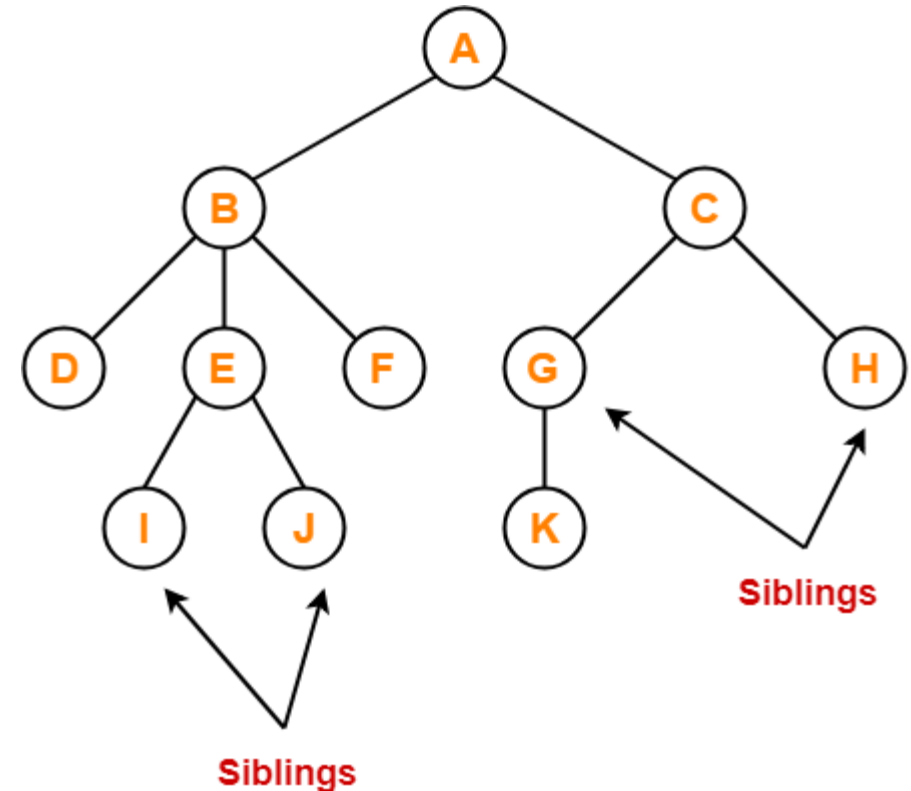
- 5. Siblings-

-

- Nodes which belong to the same parent are called as **siblings**.

- In other words, nodes with the same parent are sibling nodes.

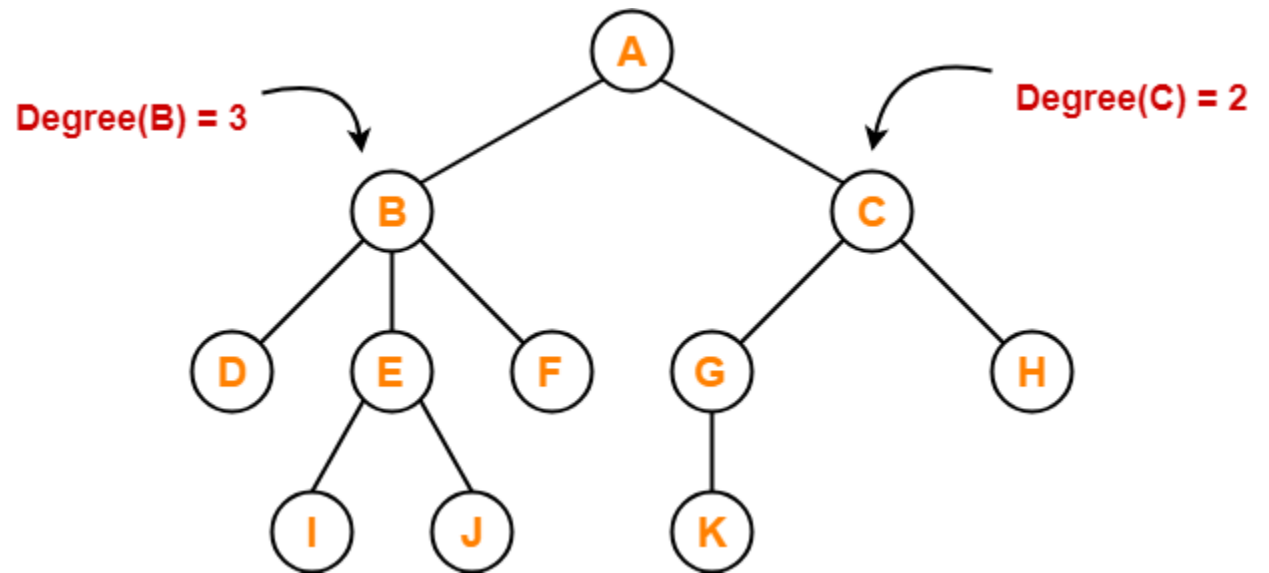
- Nodes B and C are siblings
- Nodes D, E and F are siblings
- Nodes G and H are siblings
- Nodes I and J are siblings



Tree

- **6. Degree-**

- **Degree of a node** is the total number of children of that node.
- **Degree of a tree** is the highest degree of a node among all the nodes in the tree.
- Here,
- Degree of node A = 2
- Degree of node B = 3
- Degree of node C = 2
- Degree of node D = 0
- Degree of node E = 2
- Degree of node F = 0
- Degree of node G = 1
- Degree of node H = 0
- Degree of node I = 0
- Degree of node J = 0
- Degree of node K = 0

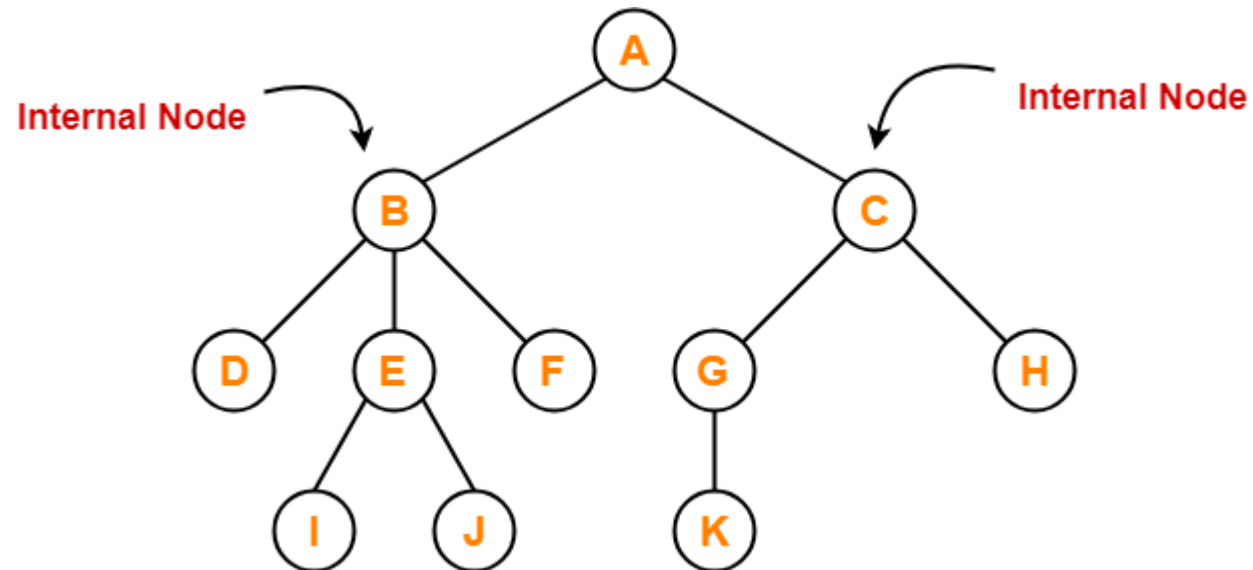


Tree

- **7. Internal Node-**

-

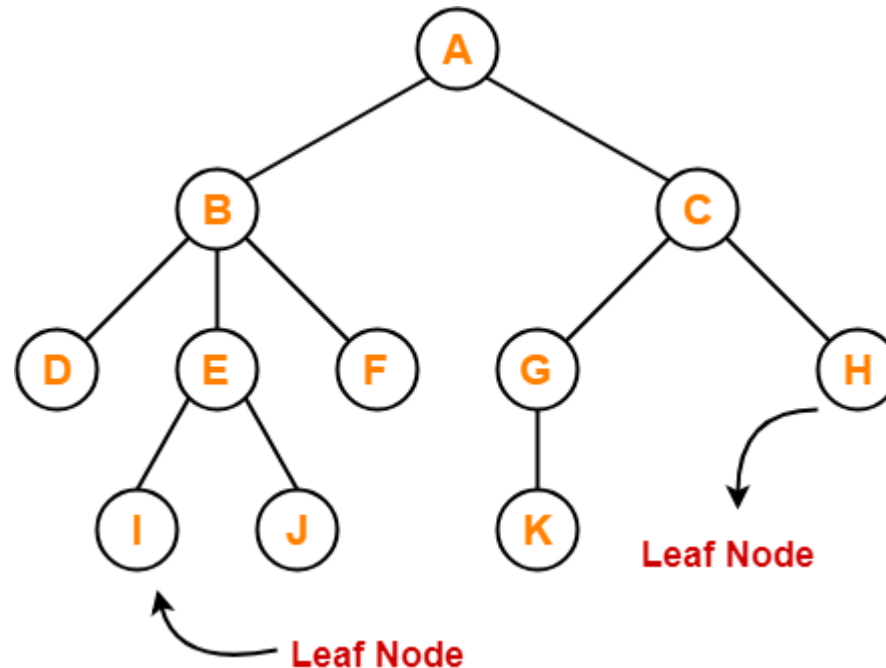
- The node which has at least one child is called as an **internal node**.
- Internal nodes are also called as **non-terminal nodes**.
- Every non-leaf node is an internal node.
- Here, nodes A, B, C, E and G are internal nodes.



Tree

- **8. Leaf Node-**

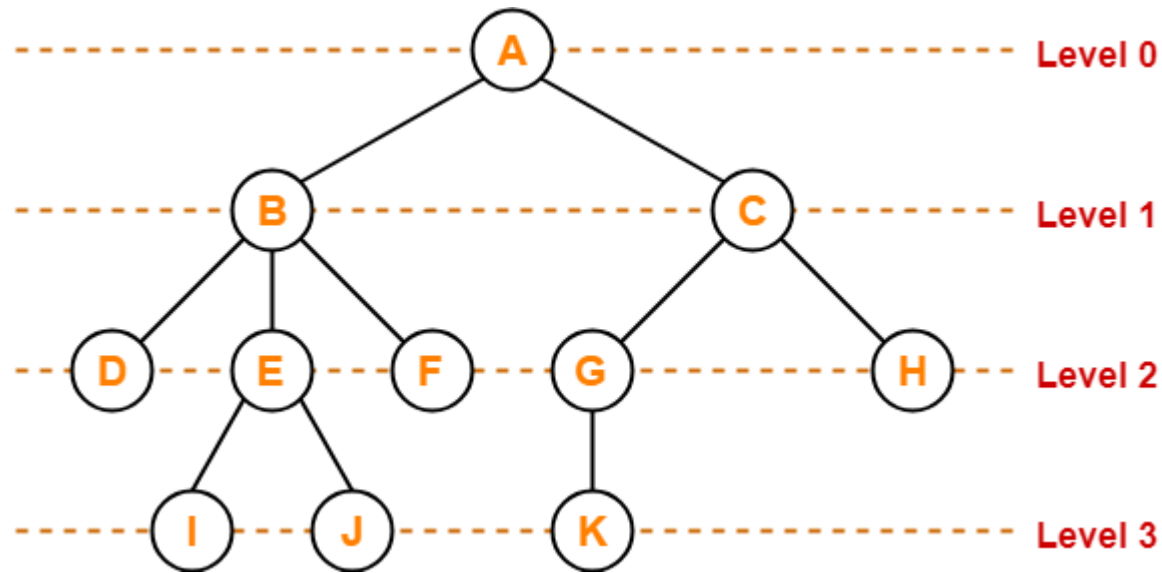
- The node which does not have any child is called as a **leaf node**.
- Leaf nodes are also called as **external nodes** or **terminal nodes**.
- Here, nodes D, I, J, F, K and H are leaf nodes.



Tree

- 9. Level-

- In a tree, each step from top to bottom is called as **level of a tree**.
- The level count starts with 0 and increments by 1 at each level or step.

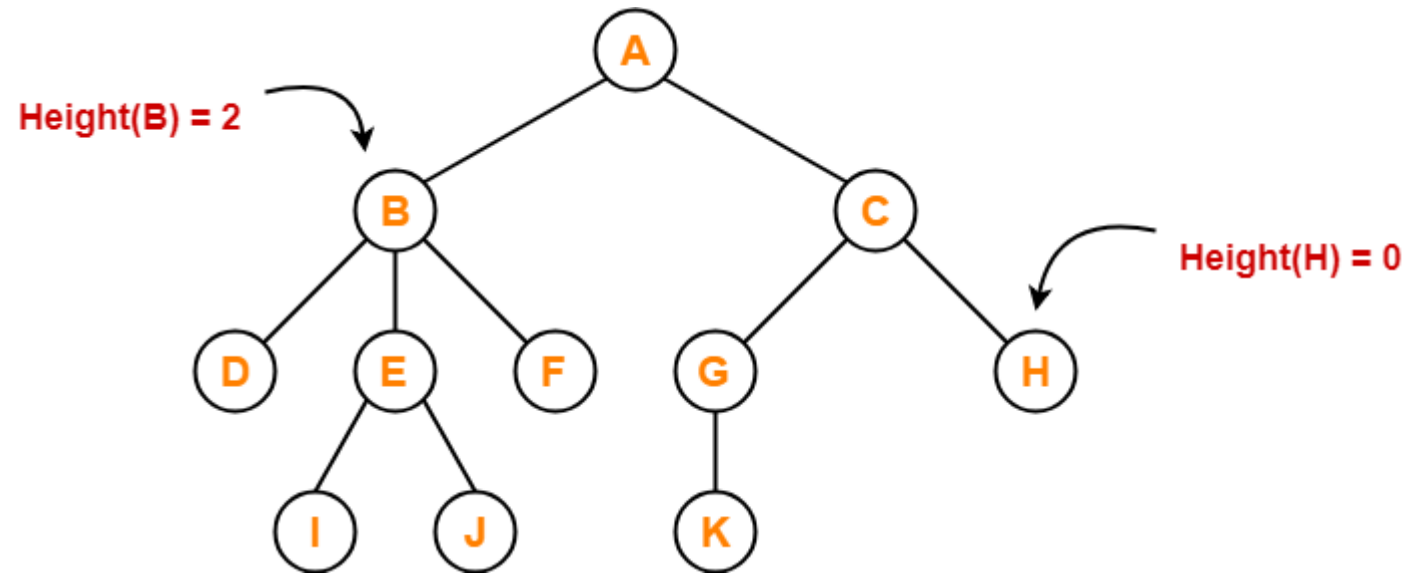


Tree

- **10. Height-**

- Total number of edges that lies on the longest path from any leaf node to a particular node is called as **height of that node**.
- **Height of a tree** is the height of root node.
- Height of all leaf nodes = 0
- Here,

- Height of node A = 3
- Height of node B = 2
- Height of node C = 2
- Height of node D = 0
- Height of node E = 1
- Height of node F = 0
- Height of node G = 1
- Height of node H = 0
- Height of node I = 0
- Height of node J = 0 Height of node K = 0



Tree

- **11. Depth-**

- Total number of edges from root node to a particular node is called as **depth of that node**.
- **Depth of a tree** is the total number of edges from root node to a leaf node in the longest path.
- Depth of the root node = 0
- The terms “level” and “depth” are used interchangeably.

- Here,

- Depth of node A = 0

- Depth of node B = 1

- Depth of node C = 1

- Depth of node D = 2

- Depth of node E = 2

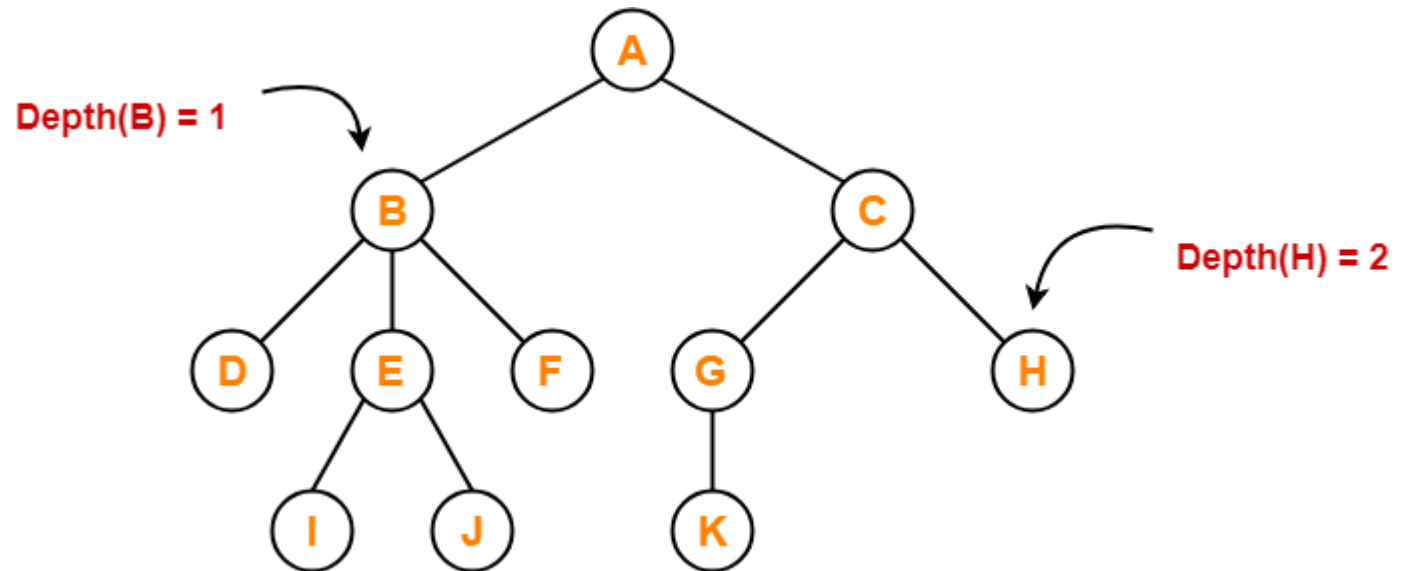
- Depth of node F = 2

- Depth of node G = 2

- Depth of node H = 2

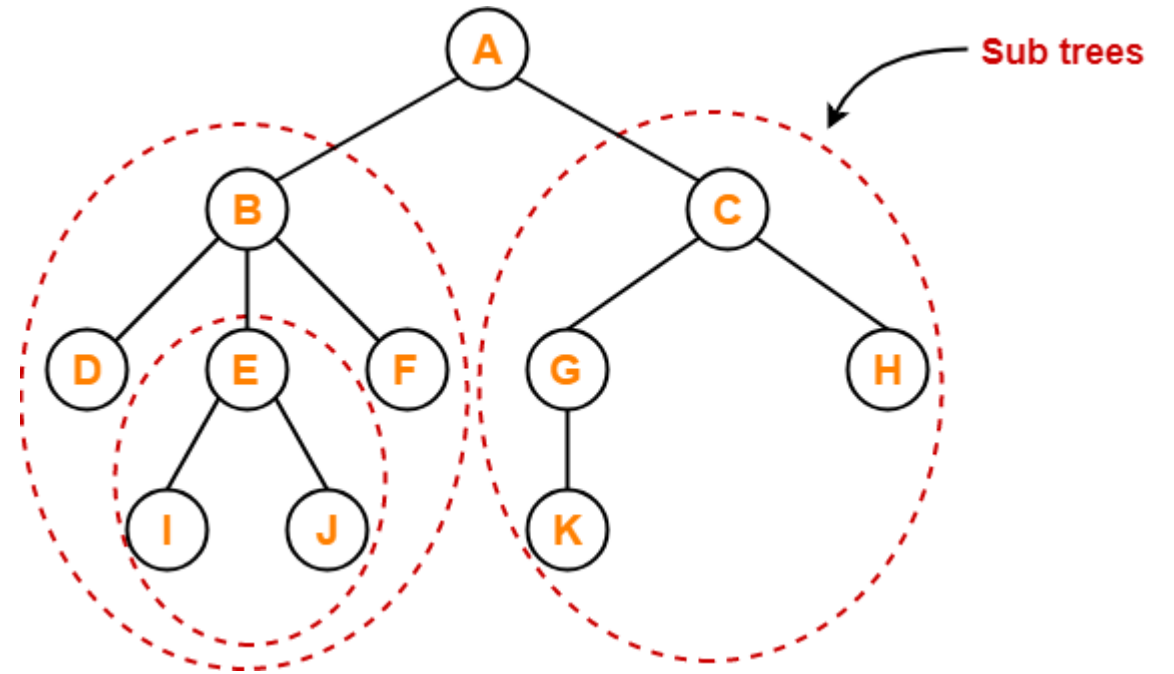
- Depth of node I = 3

- Depth of node J = 3 Depth of node K = 3



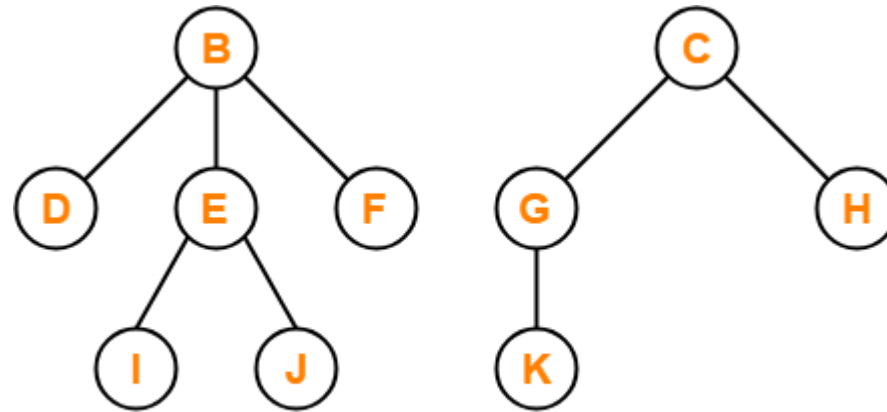
Tree

- 12. Subtree-
-
- In a tree, each child from a node forms a **subtree** recursively.
- Every child node forms a subtree on its parent node.



Tree

- **13. Forest-**
- A forest is a set of disjoint trees.

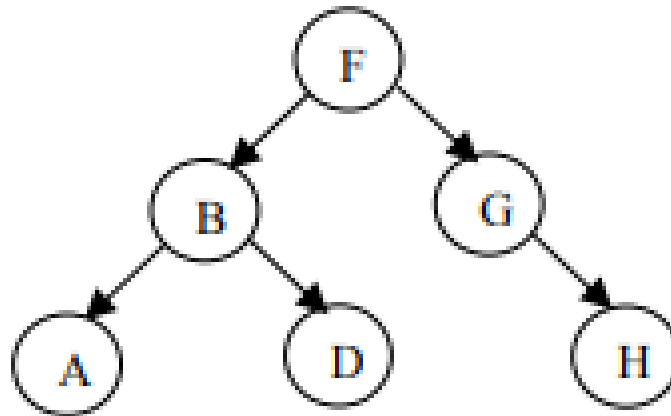


Forest

Tree

- **Outdegree:** Total number of leaving vertices is known as outdegree.
- **Indegree:** Total number of entering vertices is known as indegree.
- **Total degree:** The summation of indegree and outdegree is known as total degree.

E.g.



Indegree of a node B is 1

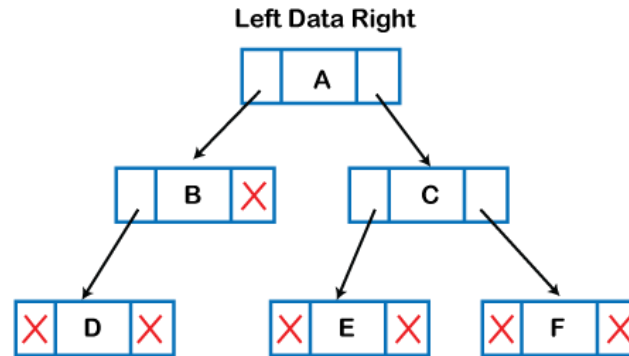
Outdegree of a node B is 2

Total degree of a node B is 3

Tree

- **Implementation of Tree**

- The tree data structure can be created by creating the nodes dynamically with the help of the pointers. The tree in the memory can be represented as shown below:



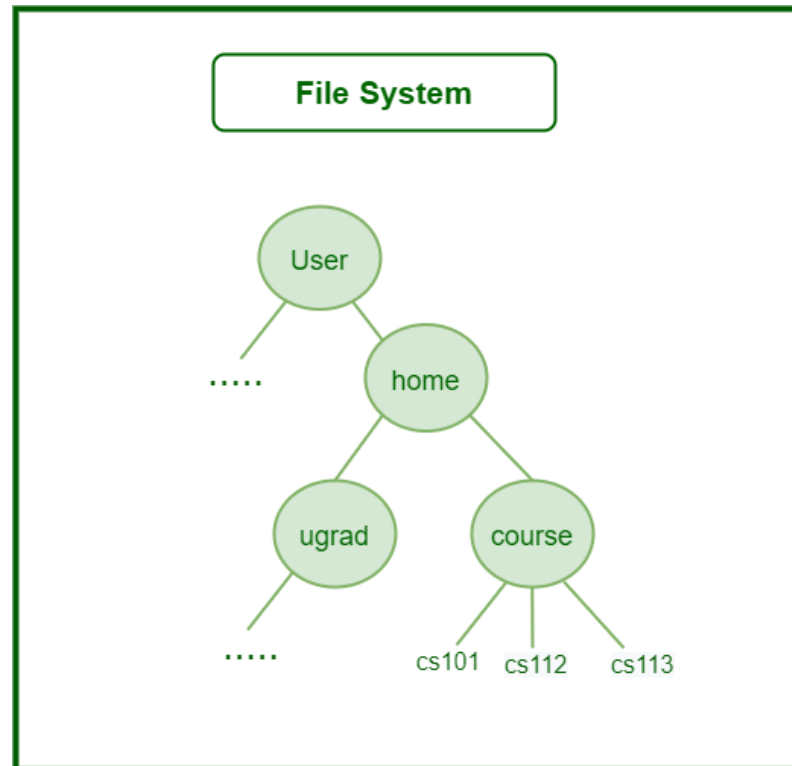
- The above figure shows the representation of the tree data structure in the memory. In the above structure, the node contains three fields. The second field stores the data; the first field stores the address of the left child, and the third field stores the address of the right child.
- In programming, the structure of a node can be defined as:
- The above structure can only be defined for the binary trees because the binary tree can have utmost two children, and generic trees can have more than two children. The structure of the node for generic trees would be different as compared to the binary tree.

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
}
```

Tree

- Applications

- One reason to use trees might be because you want to store information that naturally forms a hierarchy. For example, the file system on a computer:



Applications

- Tree data structures are widely used in computer science for organizing hierarchical data efficiently. Some important applications are:
- **1. File Systems**
- Trees are used to represent directory structures in operating systems. Folders (directories) act as parent nodes, and files/subfolders act as child nodes.
- Example: Windows/Linux file systems
- **2. Database Indexing**
- Structures like B-Trees and B+ Trees are used in databases to enable fast searching, insertion, and deletion of records.
- Used in systems like MySQL, Oracle
- **3. Expression Trees**
- Used in compilers to evaluate arithmetic expressions. Each internal node represents an operator and leaf nodes represent operands.
- Example: converting infix → postfix and evaluating expressions
- **4. Hierarchical Data Representation**
- Trees are ideal for representing hierarchical relationships such as organizational structures, family trees, etc.
- Used where data has **parent-child relationship**
- Example:
- Organization structure (CEO → Manager → Employee)
- XML/HTML DOM structure

Applications

- **5. Searching and Sorting**
- Binary Search Trees (BST) allow efficient searching, insertion, and deletion operations in $O(\log n)$ time (on average).
- **6. Routing Algorithms**
- Trees are used in networking (like spanning trees) to find optimal paths and avoid cycles in networks.
- Example: spanning trees in networks (like minimum spanning tree)
- **7. Decision Making (AI)**
- Decision trees are used in machine learning for classification and prediction problems.
- **8. Syntax Trees (Parsing)**
- Compilers use syntax trees (parse trees) to analyze the grammatical structure of programming languages. That's why it is used in **syntax analysis in compilers**
- **9. Heap (Priority Queue)**
- Heap trees are used to implement priority queues where the highest (or lowest) priority element is accessed quickly.
- Used in:
- CPU scheduling
- Dijkstra's shortest path algorithm
- **10. XML/HTML Parsing**
- Tree structures are used to represent web pages (DOM tree), where elements are nested within each other.

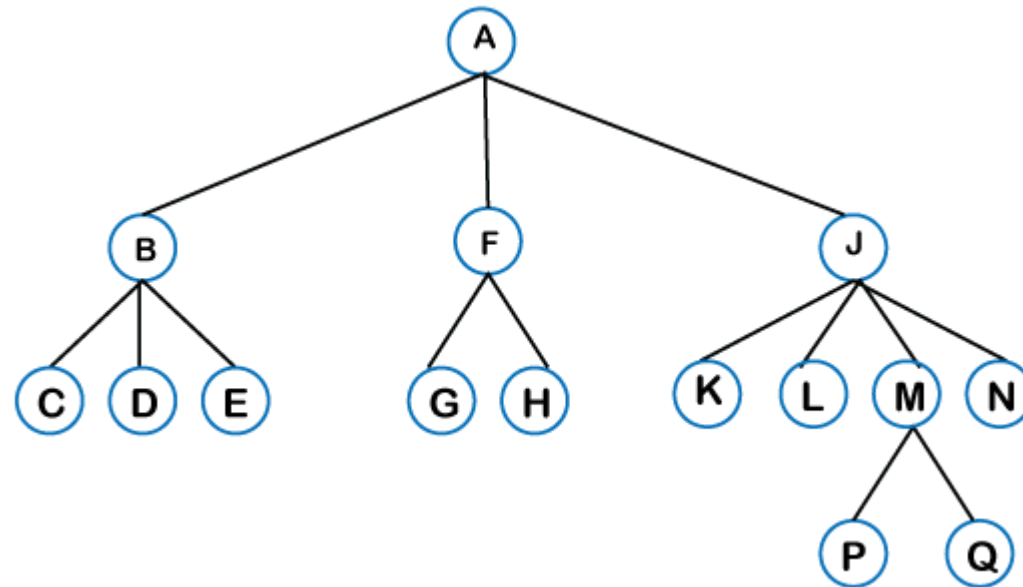
Tree

- **Types of Tree**
- General tree
- Binary Tree
- Binary Search Tree
- AVL Tree
- B-Tree

Tree

- **General tree**

- **General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum n number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain). The topmost node in a general tree is known as a root node. The children of the parent node are known as *subtrees*.



Tree

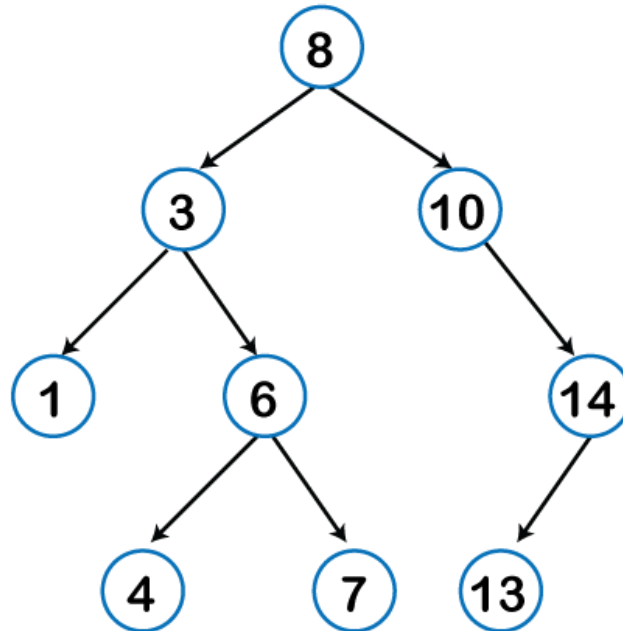
- **General tree**

- There can be ***n*** number of subtrees in a general tree. In the general tree, the subtrees are unordered as the nodes in the subtree cannot be ordered.

Every non-empty tree has a downward edge, and these edges are connected to the nodes known as ***child nodes***. The root node is labeled with level 0. The nodes that have the same parent are known as ***siblings***.

Tree

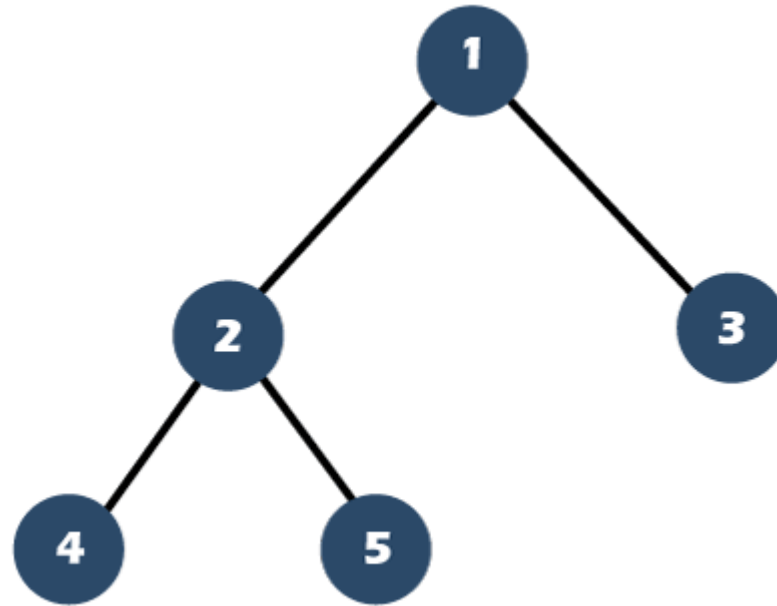
- **Binary tree:**
- Here, binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.



Tree

- **Full Binary tree:**

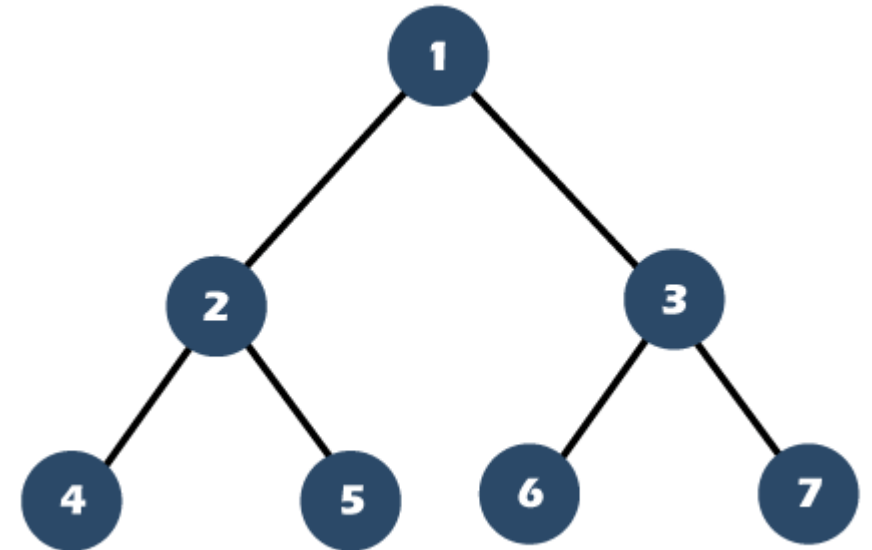
- A full binary tree can be defined as a binary tree in which all the nodes have 0 or two children. In other words, the full binary tree can be defined as a binary tree in which all the nodes have two children except the leaf nodes.
- Below is the full binary tree.



Tree

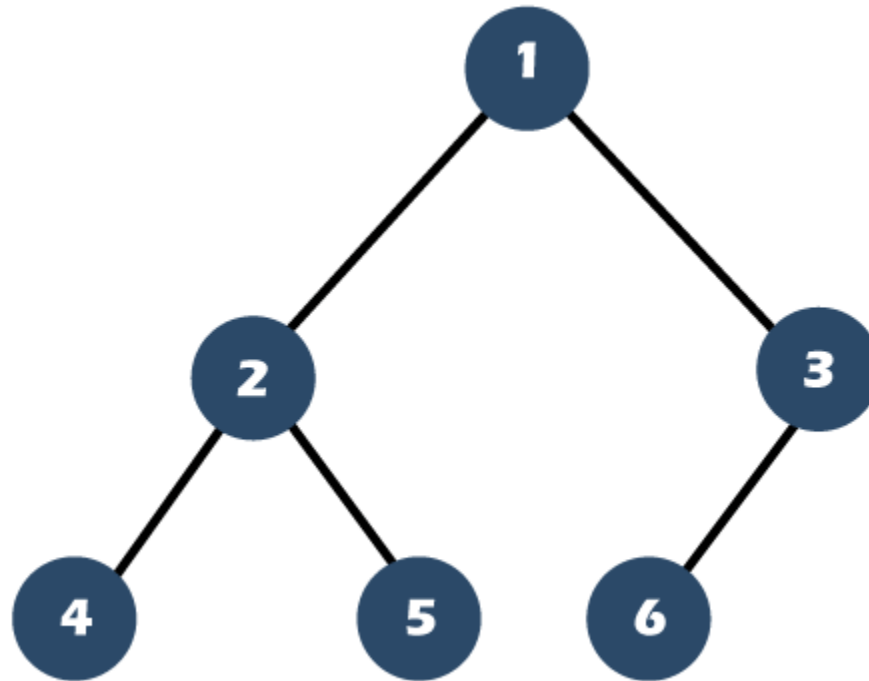
- **complete binary tree**

- A binary tree is said to be a complete binary tree when all the levels are completely filled except the last level, which is filled from the left.
- The below tree is a complete binary tree:
- The complete binary tree is similar to the full binary tree except for the two differences which are given below:
- The filling of the leaf node must start from the leftmost side.
- It is not mandatory that the last leaf node must have the right sibling.

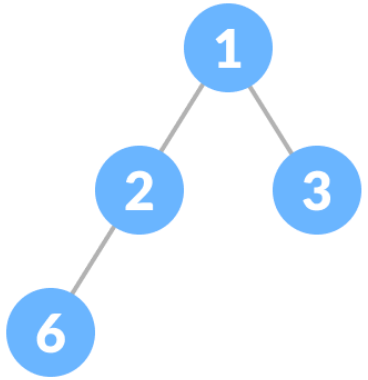


Tree

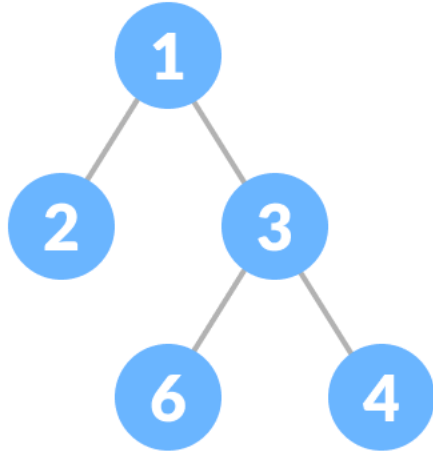
- complete binary tree
- **Let's understand the above points through an example:**
- The below tree is a complete binary tree, but not a full binary tree as node 6 does not have its right sibling.



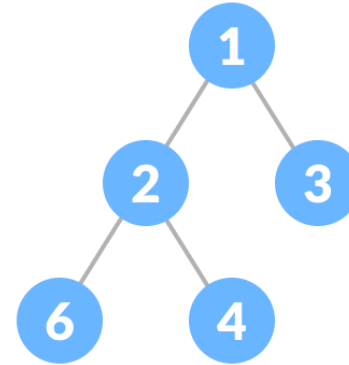
Tree



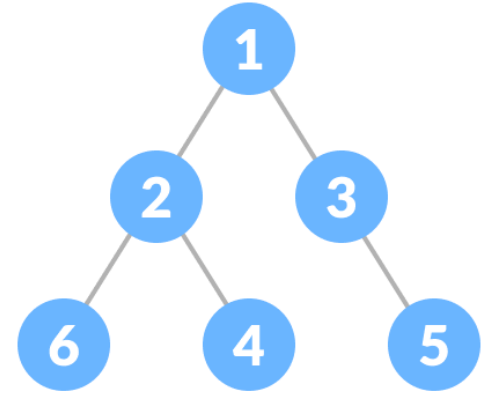
- ✗ Full Binary Tree
- ✓ Complete Binary Tree



- ✓ Full Binary Tree
- ✗ Complete Binary Tree



- ✓ Full Binary Tree
- ✓ Complete Binary Tree



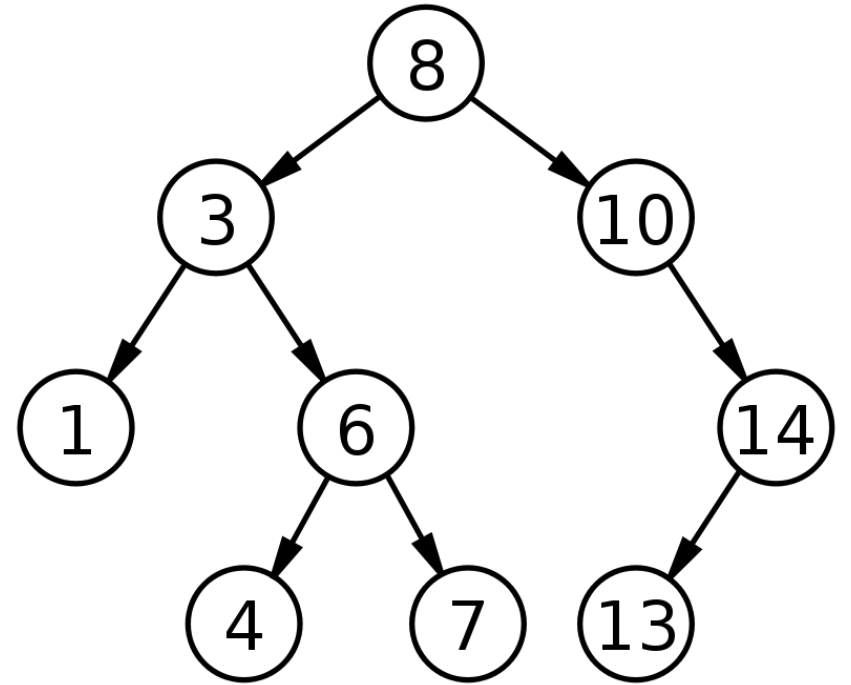
- ✗ Full Binary Tree
- ✗ Complete Binary Tree

Tree

- **Binary search tree:**
- As the name suggests, a Binary Search Tree or BST is a tree that is used in sorting, retrieving and searching data. It is also a type of non-linear data structure in which the nodes are arranged in a particular order. Hence, it is also called as “**Ordered Binary Tree**”.
- It has the following properties:
 - The left subtree of a node has nodes which are only lesser than that node's key.
 - The right subtree of a node has nodes which are only greater than that node's key.
 - Each node has distinct keys and duplicates are not allowed in Binary Search Tree.
 - The left and right subtree must also be a binary search tree.

Tree

In the above figure, we see that the value of the root node is 8. With further scrutiny, it is observed that all the values in the left subtree are lesser than the value of the root node and all the values in the right subtree have values that are greater than the root node. Furthermore, it is noted that each value in the Binary Search Tree is unique and there are no duplicates.



Binary Tree Representations

A binary tree data structure is represented using two methods. Those methods are as follows...

Array Representation

Linked List Representation

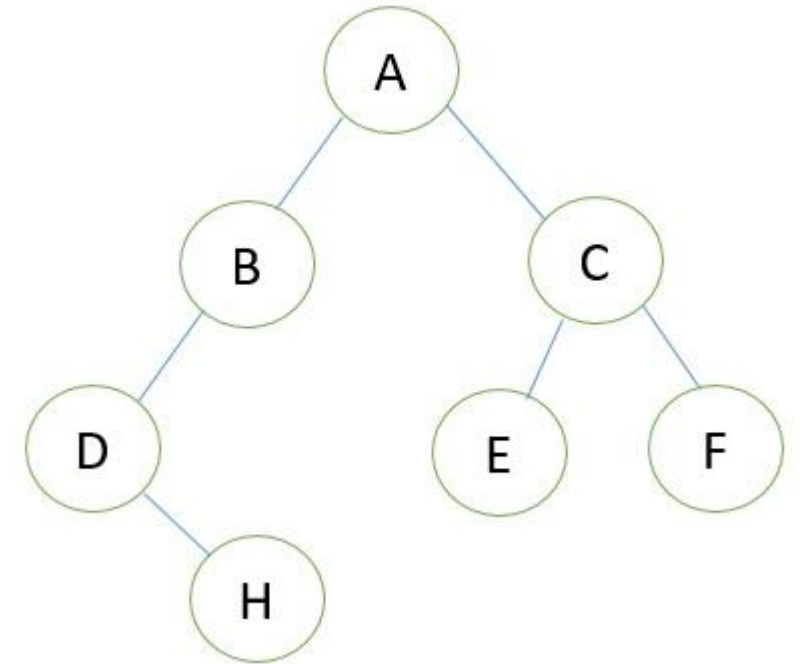
Consider the following binary tree...

1. Array Representation of Binary Tree

In array representation of a binary tree, we use one-dimensional array (1-D Array)

Consider the above example of a binary tree and it is represented as follows...

Its sequential representation is as follow:



Binary Tree Representations

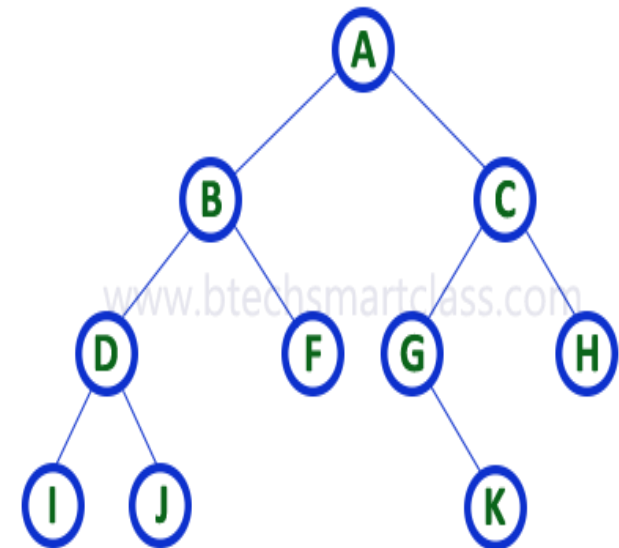
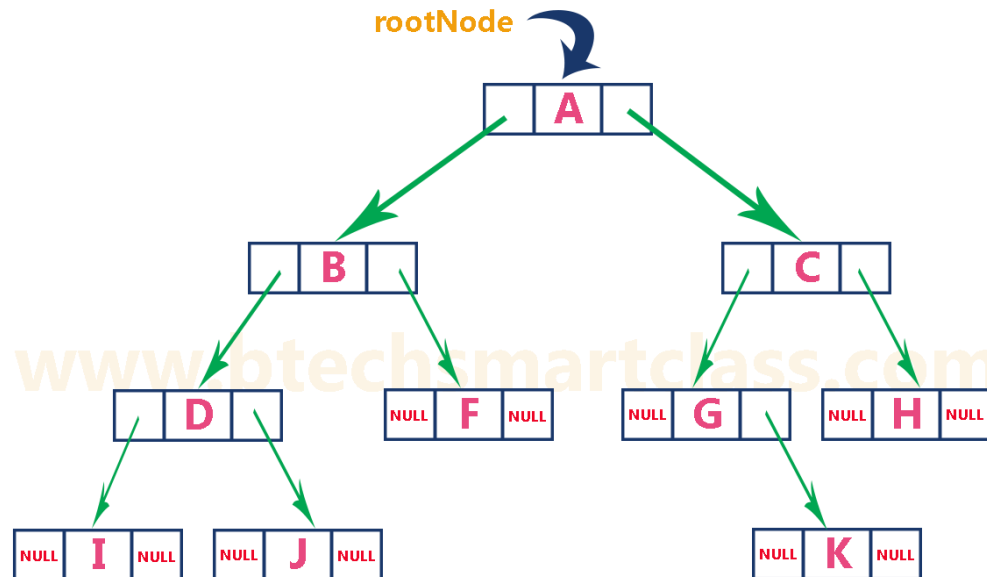
2. Linked List Representation of Binary Tree

We use a double linked list to represent a binary tree. In a double linked list, every node consists of three fields. First field for storing left child address, second for storing actual data and third for storing right child address.

In this linked list representation, a node has the following structure...



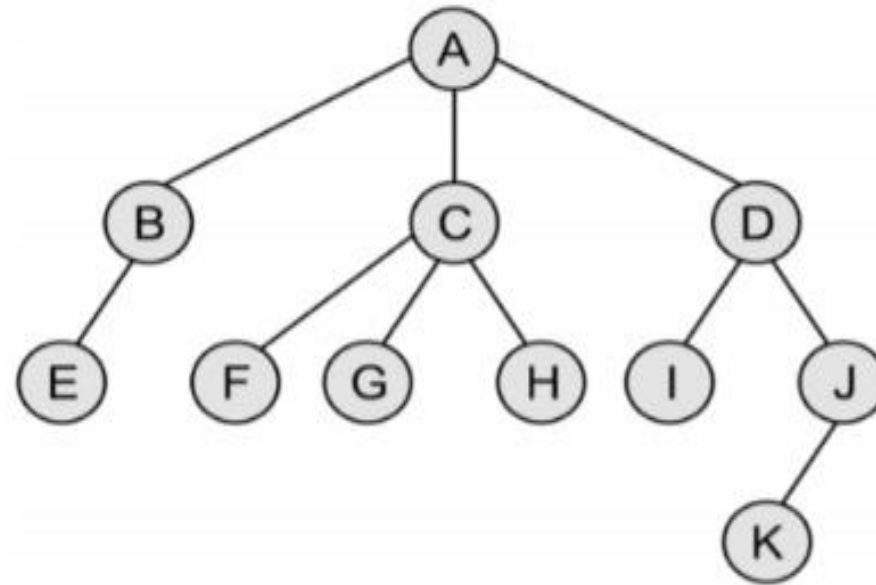
The above example of the binary tree represented using Linked list representation is shown as follows...



General tree to binary tree

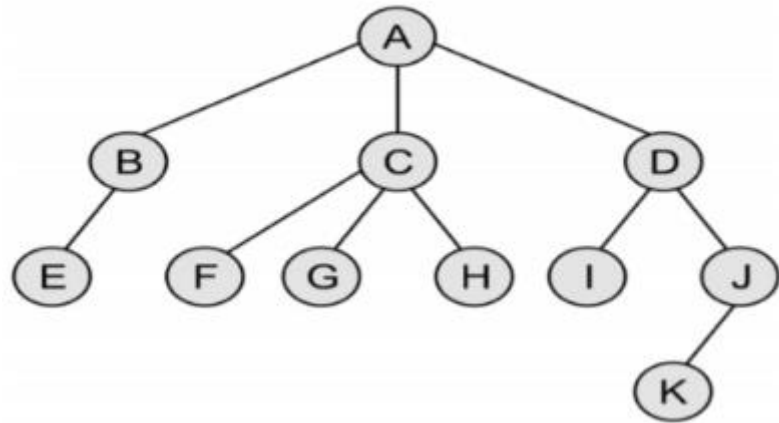
Following are the rules to convert a Generic(N-array Tree) to Binary Tree:

- The root of the Binary Tree is the Root of the Generic Tree.
 - The left child of a node in the Generic Tree is the Left child of that node in the Binary Tree.
 - The right sibling of any node in the Generic Tree is the Right child of that node in the Binary Tree.
- Convert the following Generic Tree to Binary Tree:



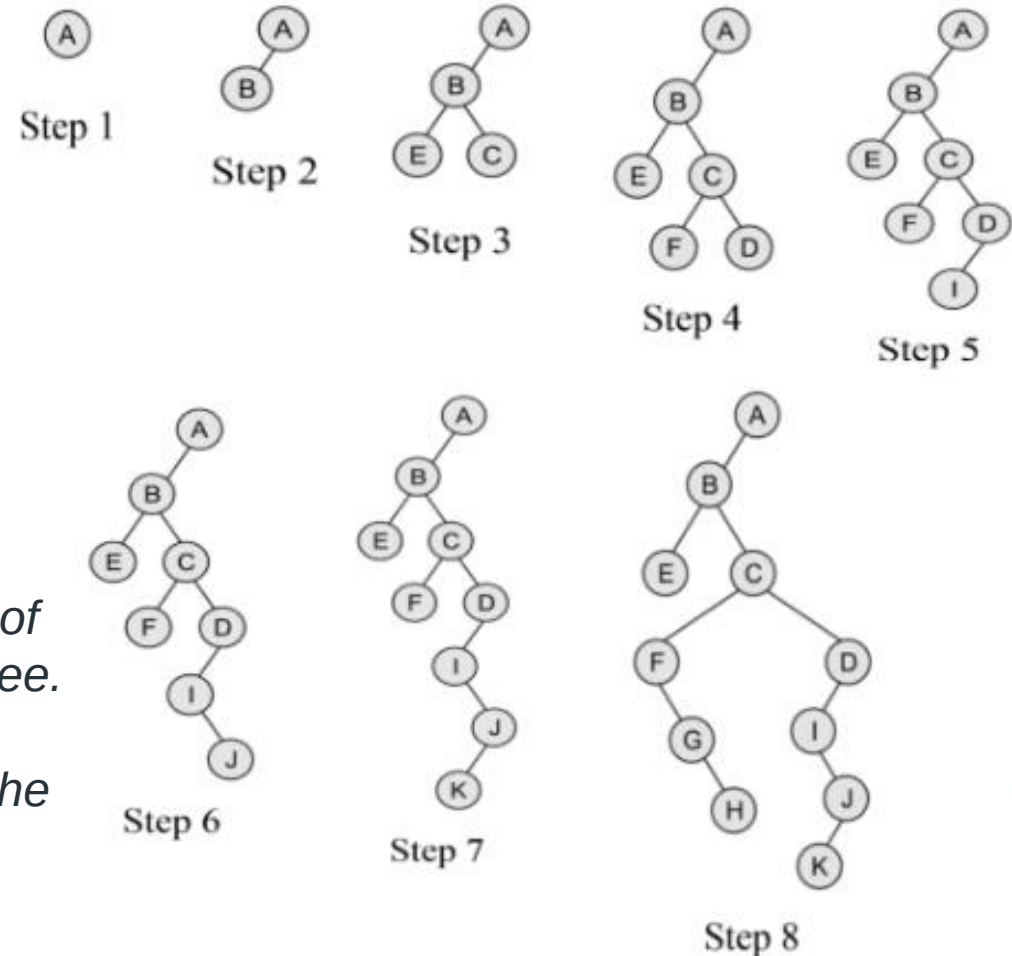
General tree to binary tree

- Convert the following Generic Tree to Binary Tree:



Note: If the parent node has only the right child in the general tree then it becomes the rightmost child node of the last node following the parent node in the binary tree. In the above example, if node **B** has the right child node **L** then in binary tree representation **L** would be the right child of node **D**.

Below is the Binary Tree of the above Generic Tree:



General tree to binary tree

Below are the steps for the conversion of Generic Tree to Binary Tree:

As per the rules mentioned above, the root node of general tree **A** is the root node of the binary tree.

Now the leftmost child node of the root node in the general tree is **B** and it is the leftmost child node of the binary tree.

Now as **B** has **E** as its leftmost child node, so it is its leftmost child node in the binary tree whereas it has **C** as its rightmost sibling node so it is its right child node in the binary tree.

Now **C** has **F** as its leftmost child node and **D** as its rightmost sibling node, so they are its left and right child node in the binary tree respectively.

Now **D** has **I** as its leftmost child node which is its left child node in the binary tree but doesn't have any rightmost sibling node, so doesn't have any right child in the binary tree.

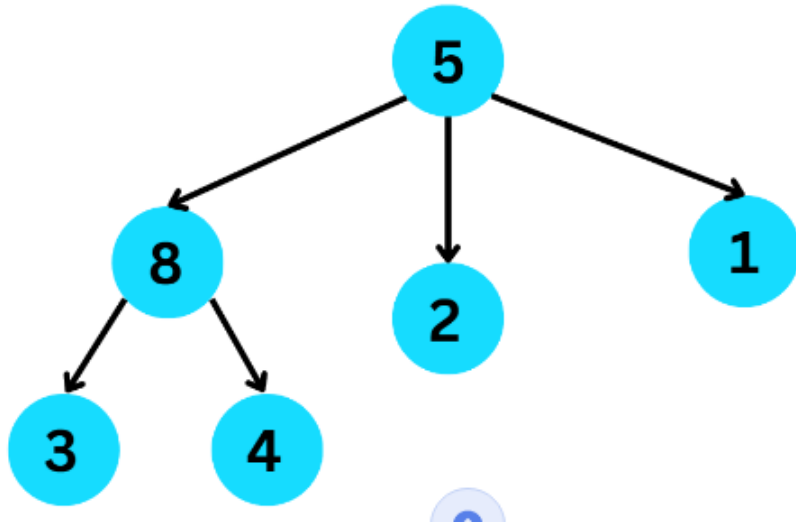
Now for **I**, **J** is its rightmost sibling node and so it is its right child node in the binary tree.

Similarly, for **J**, **K** is its leftmost child node and thus it is its left child node in the binary tree.

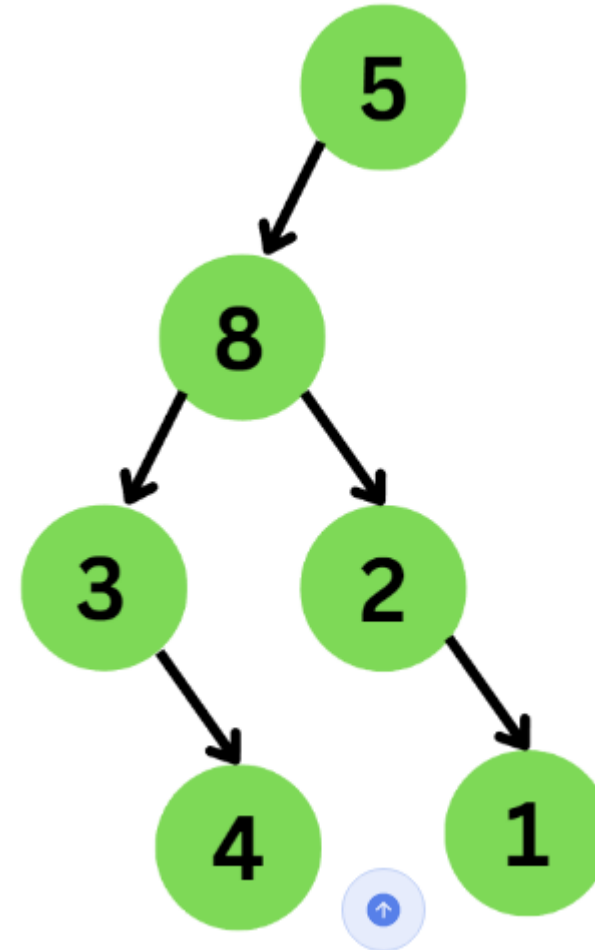
Now for **C**, **F** is its leftmost child node, which has **G** as its rightmost sibling node, which has **H** as its just right sibling node and thus they form their left, right, and right child node respectively.

General tree to binary tree

Generic tree -

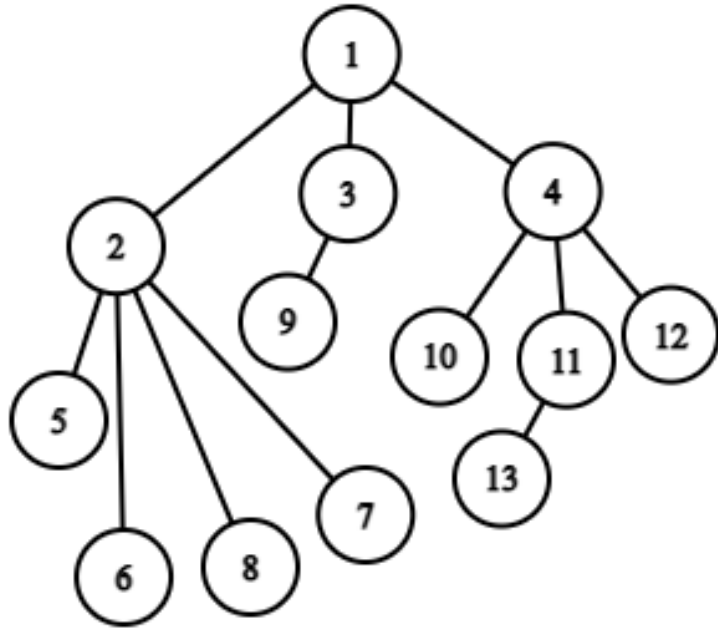


Converted Binary tree -

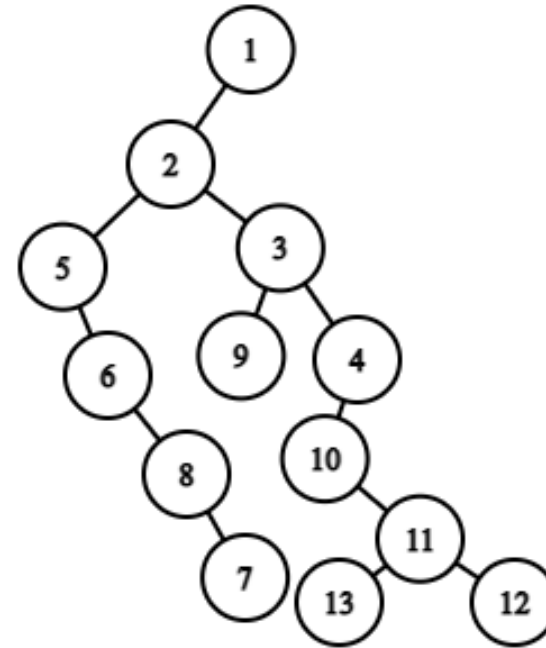


General tree to binary tree

Generic tree -

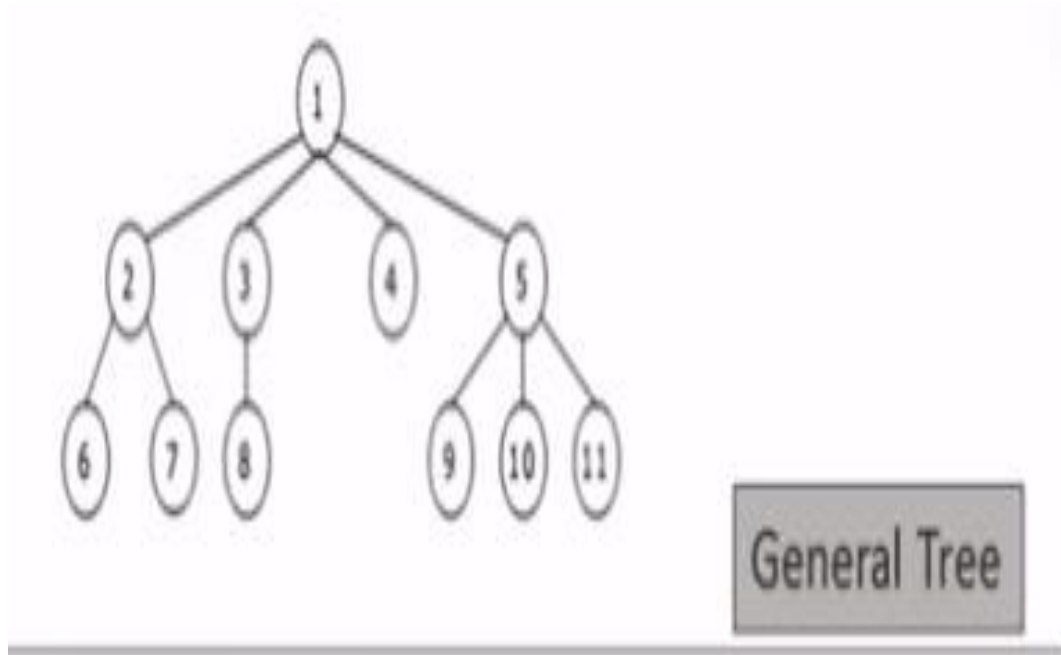


Converted Binary tree -

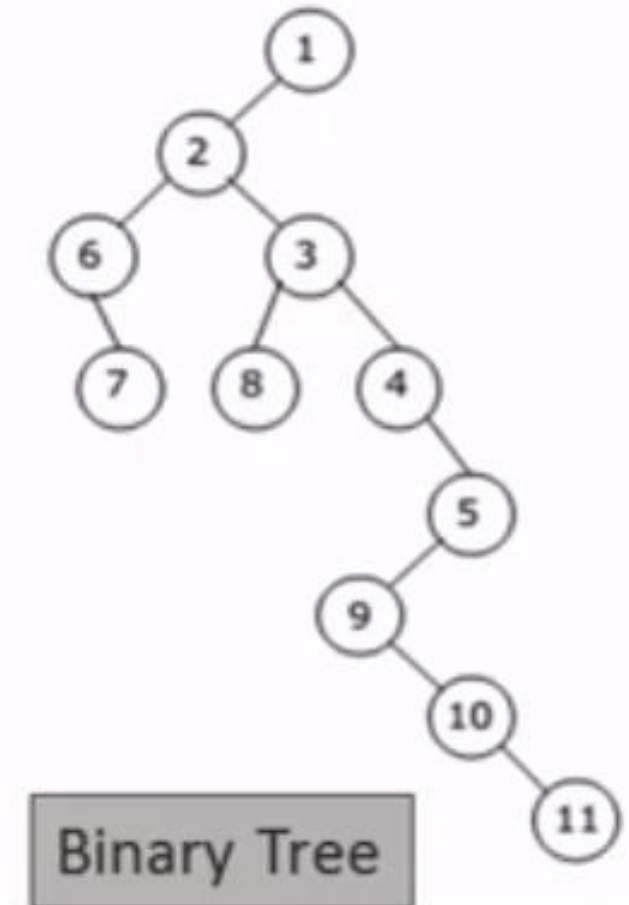


General tree to binary tree

Generic tree -

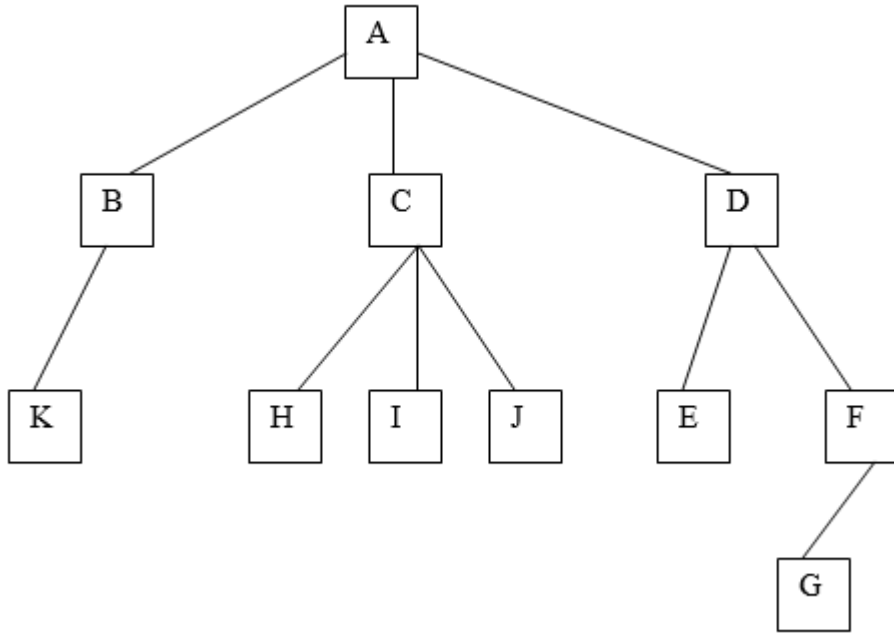


Converted Binary tree -

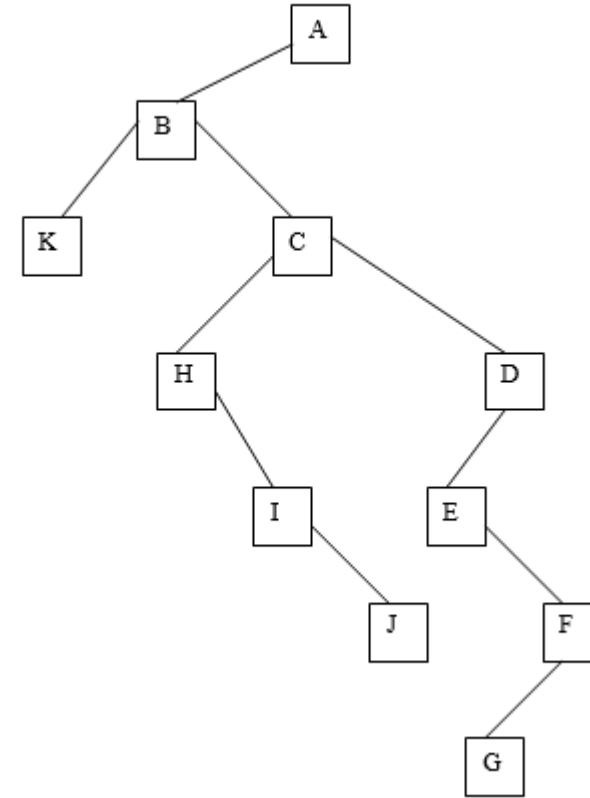


General tree to binary tree

Generic tree -



Converted Binary tree -



Binary search tree

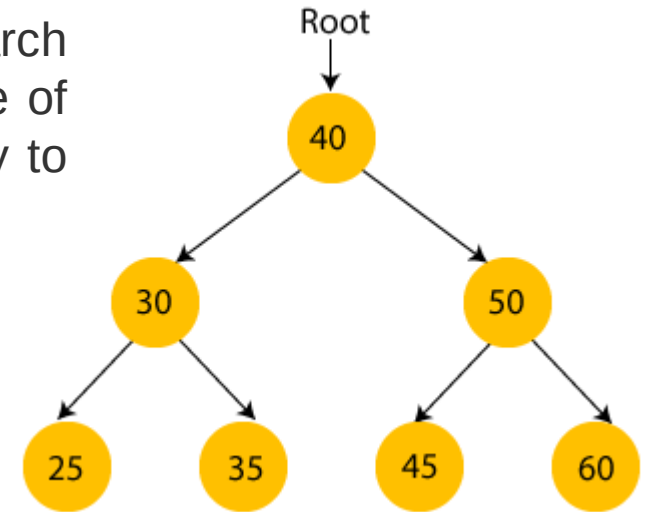
What is a Binary Search tree?

A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

Let's understand the concept of Binary search tree with an example.

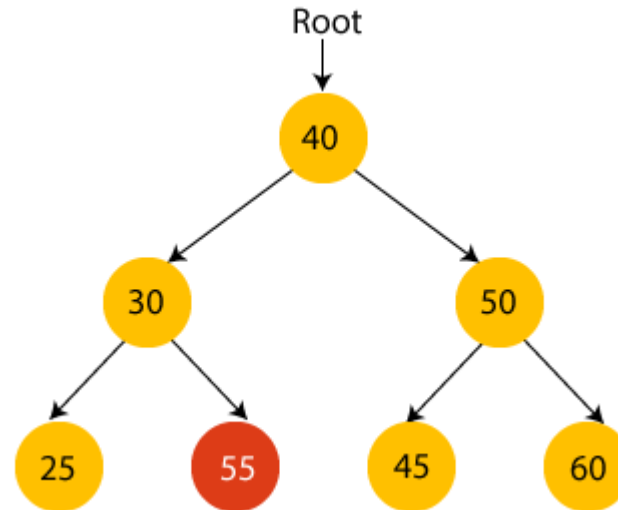
In the above figure, we can observe that the root node is 40, and all the nodes of the left subtree are smaller than the root node, and all the nodes of the right subtree are greater than the root node.

Similarly, we can see the left child of root node is greater than its left child and smaller than its right child. So, it also satisfies the property of binary search tree. Therefore, we can say that the tree in the above image is a binary search tree.



Binary search tree

Suppose if we change the value of node 35 to 55 in the above tree, check whether the tree will be binary search tree or not.



In the above tree, the value of root node is 40, which is greater than its left child 30 but smaller than right child of 30, i.e., 55. So, the above tree does not satisfy the property of Binary search tree. Therefore, the above tree is not a binary search tree.

Advantages of Binary search tree

- Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.
- As compared to array and linked lists, insertion and deletion operations are faster in BST.

Binary search tree

Example of creating a binary search tree

Suppose the data elements are - **45, 15, 79, 90, 10, 55, 12, 20, 50**

- First, we have to insert **45** into the tree as the root of the tree.
- Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.
- Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

Binary search tree

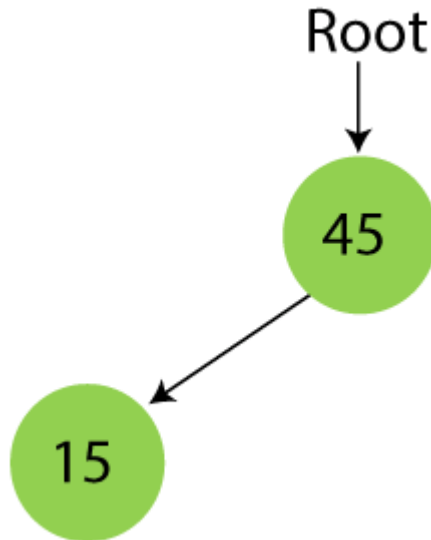
Example of creating a binary search tree - 45, 15, 79, 90, 10, 55, 12, 20, 50

Step 1 - Insert 45.



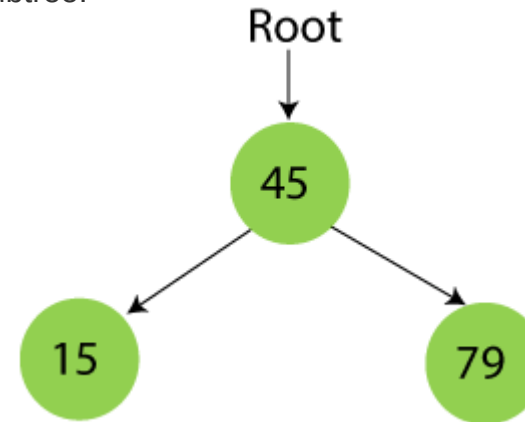
Step 2 - Insert 15.

As 15 is smaller than 45, so insert it as the root node of the left subtree.



Step 3 - Insert 79.

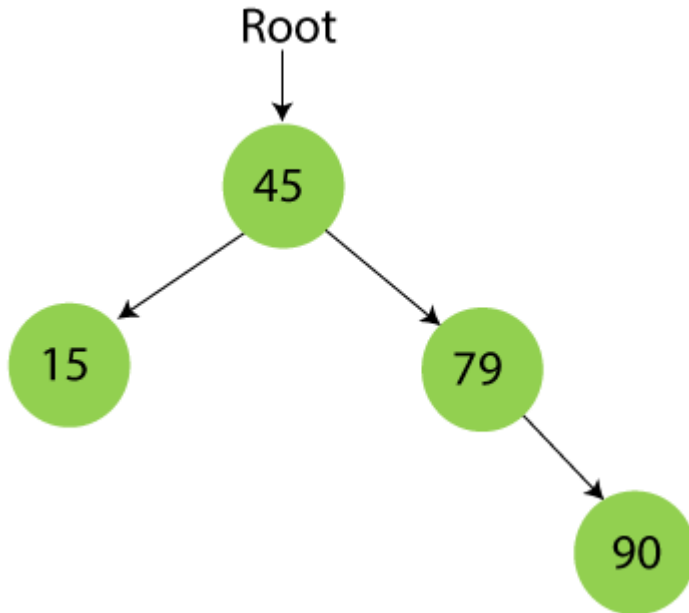
As 79 is greater than 45, so insert it as the root node of the right subtree.



Binary search tree

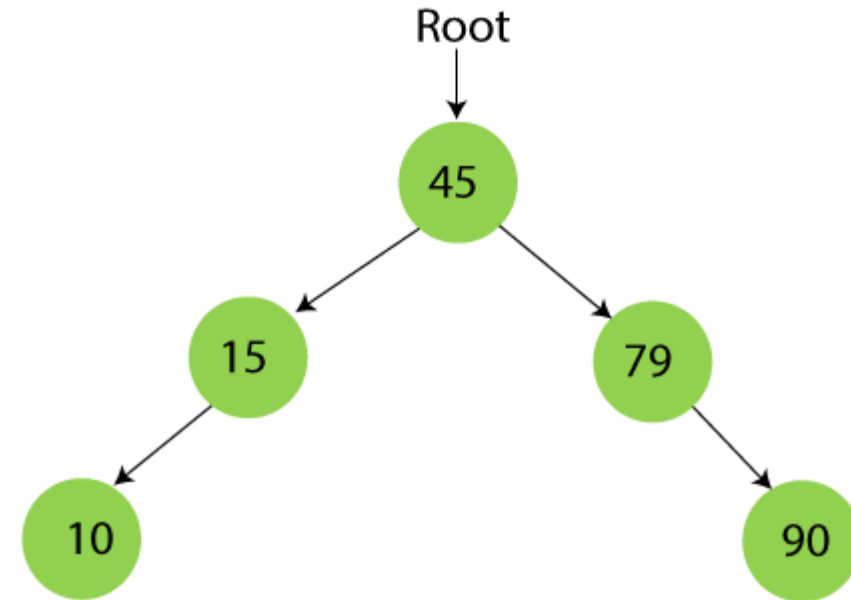
Step 4 - Insert 90.

90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.



Step 5 - Insert 10.

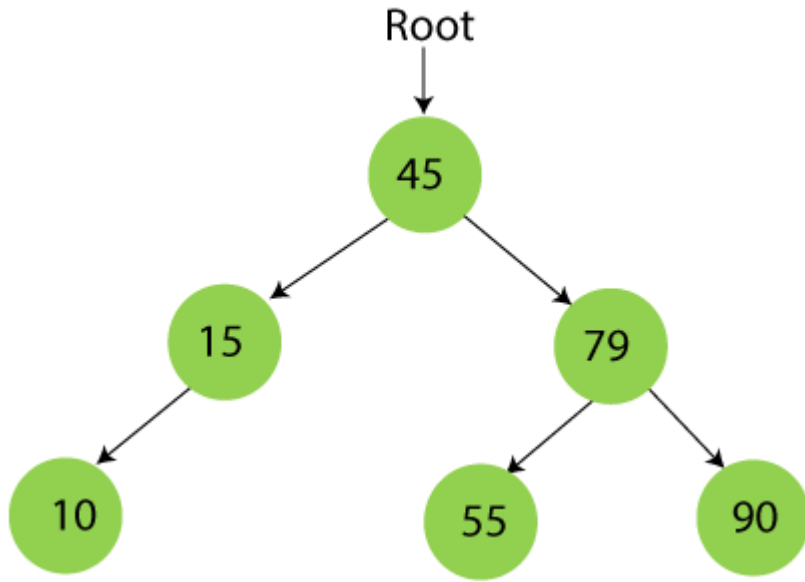
10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.



Binary search tree

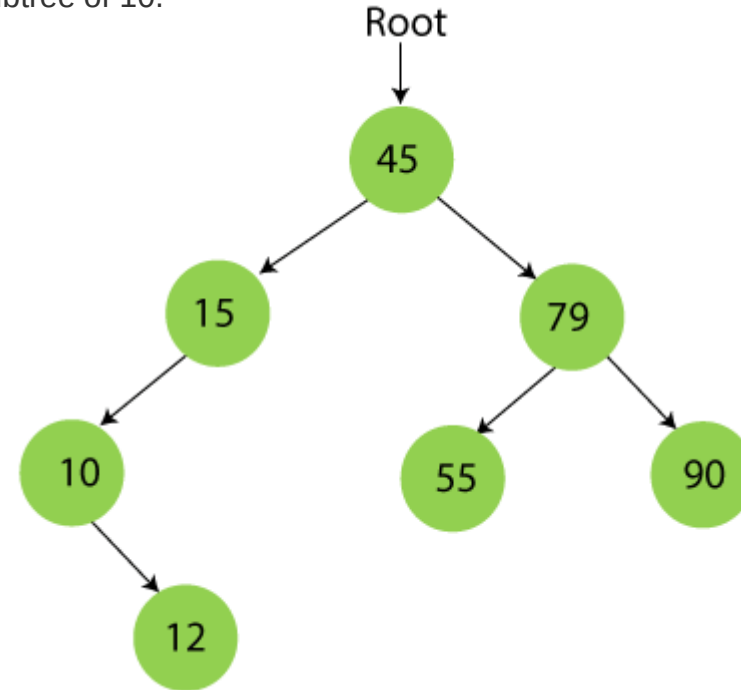
Step 6 - Insert 55.

55 is larger than 45 and smaller than 79, so it will be inserted as the left subtree of 79.



Step 7 - Insert 12.

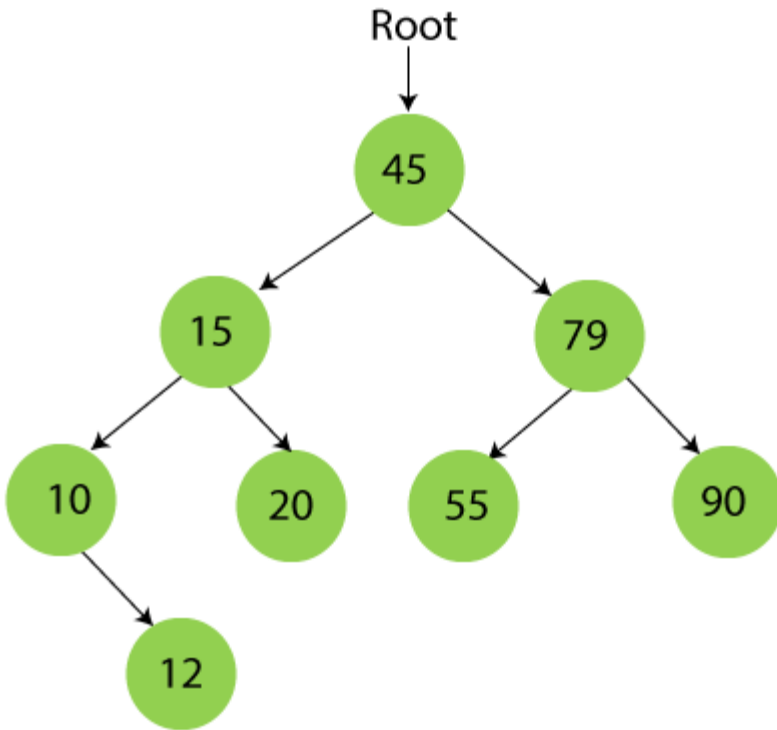
12 is smaller than 45 and 15 but greater than 10, so it will be inserted as the right subtree of 10.



Binary search tree

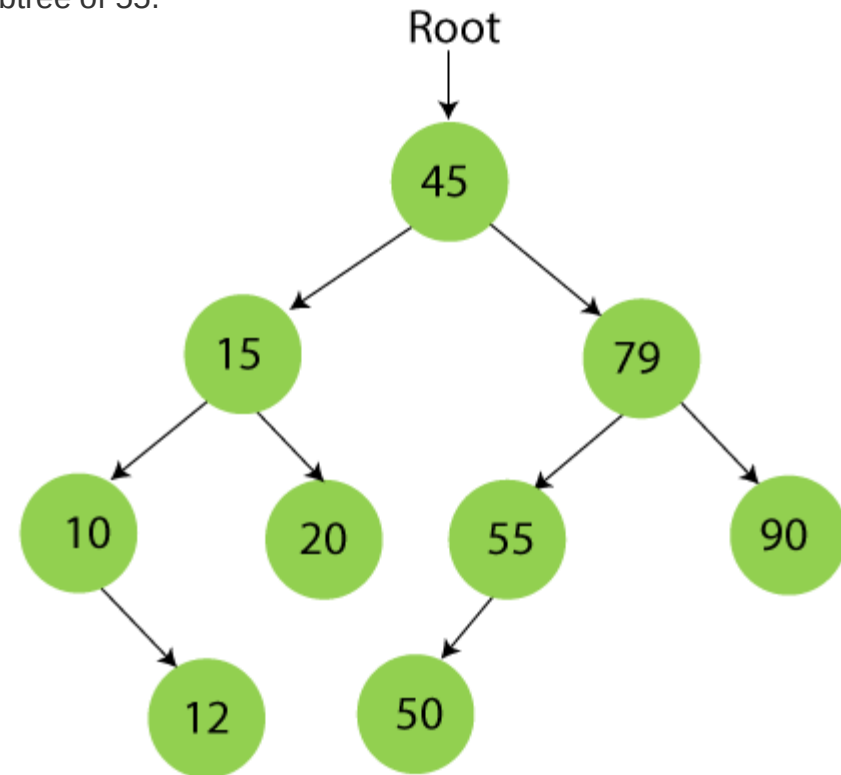
Step 8 - Insert 20.

20 is smaller than 45 but greater than 15, so it will be inserted as the right subtree of 15.



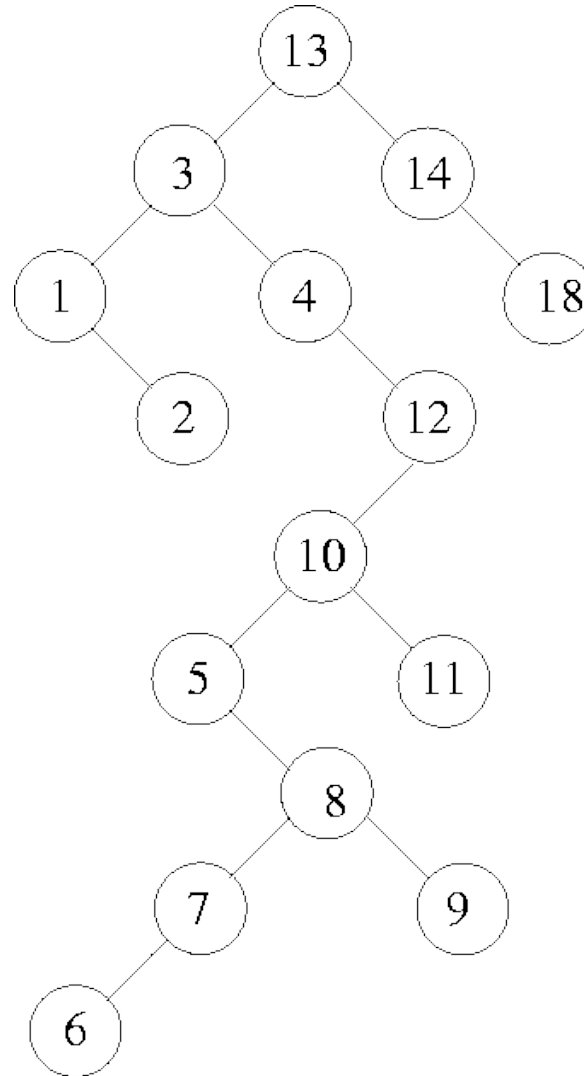
Step 9 - Insert 50.

50 is greater than 45 but smaller than 79 and 55. So, it will be inserted as a left subtree of 55.



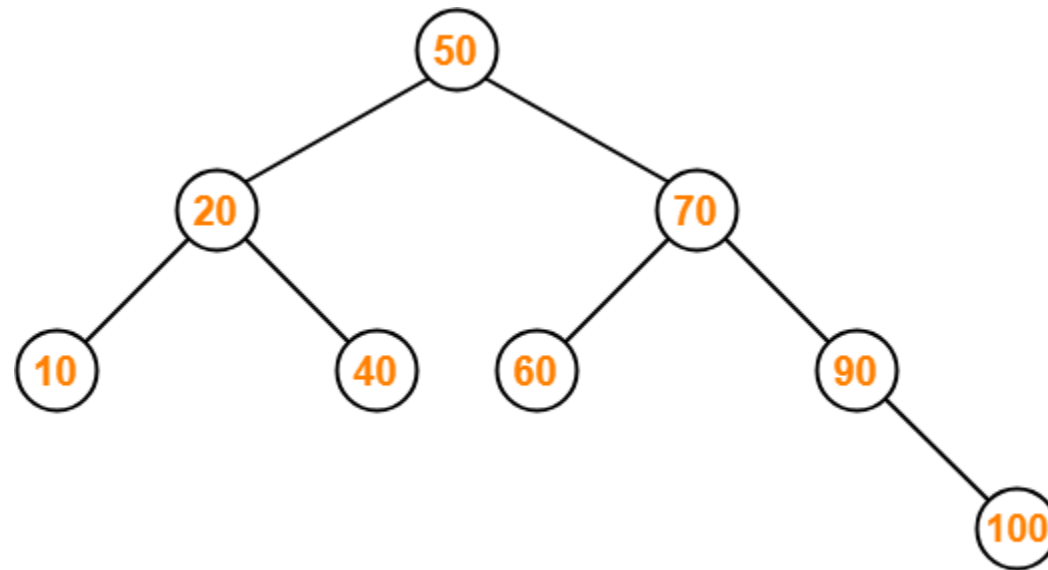
Binary search tree

13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18



Binary search tree

50, 70, 60, 20, 90, 10, 40, 100



Binary Search Tree

Binary search tree

10,12,5,4,20,8,7,15 and 13

insert (10)



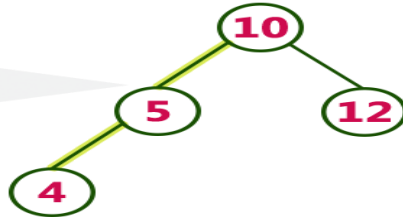
insert (12)



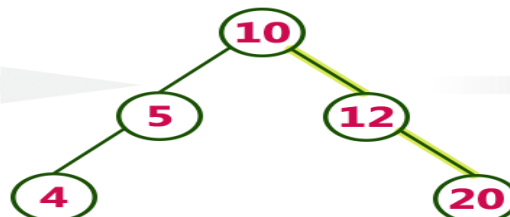
insert (5)



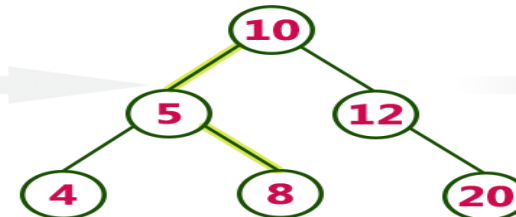
insert (4)



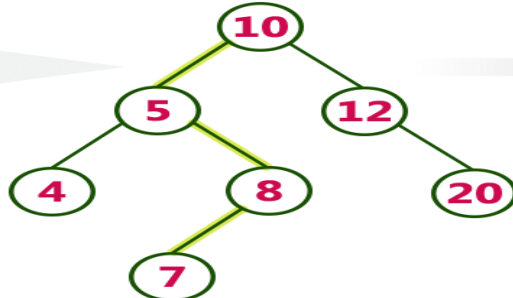
insert (20)



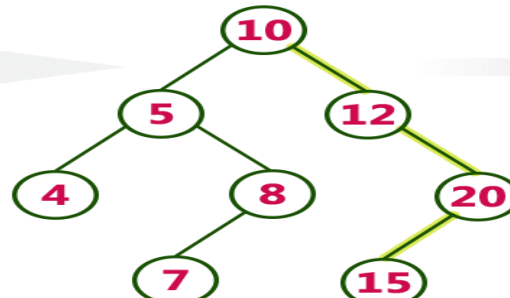
insert (8)



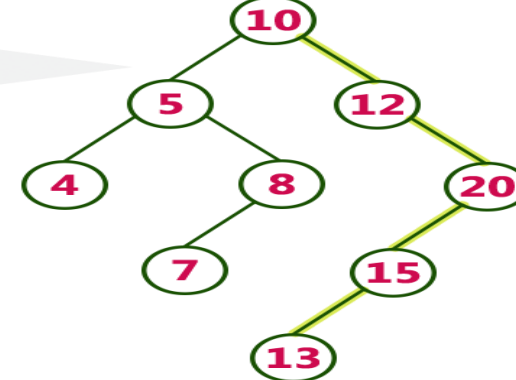
insert (7)



insert (15)

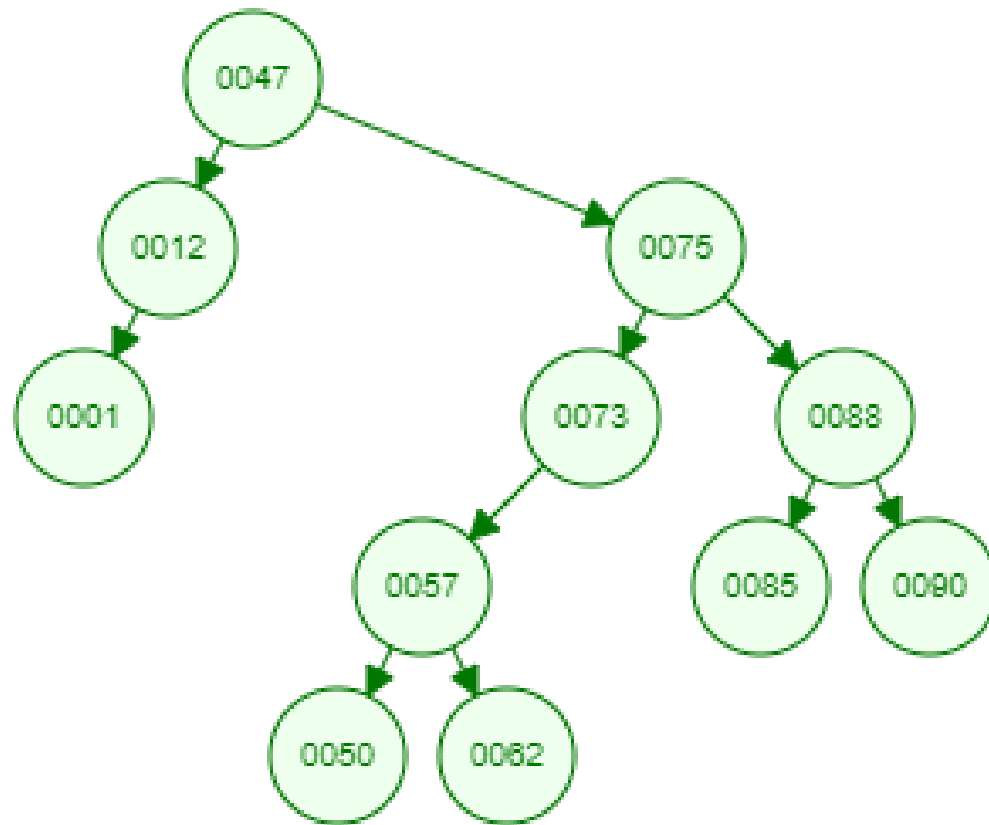


insert (13)



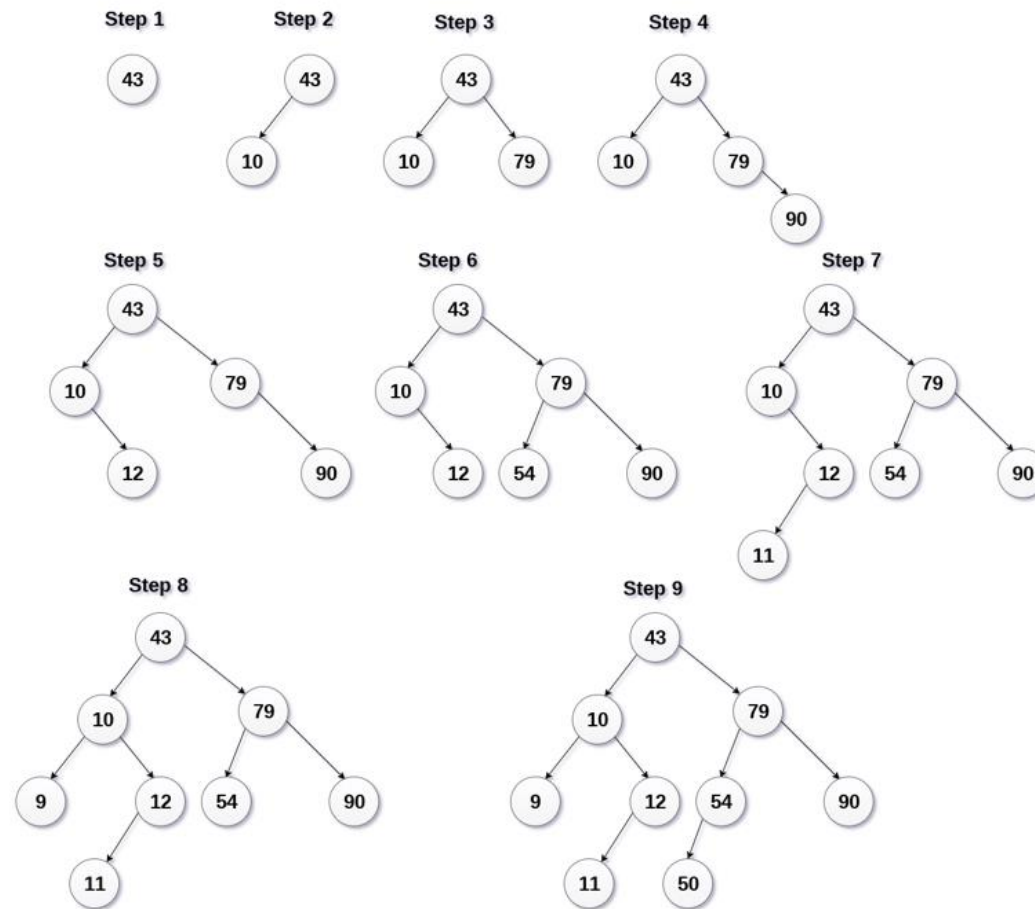
Binary search tree

47, 12, 75, 88, 90, 73, 57, 1, 85, 50, 62



Binary search tree

43, 10, 79, 90, 12, 54, 11, 9, 50



Binary search Tree Creation

Binary search tree

49,23,3,5,8,78,90,38,29,59,13,83,94

Binary search tree operations

Basic Operations

Following are the basic operations of a tree –

- **Search** – Searches an element in a tree.
- **Insert** – Inserts an element in a tree.
- **delete** – Inserts an element in a tree.
- **Pre-order Traversal** – Traverses a tree in a pre-order manner.
- **In-order Traversal** – Traverses a tree in an in-order manner.
- **Post-order Traversal** – Traverses a tree in a post-order manner.

Binary search tree operations

- **Insert** – Inserts an element in a tree.

Insertion

Insert function is used to add a new element in a binary search tree at appropriate location. Insert function is to be designed in such a way that, it must not violate the property of binary search tree at each value.

1. Allocate the memory for tree.
2. Set the data part to the value and set the left and right pointer of tree, point to NULL.
3. If the item to be inserted, will be the first element of the tree, then the left and right of this node will point to NULL.
4. Else, check if the item is less than the root element of the tree, if this is true, then recursively perform this operation with the left of the root.
5. If this is false, then perform this operation recursively with the right sub-tree of the root.

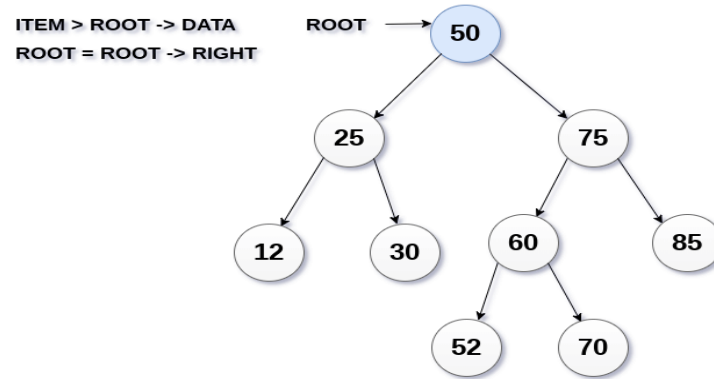
Insert (TREE, ITEM)

- **Step 1:** IF TREE = NULL
 Allocate memory for TREE
 SET TREE -> DATA = ITEM
 SET TREE -> LEFT = TREE -> RIGHT = NULL
ELSE
 IF ITEM < TREE -> DATA
 Insert(TREE -> LEFT, ITEM)
ELSE
 Insert(TREE -> RIGHT, ITEM)
[END OF IF]
[END OF IF]
- **Step 2:** END

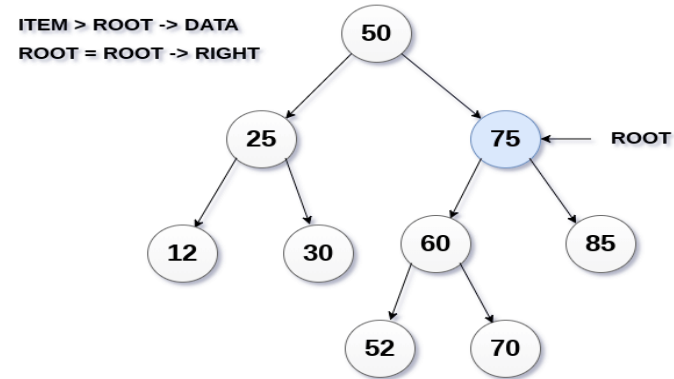
Binary search tree operations

- **Insert** – Inserts an element in a tree.

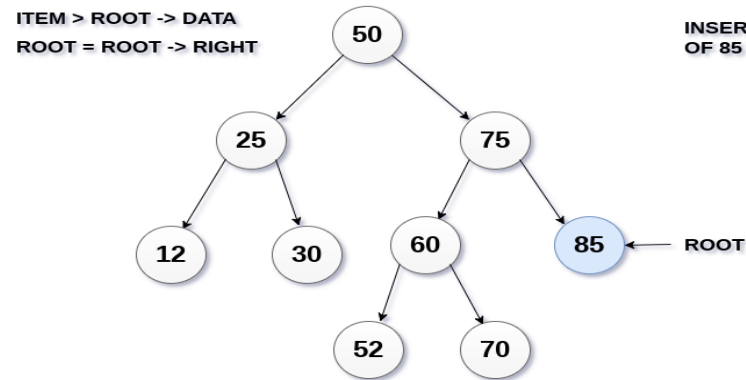
Item = 95



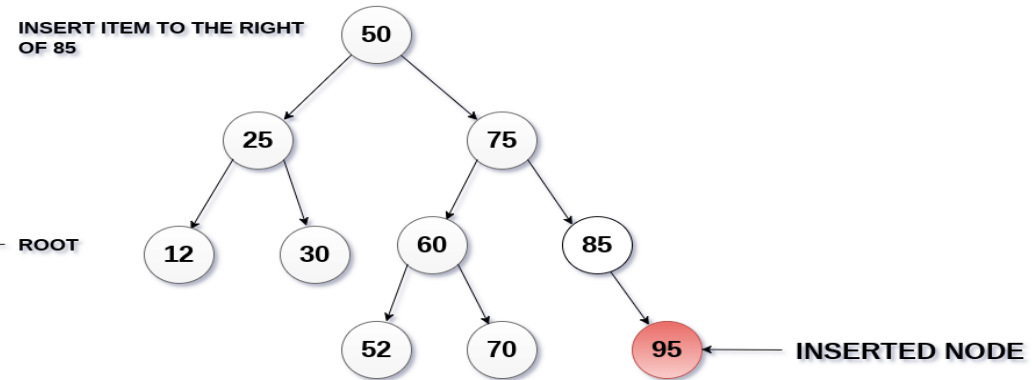
STEP 1



STEP 2



STEP 3



STEP 4

Binary search tree operations

Search a Node – search() method

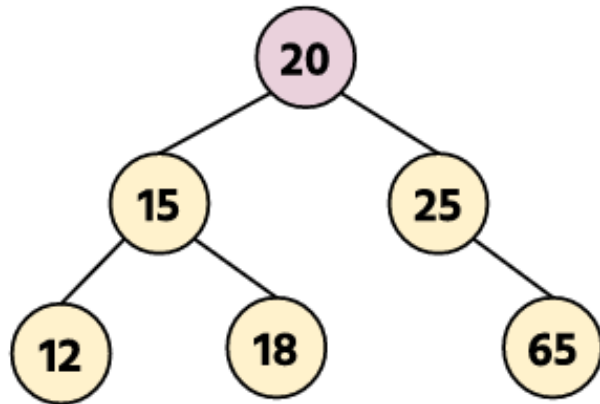
Searching a node in a Binary search tree takes the following steps:

1. Compare the current node data with the key if:

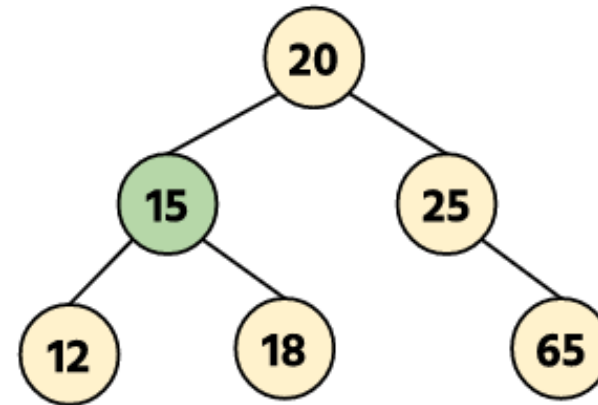
- If the key is found, then return the node.
- If the key is lesser than the node data, move the current to the left node and again repeat step 1.
- If the key is greater then move to the right and repeat step 1.

2. If the node is not found then return NULL.

Example: Let's find the node with data 15 in the given BST.



node 20 is greater than the key 15. So move to the left of 20.



node 15 and key is matched

Tree: Delete

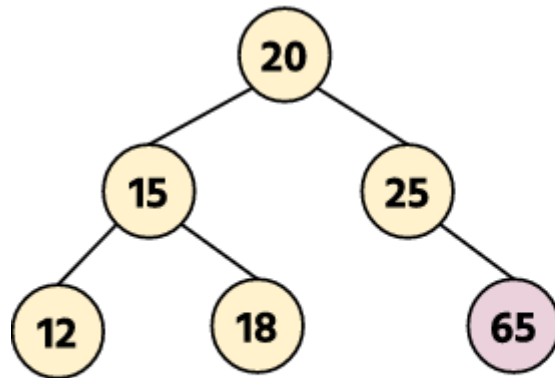
Delete a Node – delete() Method

There are three cases to delete the node from the tree. Let's discuss all the cases using the examples that are given below.

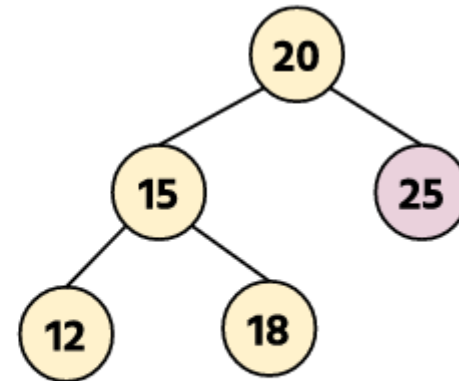
case 1: Deleting the leaf node

To delete the leaf node we need to traverse to the required leaf node then delete that node and update its parent.

Example: Deleting a leaf node 65.



traverse through the leaf
node



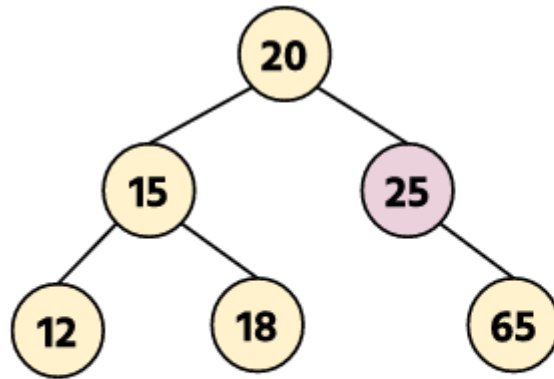
delete the node 65 and
update its parent node

Tree: Delete

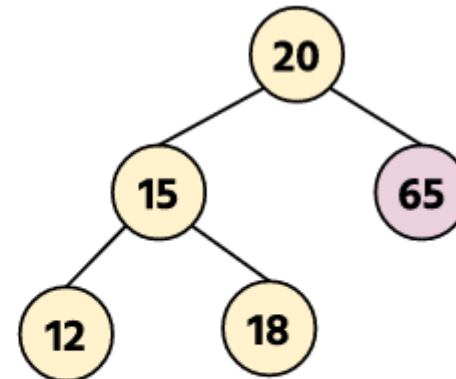
case 2: Delete a node with one child

Remove that given node and replace it with its children.

Example: Deleting node 25, to delete node 25, we have to update it with its children node 65.



reach to the node 25



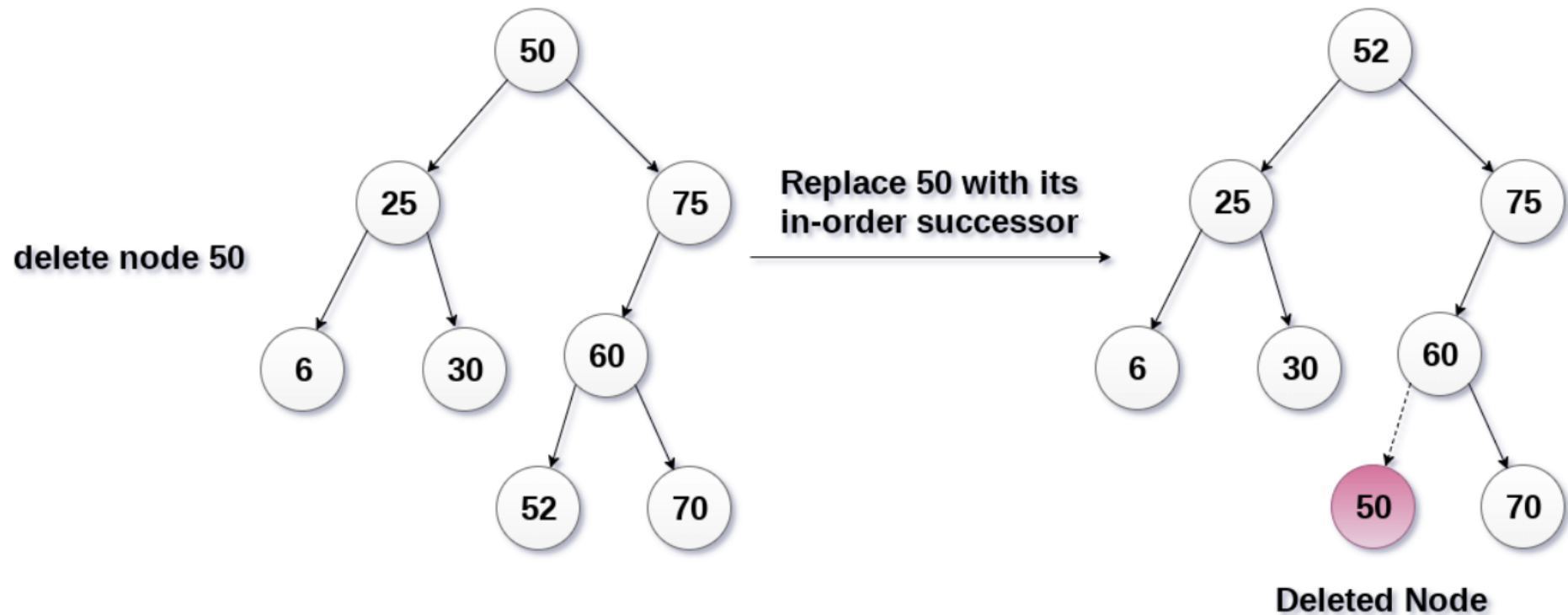
Copy the value of leaf node 65 to node 25, and delete the leaf node

Binary search tree - Deletion

- **Case 3: Node to be deleted has two children:** Find inorder successor of the node. Copy contents of the inorder successor to the node and delete the inorder successor.
- However, the node which is to be deleted, is replaced with its in-order successor or predecessor recursively until the node value (to be deleted) is placed on the leaf of the tree. After the procedure, replace the node with NULL and free the allocated space.
- In the following image, the node 50 is to be deleted which is the root node of the tree. The in-order traversal of the tree given below.
- 6, 25, 30, 50, 52, 60, 70, 75.
- replace 50 with its in-order successor 52. Now, 50 will be moved to the leaf of the tree, which will simply be deleted.

Binary search tree - Deletion

- Node to be deleted has two children-example



Tree traversal

Traversal is a process to visit all the nodes of a tree and may print their values too. Because, all nodes are connected via edges (links) we always start from the root (head) node. That is, we cannot randomly access a node in a tree. There are three ways which we use to traverse a tree –

- In-order Traversal
- Pre-order Traversal
- Post-order Traversal

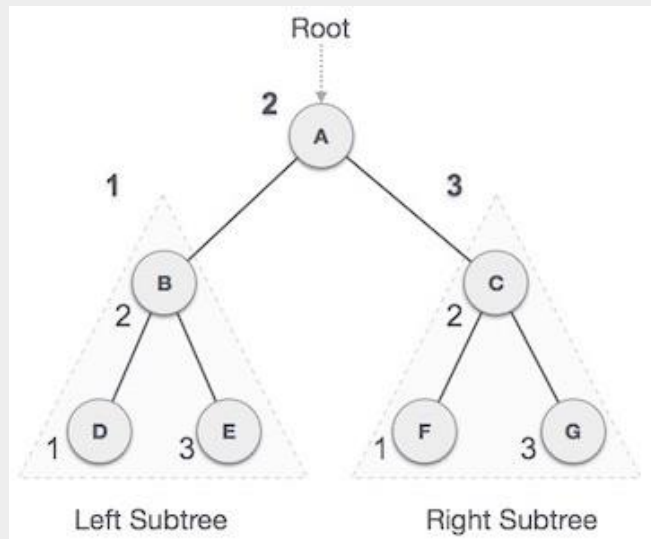
Generally, we traverse a tree to search or locate a given item or key in the tree or to print all the values it contains.

Tree traversal

In-order Traversal (left root right)

In this traversal method, the left subtree is visited first, then the root and later the right sub-tree. We should always remember that every node may represent a subtree itself.

If a binary tree is traversed **in-order**, the output will produce sorted key values in an ascending order.



We start from **A**, and following in-order traversal, we move to its left subtree **B**. **B** is also traversed in-order. The process goes on until all the nodes are visited. The output of inorder traversal of this tree will be –

$D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$

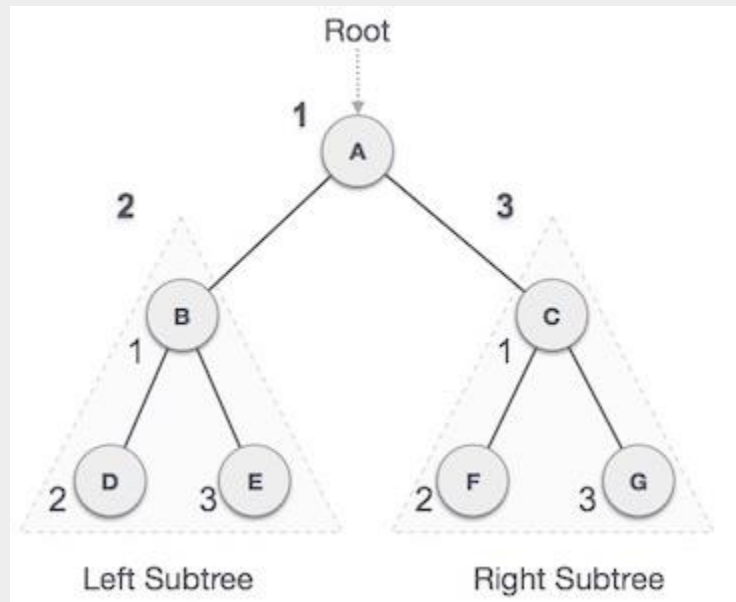
Algorithm

Until all nodes are traversed – **Step 1** – Recursively traverse left subtree. **Step 2** – Visit root node. **Step 3** – Recursively traverse right subtree.

Tree traversal

Pre-order Traversal (root left right)

In this traversal method, the root node is visited first, then the left subtree and finally the right subtree.



We start from **A**, and following pre-order traversal, we first visit **A** itself and then move to its left subtree **B**. **B** is also traversed pre-order. The process goes on until all the nodes are visited. The output of pre-order traversal of this tree will be –

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

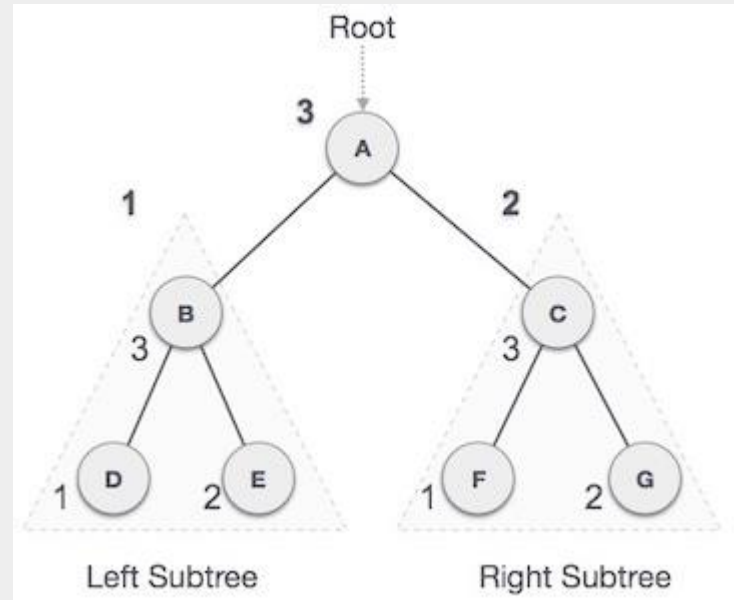
Algorithm

Until all nodes are traversed – **Step 1** – Visit root node. **Step 2** – Recursively traverse left subtree. **Step 3** – Recursively traverse right subtree.

Tree traversal

Post-order Traversal (left-right-root)

In this traversal method, the root node is visited last, hence the name. First we traverse the left subtree, then the right subtree and finally the root node.



We start from **A**, and following Post-order traversal, we first visit the left subtree **B**. **B** is also traversed post-order. The process goes on until all the nodes are visited. The output of post-order traversal of this tree will be –

$D \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$

Algorithm

Until all nodes are traversed – **Step 1** – Recursively traverse left subtree. **Step 2** – Recursively traverse right subtree. **Step 3** – Visit root node.

Tree traversal

InOrder(root) visits nodes in the following order:

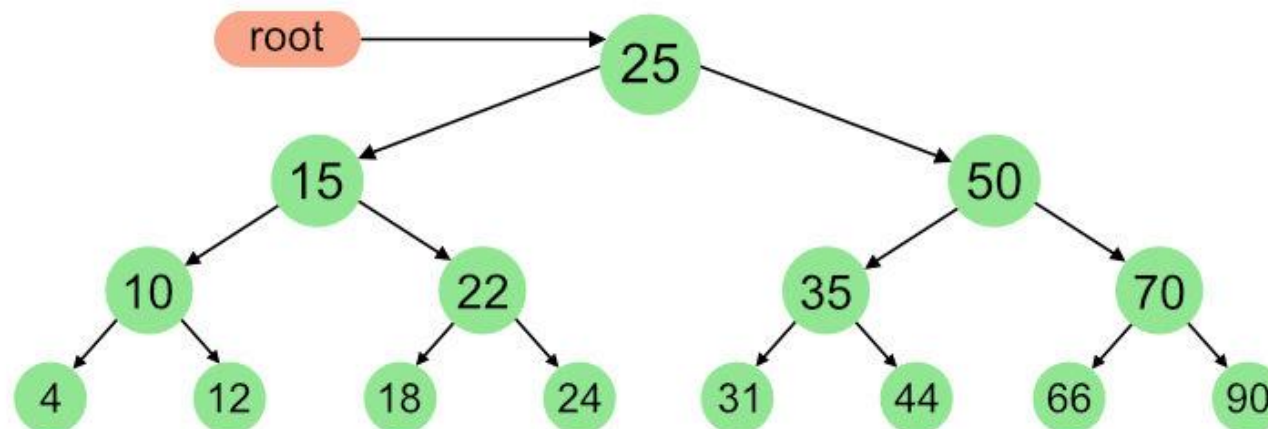
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:

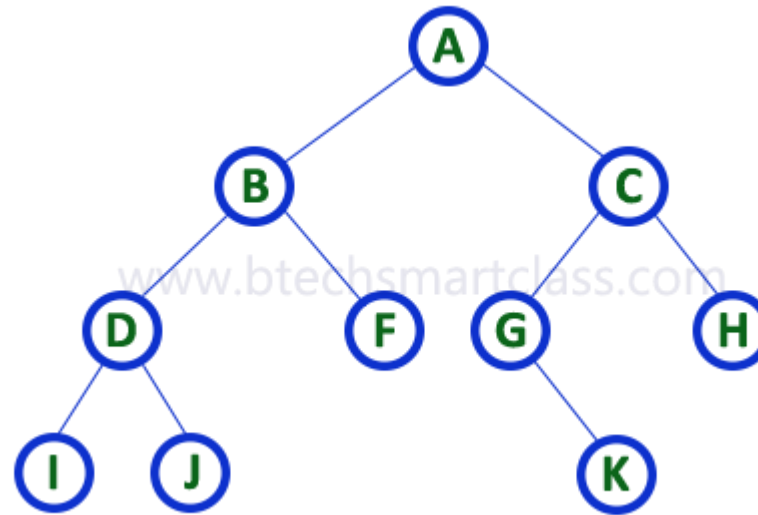
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



Tree traversal



In-Order Traversal for above example of binary tree is

I - D - J - B - F - A - G - K - C - H

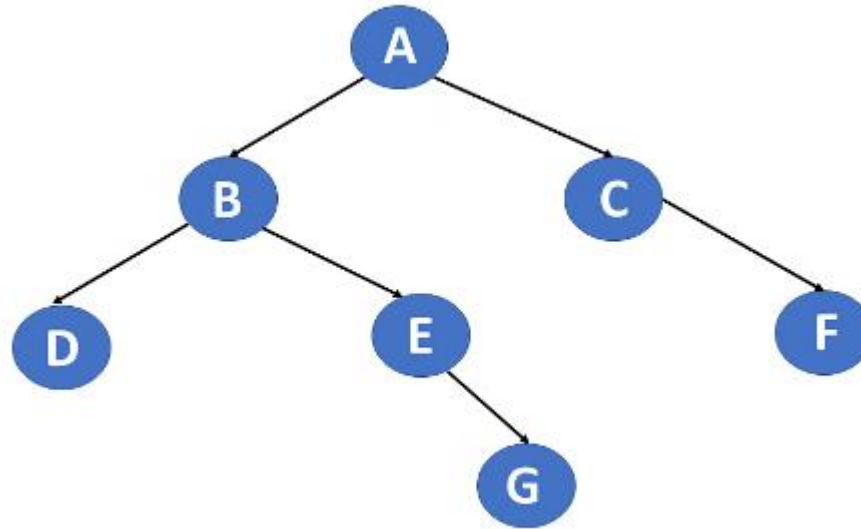
Pre-Order Traversal for above example binary tree is

A - B - D - I - J - F - C - G - K - H

Post-Order Traversal for above example binary tree is

I - J - D - F - B - K - G - H - C - A

Tree traversal



In-order Traversal

D	B	E	G	A	C	F
---	---	---	---	---	---	---

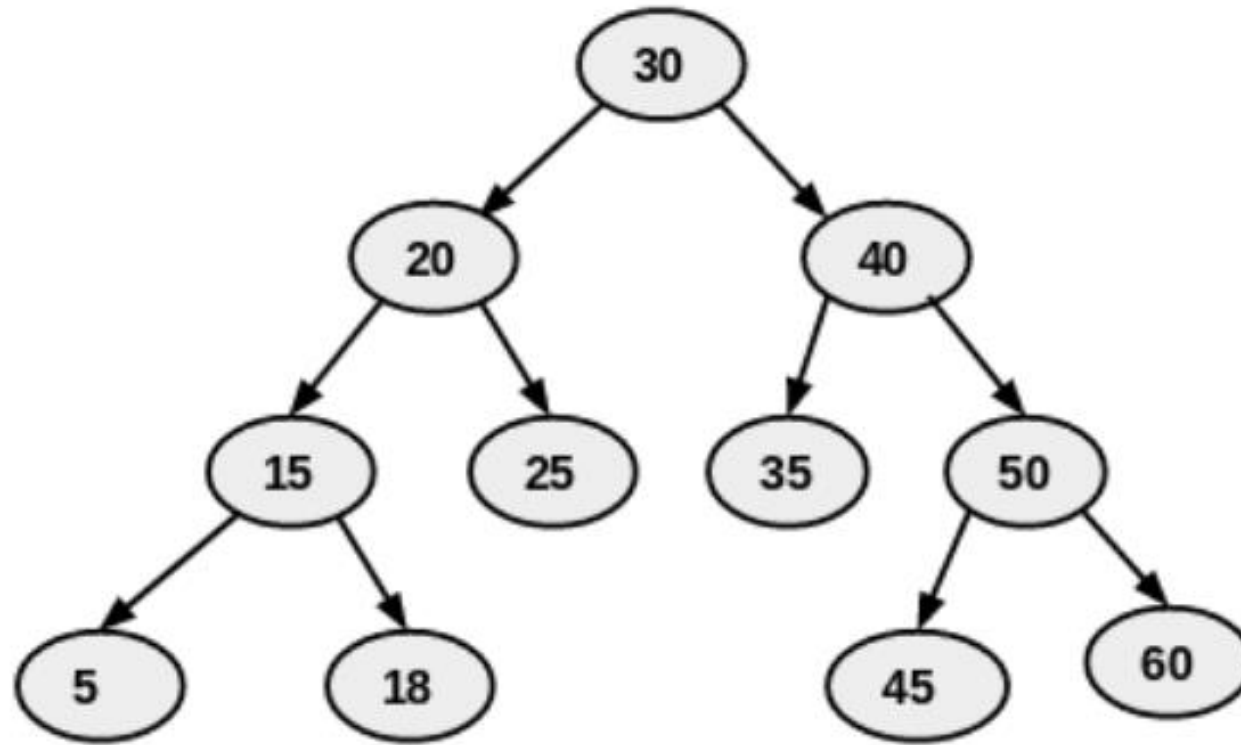
post-order Traversal

D	G	E	B	F	C	A
---	---	---	---	---	---	---

pre-order Traversal

A	B	D	E	G	C	F
---	---	---	---	---	---	---

Tree traversal

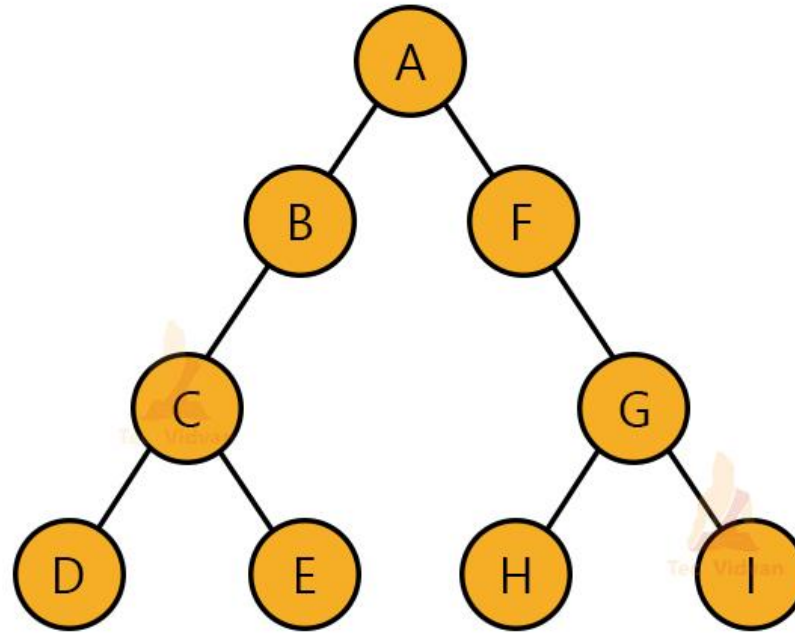


In order {5 , 15 , 18 , 20 , 25 , 30 , 35 , 40 , 45 , 50 , 60}

Pre order {30 , 20 , 15 , 5 , 18 , 25 , 40 , 35 , 50 , 45 , 60}

Post order {5 , 18 , 15 , 25 , 20 , 35 , 45 , 60 , 50 , 40 , 30}

Tree traversal

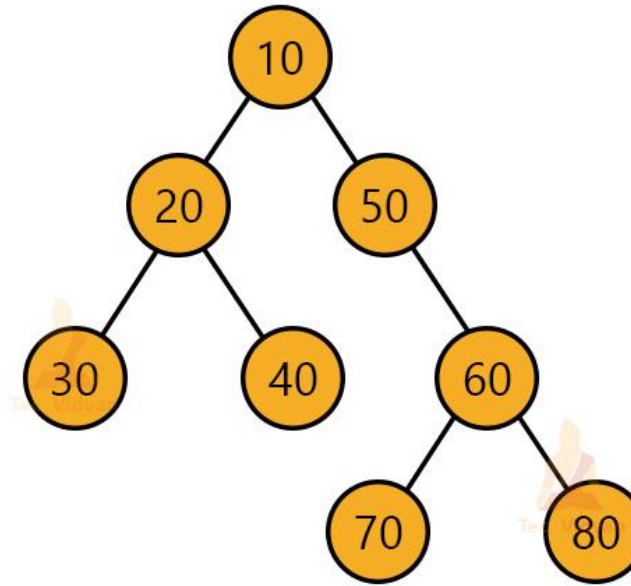


In order

Pre order $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H \rightarrow I.$

Post order

Tree traversal

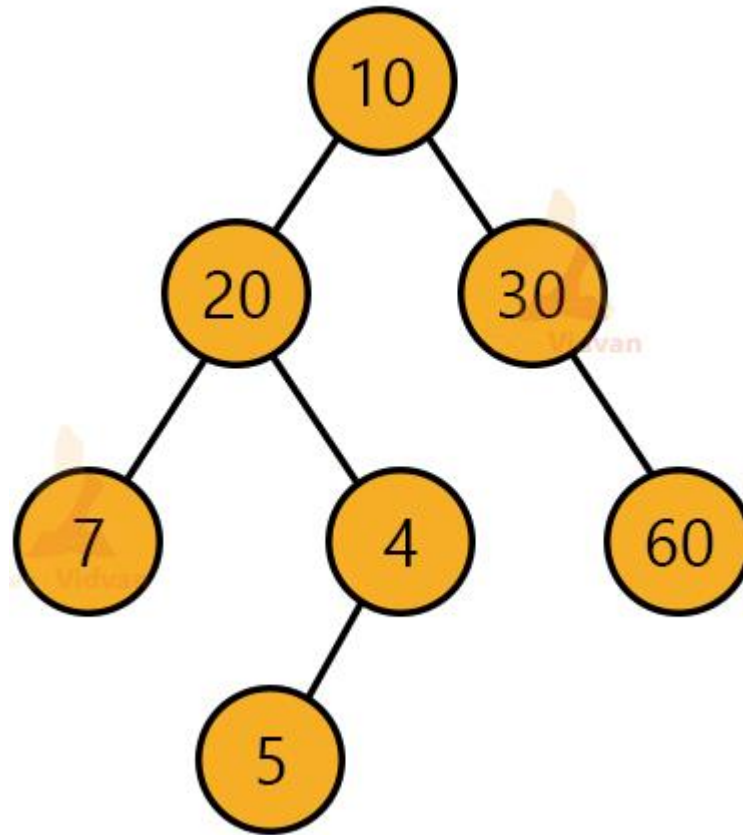


In order 30→20→40→10→50→70→60→80.

Pre order

Post order

Tree traversal



In order

Pre order

Post order $7 \rightarrow 5 \rightarrow 4 \rightarrow 20 \rightarrow 60 \rightarrow 30 \rightarrow 10.$

The Three Traversals and Their Purpose

Inorder Traversal (Left → Root → Right)

Main use: Get **sorted data** from a Binary Search Tree (BST)

- In a BST, inorder traversal gives values in **ascending order**
- Used in:
 - Sorting-like output
 - Validating BST

Preorder Traversal (Root → Left → Right)

Main use: Copy or reconstruct the tree

- Root is processed first → structure is preserved
- Used in:
 - Creating a **copy of tree**

Postorder Traversal (Left → Right → Root)

Main use: Delete/free the tree

- Children are processed before the parent
- Used in:
 - Deleting nodes safely (Delete children first, then parent → avoids memory issues)
 - Evaluating expression trees

Binary Expression Tree

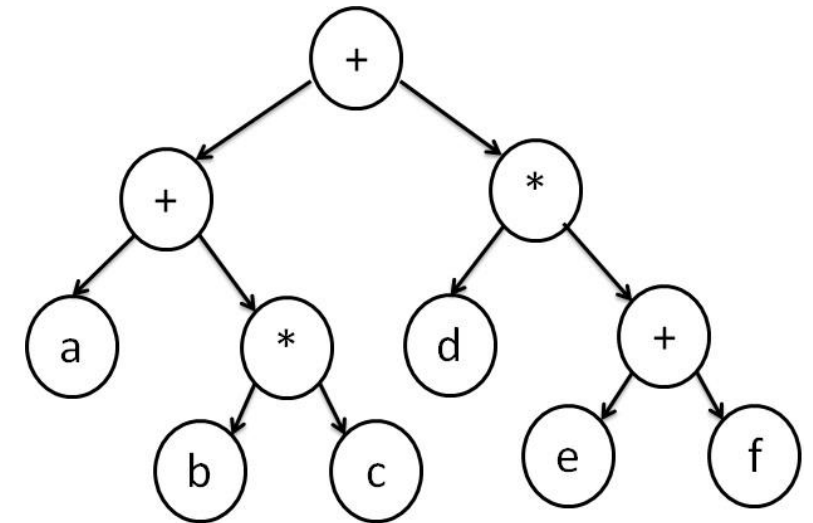
Expression Tree is used to represent expressions.

An expression and expression tree shown below

$a + (b * c) + d * (e + f)$

Expression Tree is a special kind of binary tree with the following properties:

- Each leaf is an operand. Examples: a, b, c, 6, 100
- The root and internal nodes are operators. Examples: +, -, *, /, ^
- Subtrees are subexpressions with the root being an operator.



Binary Expression Tree

Use of Expression tree

- 1.The main objective of using the expression trees is to make complex expressions and can be easily be evaluated using these expression trees.
- 2.It is also used to find out the associativity of each operator in the expression.
- 3.It is also used to solve the postfix, prefix, and infix expression evaluation.
- 4.A binary expression tree is used in a compiler to represent arithmetic expressions during parsing. It helps maintain operator precedence, generate intermediate code, and perform optimizations efficiently.

Source Code → Parser (checks whether the program's **syntax (grammar)** is correct) → Expression Tree → Intermediate Code → Machine Code

Binary Expression Tree

- Traversal Techniques

Inorder Traversal -

We can produce an infix expression by recursively printing out the left expression, the root, and the right expression.

Postorder Traversal

The postfix expression can be evaluated by recursively printing out the left expression, the right expression and then the root

Preorder Traversal

We can also evaluate prefix expression by recursively printing out: the root, the left expression and the right expression.

Binary Expression Tree

If we apply all these strategies to the sample tree above, the outputs are:

- Infix expression:

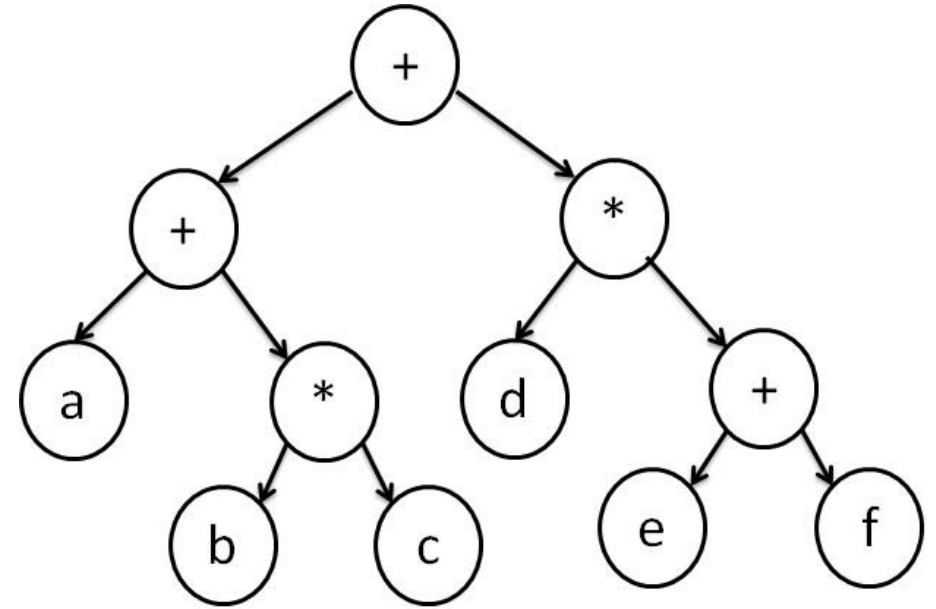
```
(a+(b*c))+(d*(e + f))
```

- Postfix Expression:

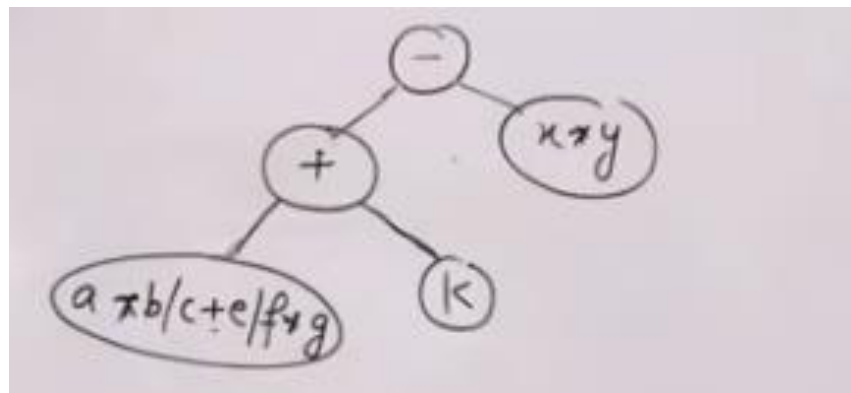
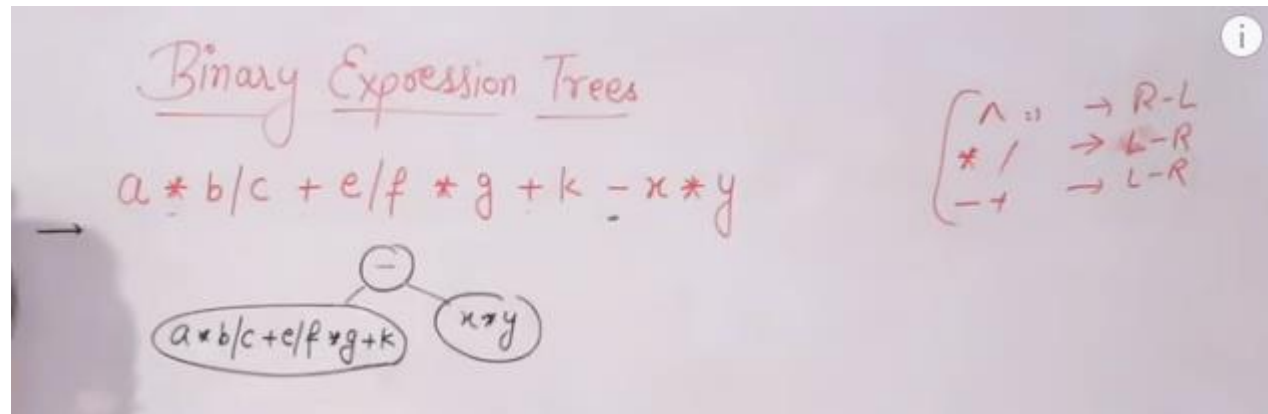
```
a b c * + d e f + * +
```

- Prefix Expression:

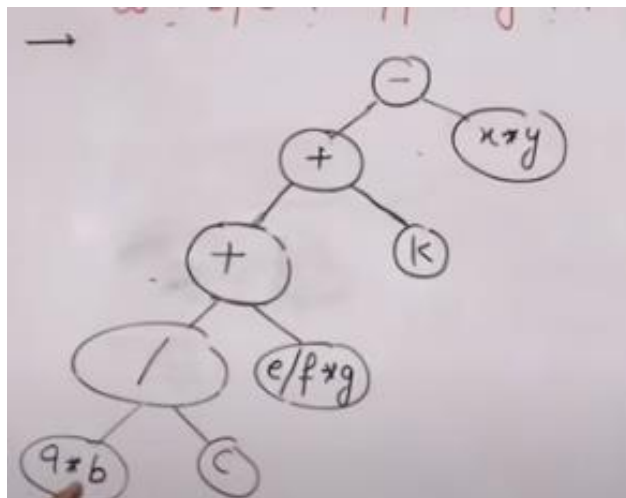
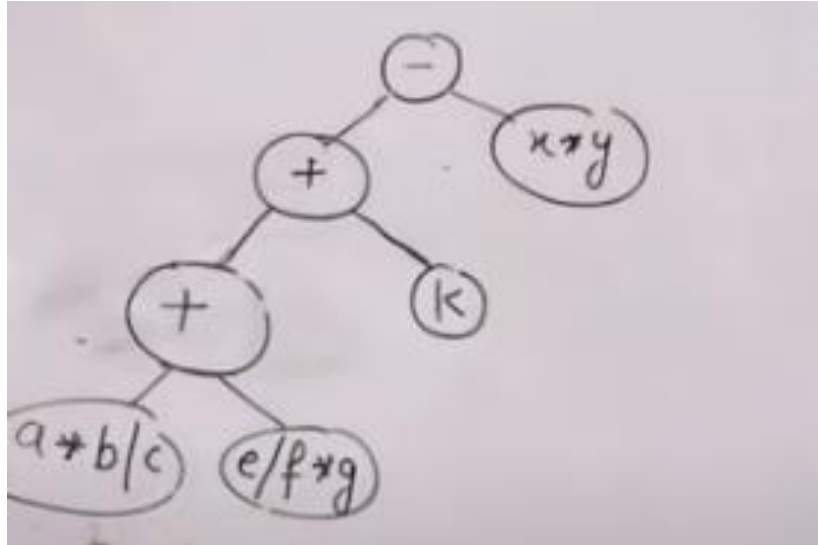
```
+ + a * b c * d + e f
```



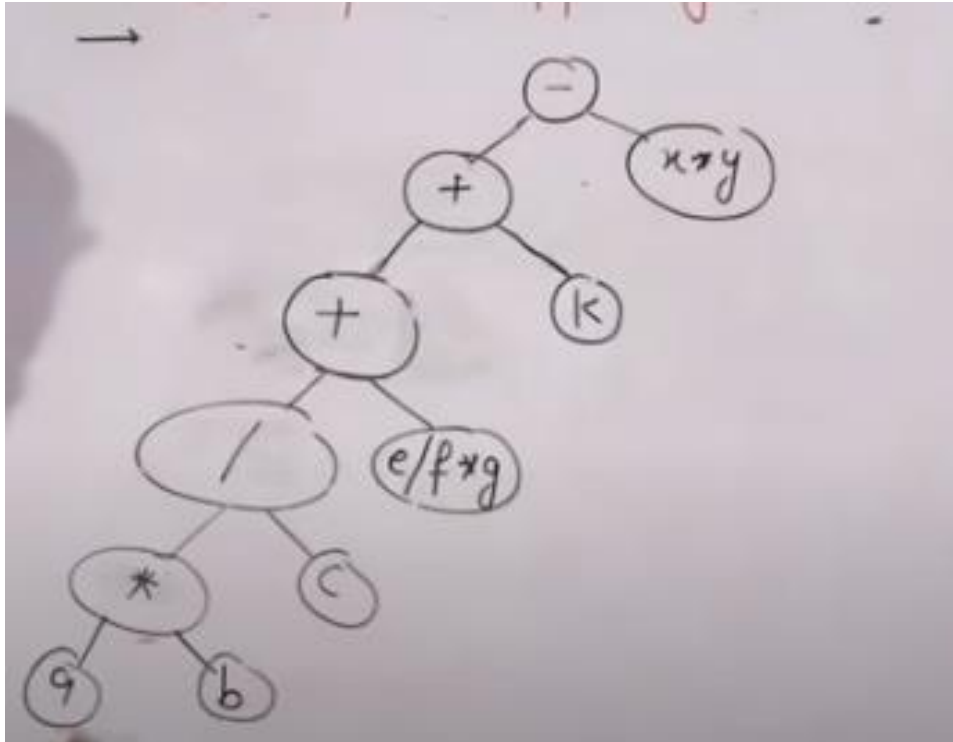
Binary Expression Tree – construct expression tree from infix exp : Method 1 (direct-without stack)



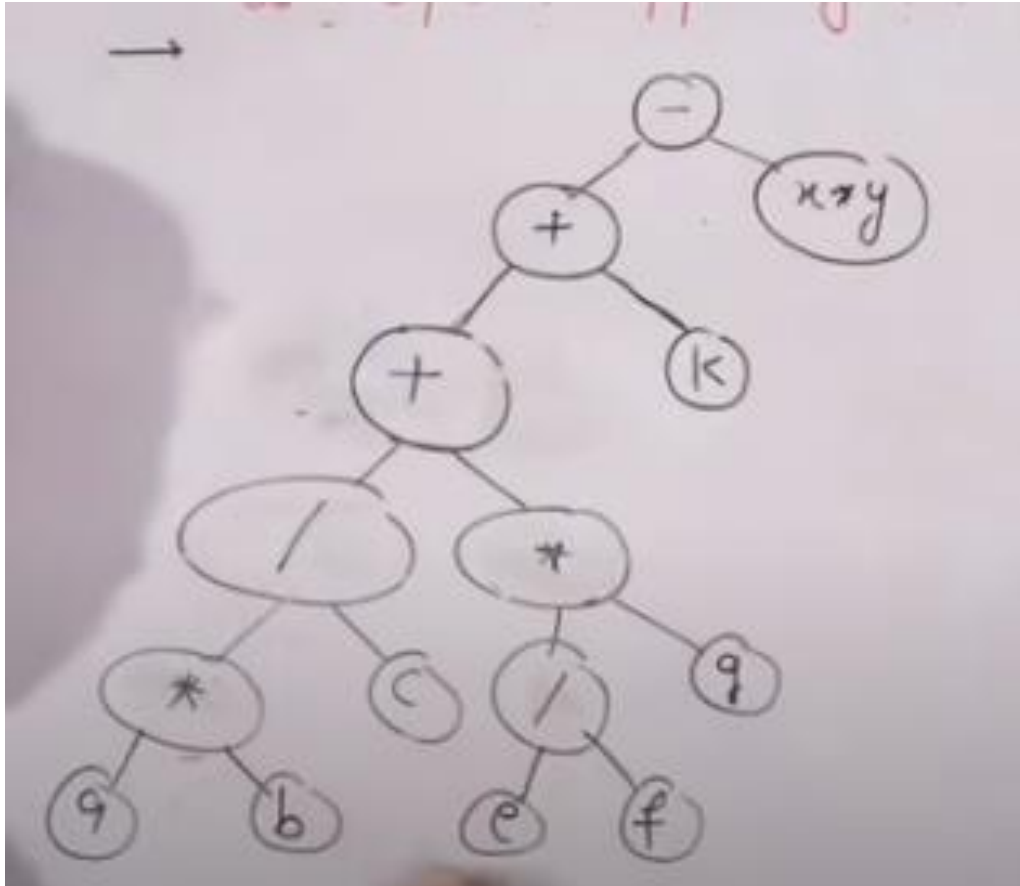
Binary Expression Tree – construct expression tree from infix exp



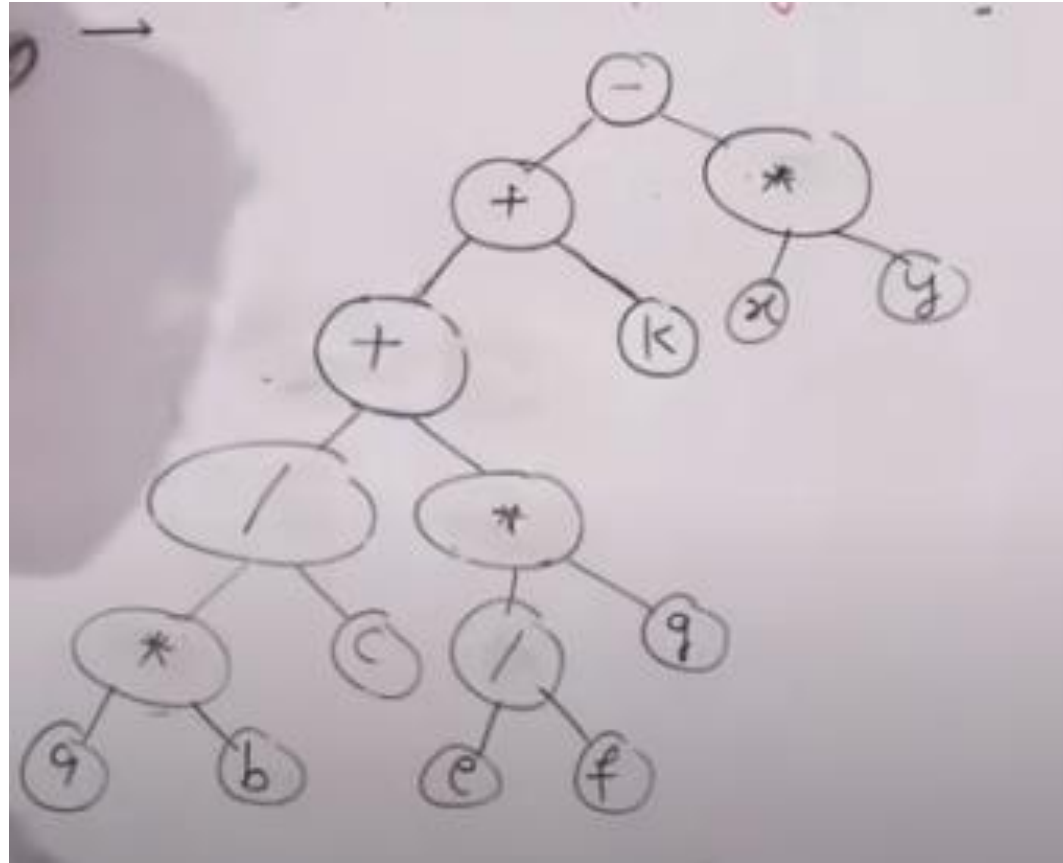
Binary Expression Tree – construct expression tree from infix exp



Binary Expression Tree – construct expression tree from infix exp

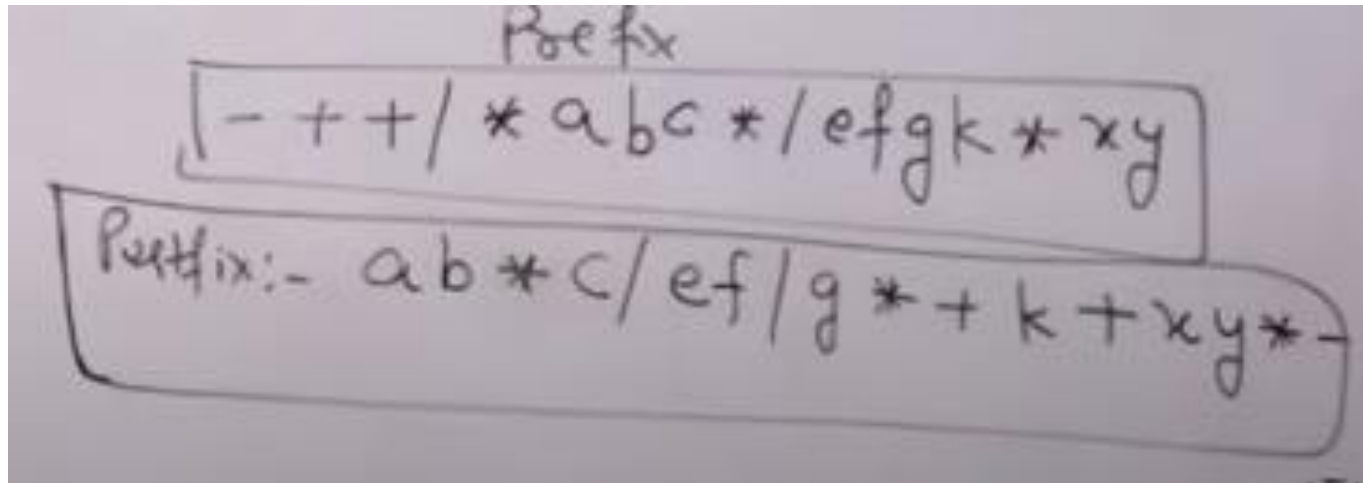


Binary Expression Tree – construct expression tree from infix exp



Binary Expression Tree – construct expression tree from infix exp

Now we can create prefix and postfix expression using this expression tree.



Binary Expression Tree – construct expression tree from postfix exp

Let us consider a **postfix expression** is given as an input for constructing an expression tree.

Following are the step to construct an expression tree:

1. Read one symbol at a time from the postfix expression.
2. Check if the symbol is an operand or operator.
3. If the symbol is an operand, create a one node tree and push a pointer onto a stack
4. If the symbol is an operator, pop two pointers from the stack namely T_1 & T_2 and form a new tree with root as the operator, T_1 & T_2 as a left and right child
5. A pointer to this new tree is pushed onto the stack

Thus, An expression is created or constructed by reading the symbols or numbers from the left. If operand, create a node. If operator, create a tree with operator as root and two pointers to left and right subtree

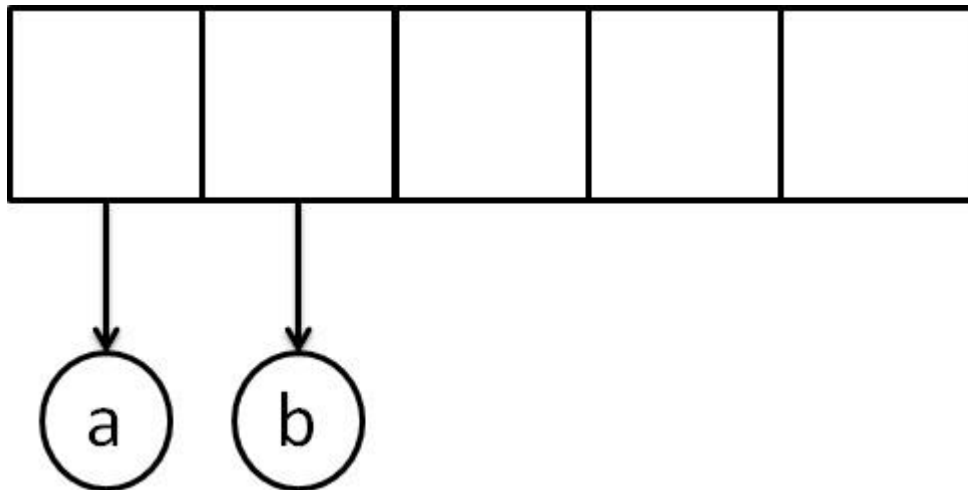
Binary Expression Tree – construct expression tree from postfix exp

Example - Postfix Expression Construction

The input is:

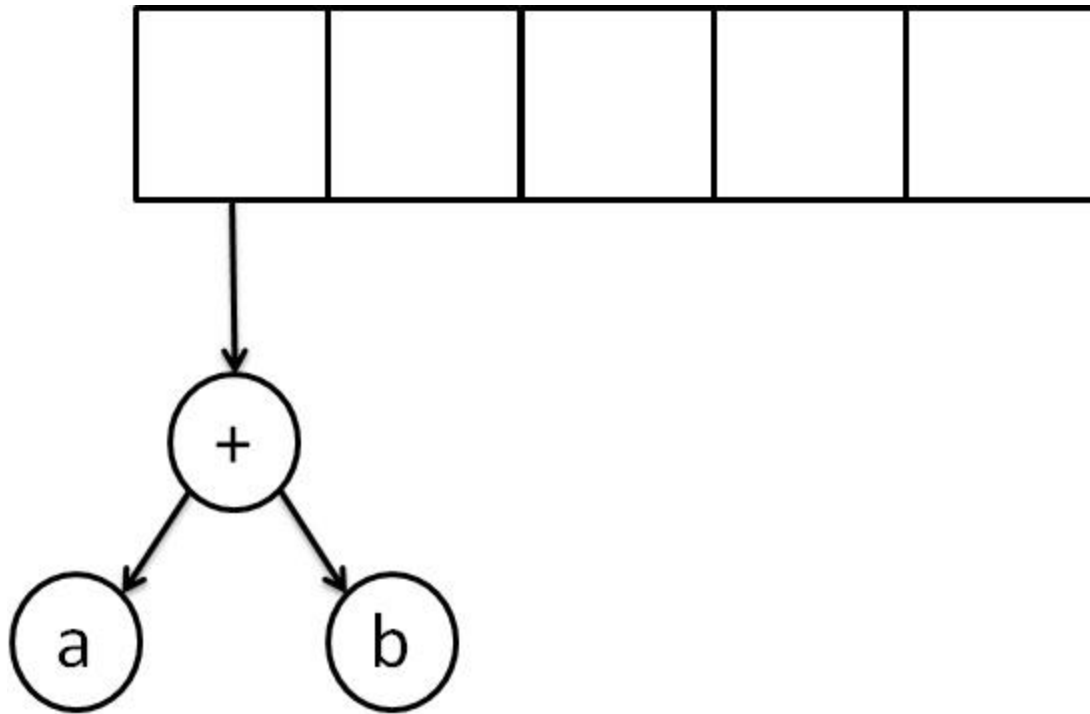
a b + c *

The first two symbols are operands, we create one-node tree and push a pointer to them onto the stack.



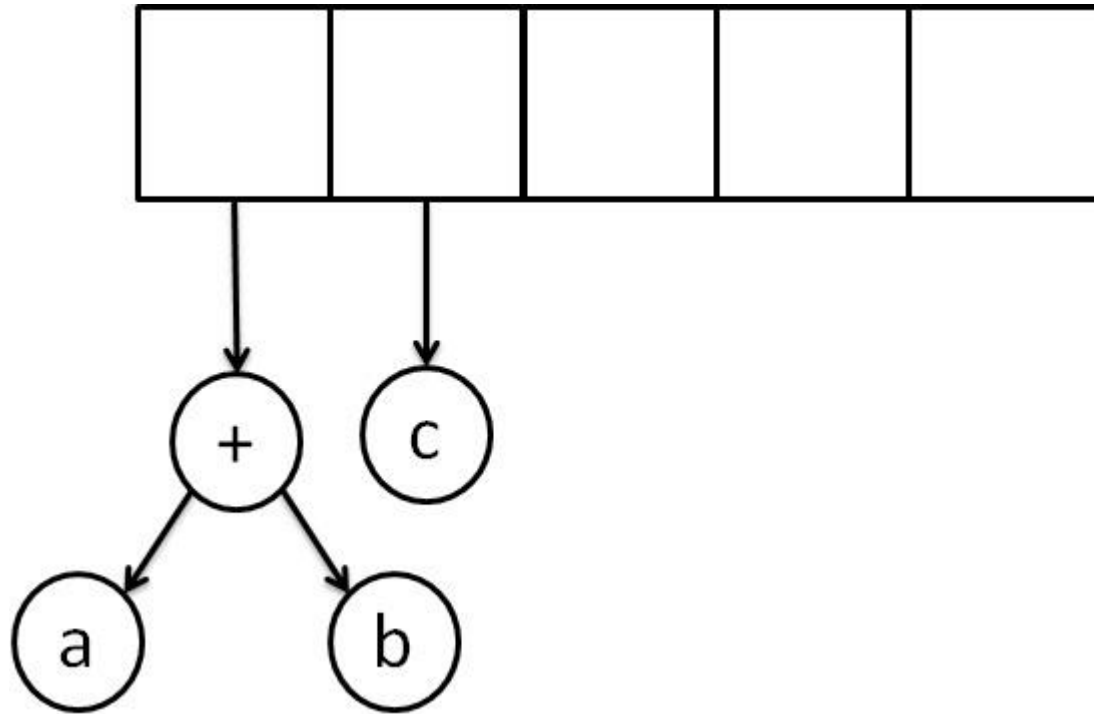
Binary Expression Tree – construct expression tree from postfix exp

Next, read a '+' symbol, so two pointers to tree are popped, a new tree is formed and push a pointer to it onto the stack.



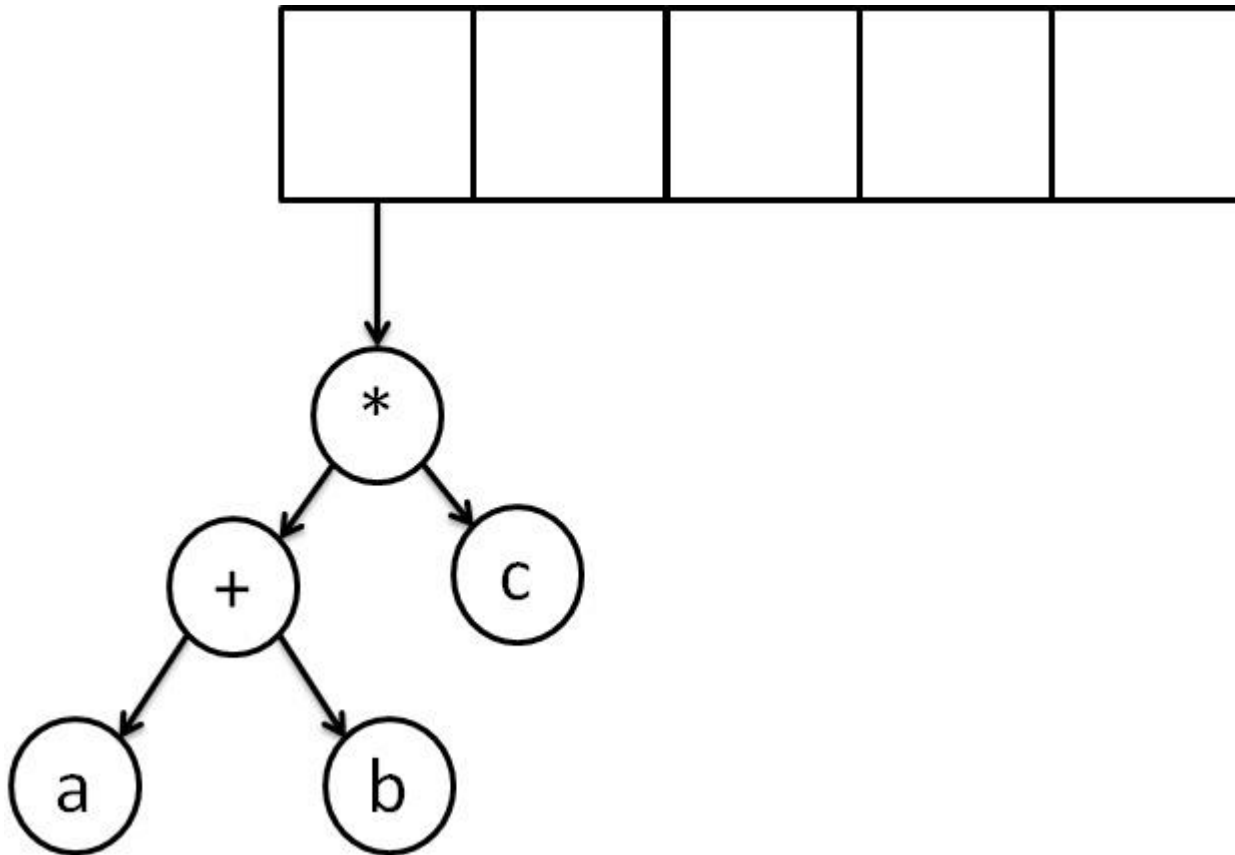
Binary Expression Tree – construct expression tree from postfix exp

Next, 'c' is read, we create one node tree and push a pointer to it onto the stack.



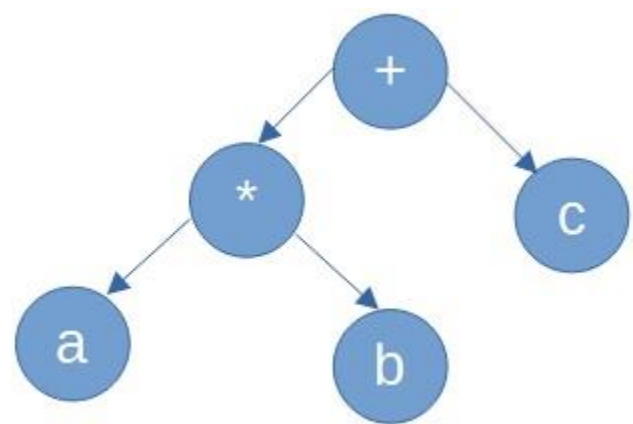
Binary Expression Tree – construct expression tree from postfix exp

Finally, the last symbol is read '*', we pop two tree pointers and form a new tree with a, '*' as root, and a pointer to the final tree remains on the stack.



Binary Expression Tree – construct expression tree

•a*b+c



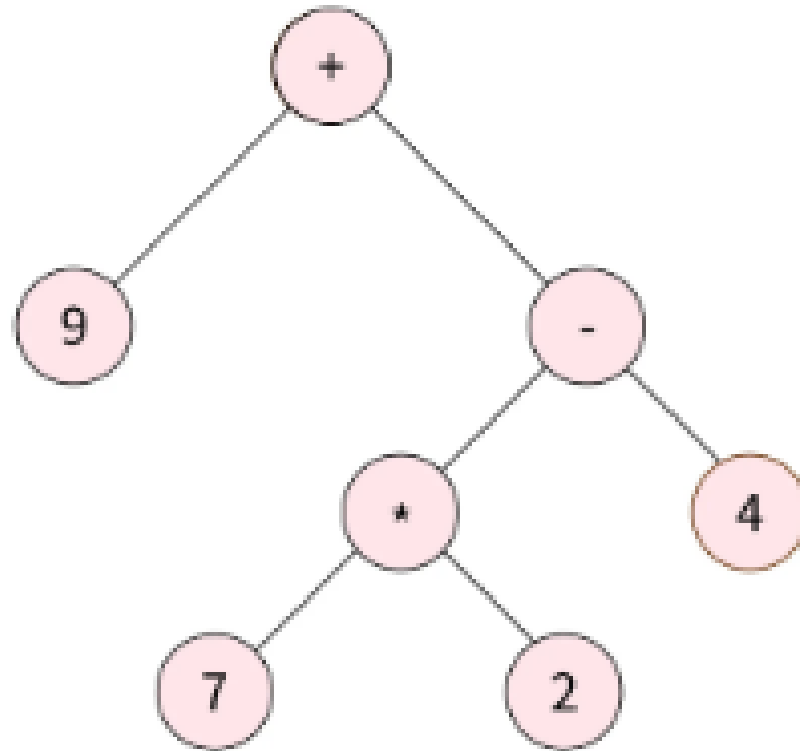
3+((5+9)*2)

Expressions from Expression Tree

Prefix expression	+ * a b c
Postfix expression	a b * c +

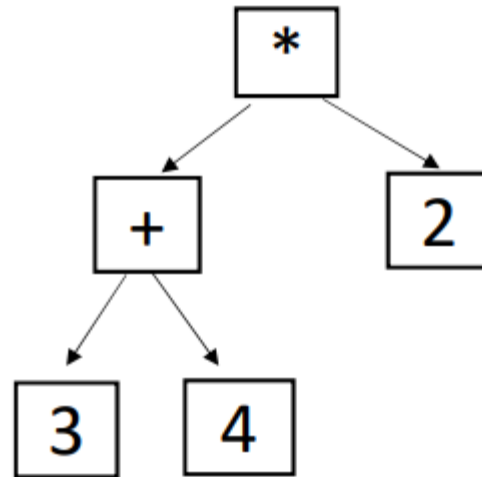
Binary Expression Tree – construct expression tree

$9 + ((7 * 2) - 4)$



Binary Expression Tree – construct expression tree

$(3 + 4) * 2$



AVL Tree

AVL trees are special kind of binary search trees.

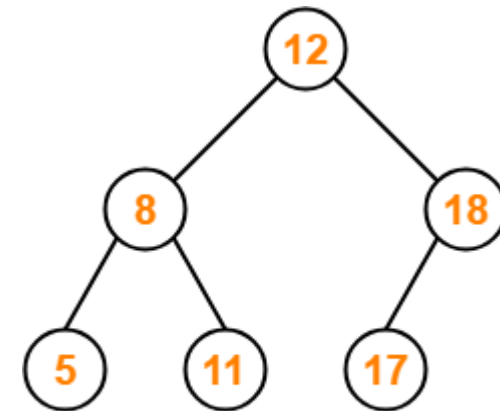
In AVL trees, height of left subtree and right subtree of every node differs by at most one.

AVL trees are also called as **self-balancing binary search trees**.

Following tree is an example of AVL tree-

This tree is an AVL tree because-

- It is a binary search tree.
- The difference between height of left subtree and right subtree of every node is at most one.
- In AVL trees, balancing factor of each node is either 0 or 1 or -1.



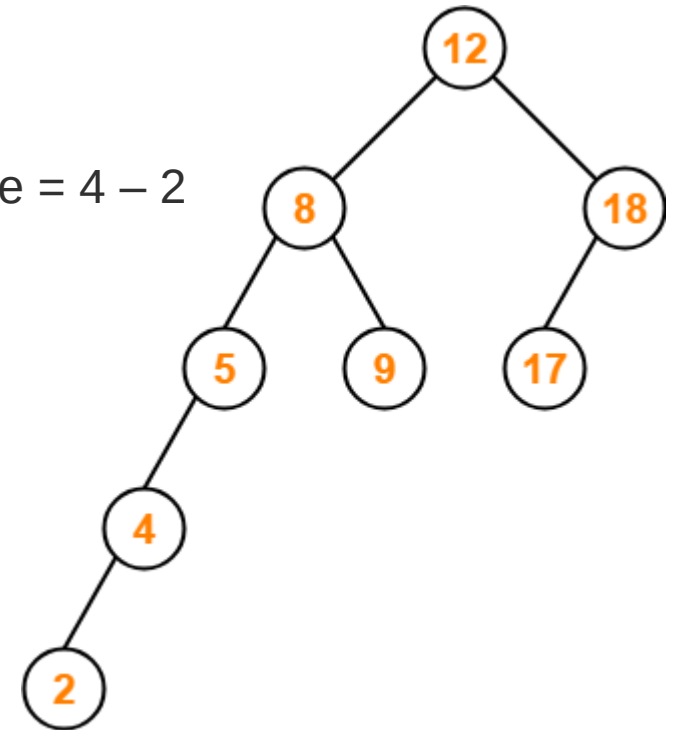
AVL Tree Example

AVL Tree

Following tree is not an example of AVL Tree-

This tree is not an AVL tree because-

- The difference between height of left subtree and right subtree of root node = $4 - 2 = 2$.
- This difference is greater than one.



Not an AVL Tree

AVL Tree

Balance Factor-

In AVL tree,

- Balance factor is defined for every node.
- Balance factor of a node = Height of its left subtree – Height of its right subtree

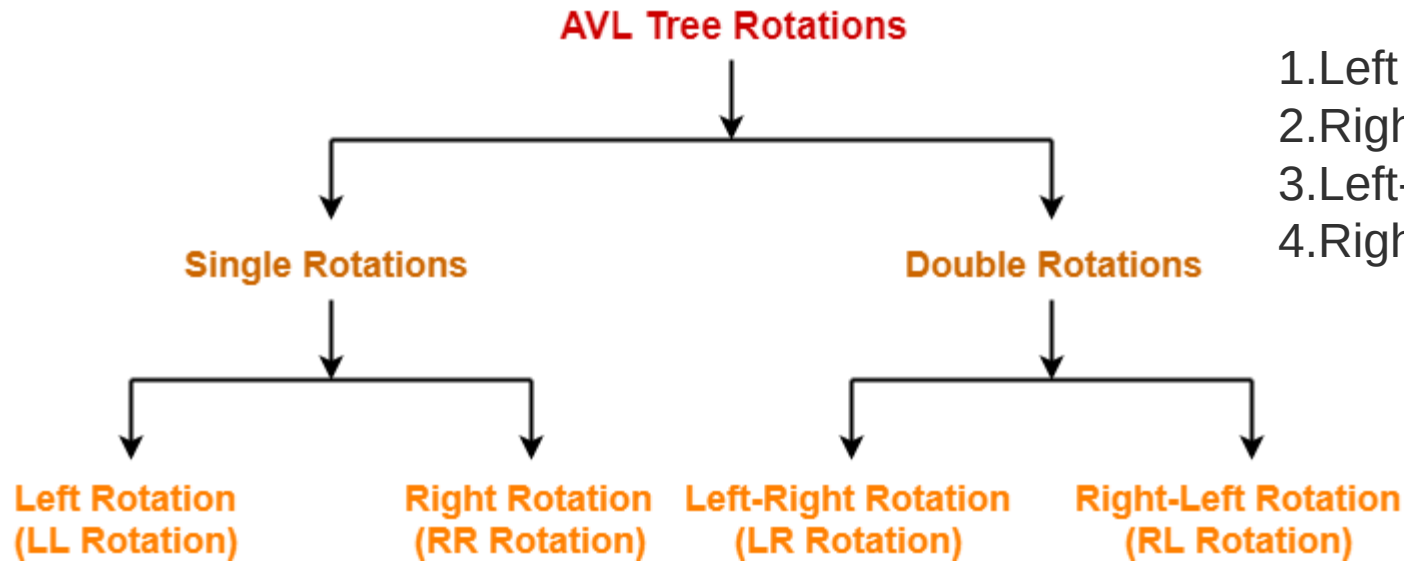
In AVL tree,

Balance factor of every node is either 0 or 1 or -1.

AVL Tree Rotations-

Rotation is the process of moving the nodes to make tree balanced.

There are 4 kinds of rotations possible in AVL Trees-

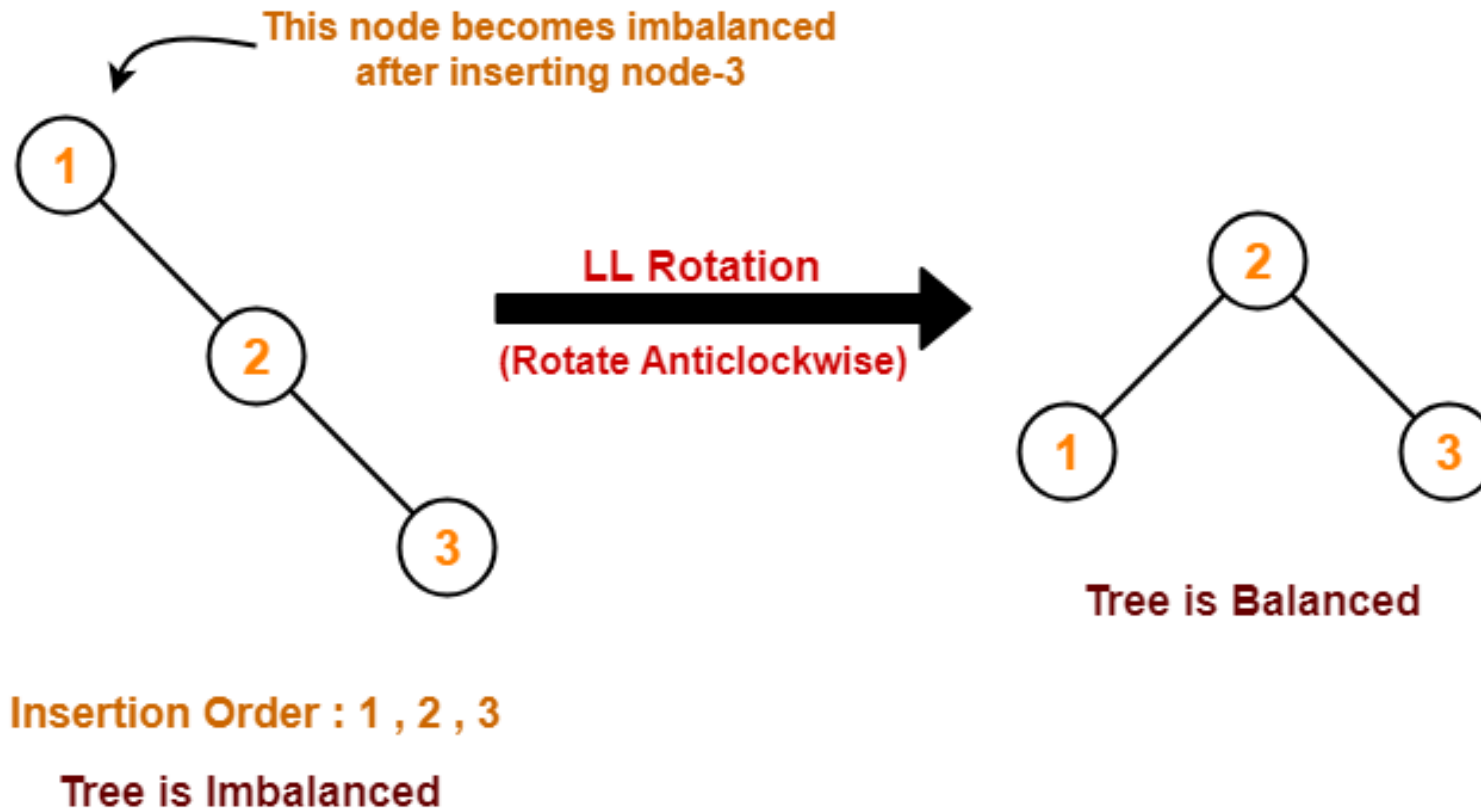


1. Left Rotation (LL Rotation)
2. Right Rotation (RR Rotation)
3. Left-Right Rotation (LR Rotation)
4. Right-Left Rotation (RL Rotation)

AVL Tree Rotations-

Cases Of Imbalance And Their Balancing Using Rotation Operations-

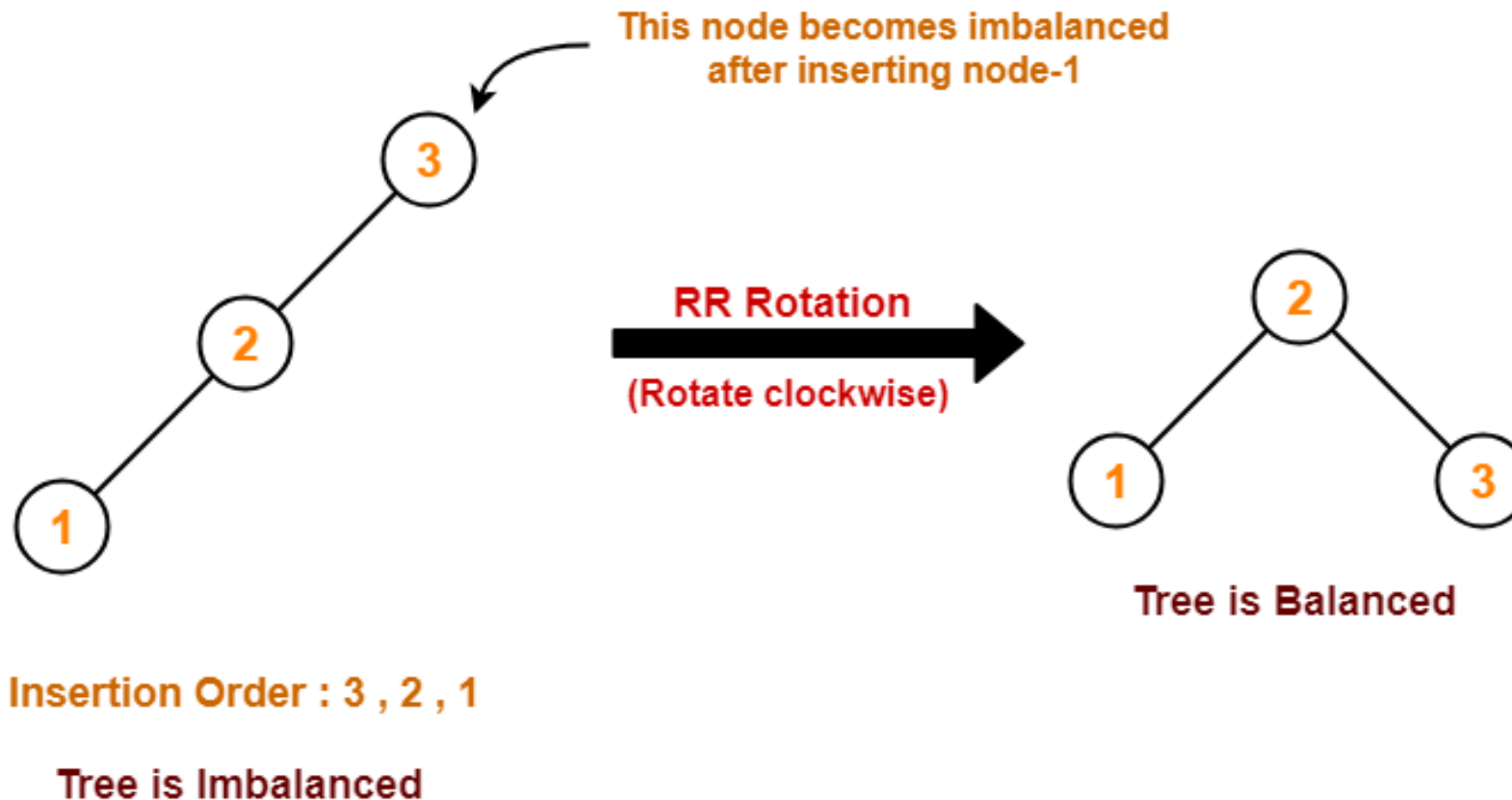
Case-01:



AVL Tree Rotations-

Cases Of Imbalance And Their Balancing Using Rotation Operations-

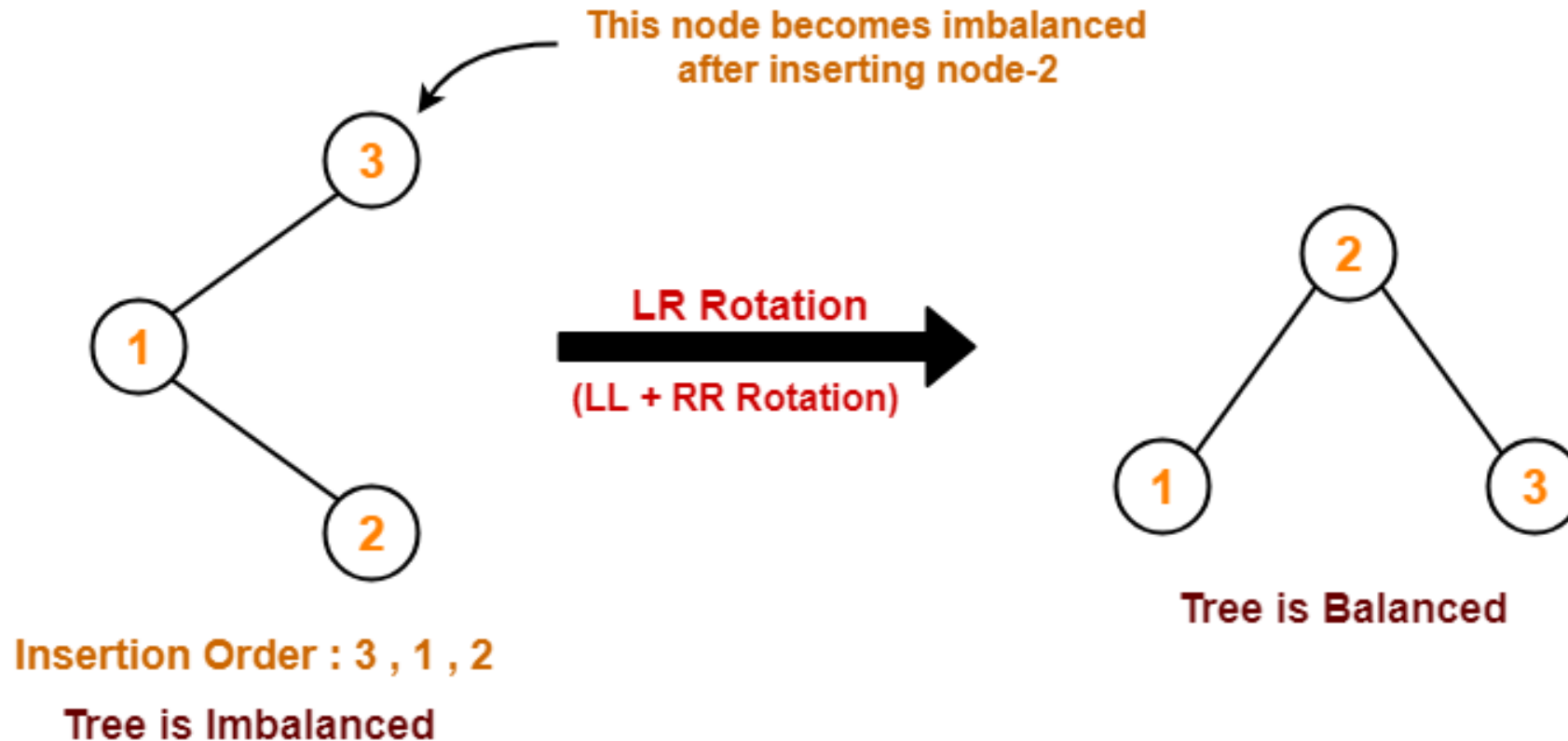
Case-02:



AVL Tree Rotations-

Cases Of Imbalance And Their Balancing Using Rotation Operations-

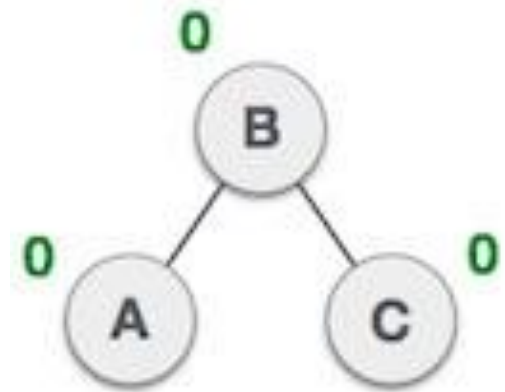
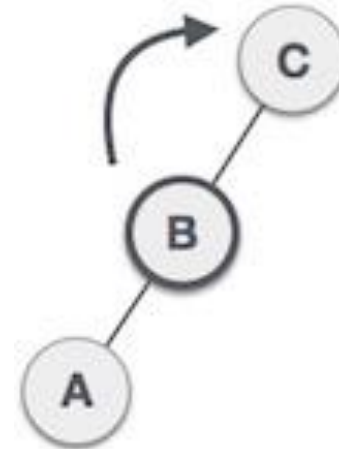
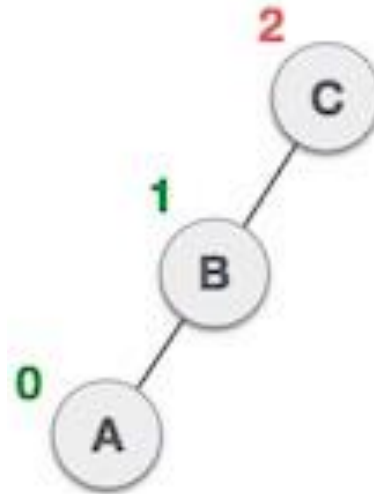
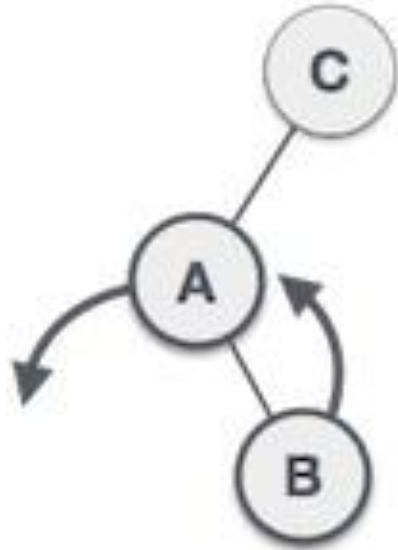
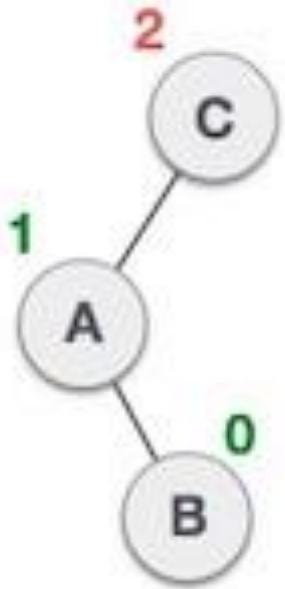
Case-03:



AVL Tree Rotations-

Cases Of Imbalance And Their Balancing Using Rotation Operations-

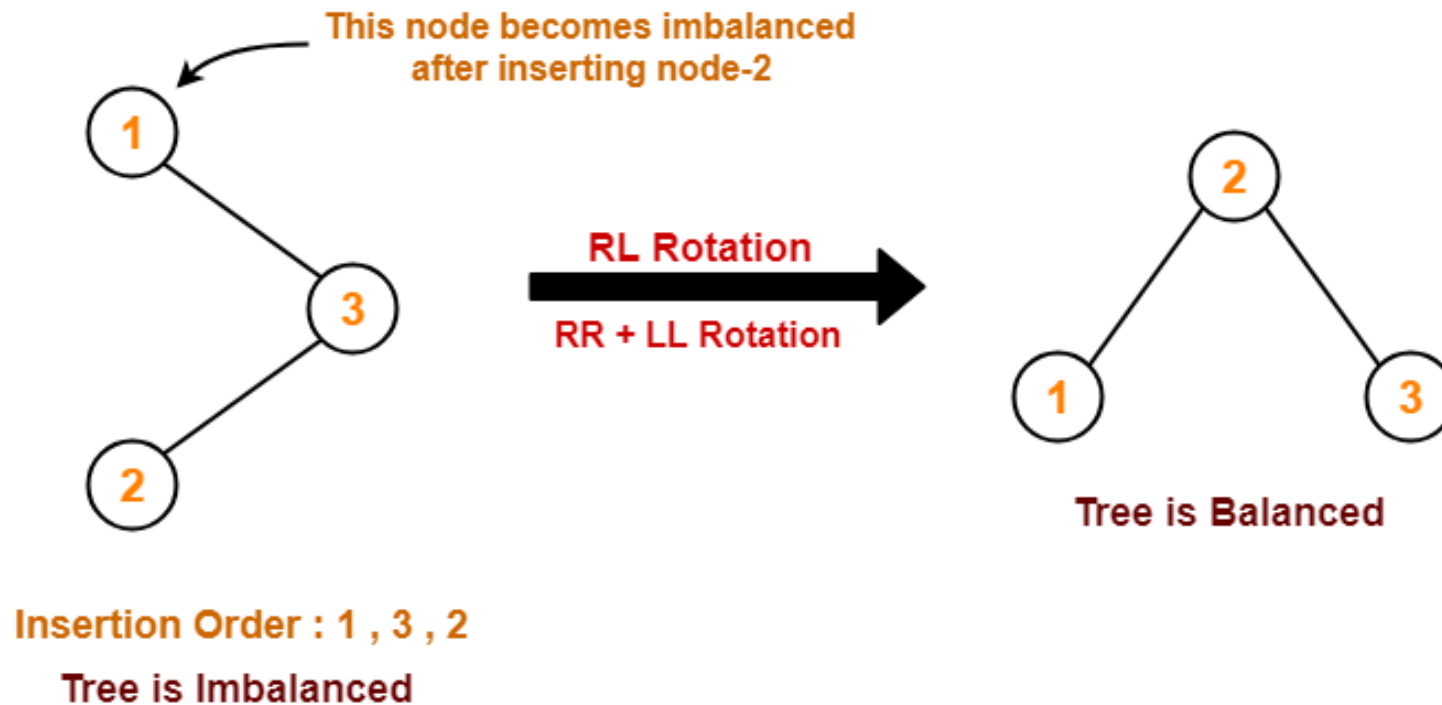
Case-03:



AVL Tree Rotations-

Cases Of Imbalance And Their Balancing Using Rotation Operations-

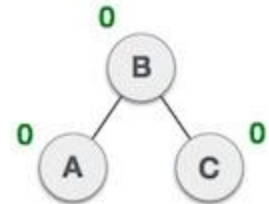
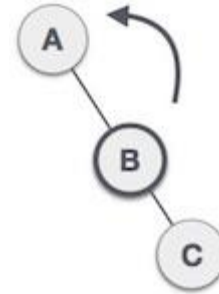
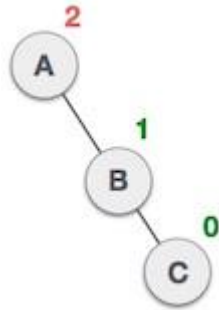
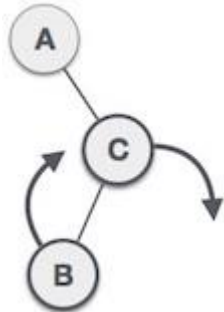
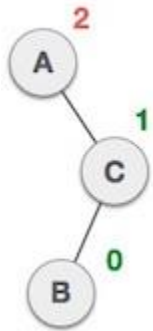
Case-04:



AVL Tree Rotations-

Cases Of Imbalance And Their Balancing Using Rotation Operations-

Case-04:



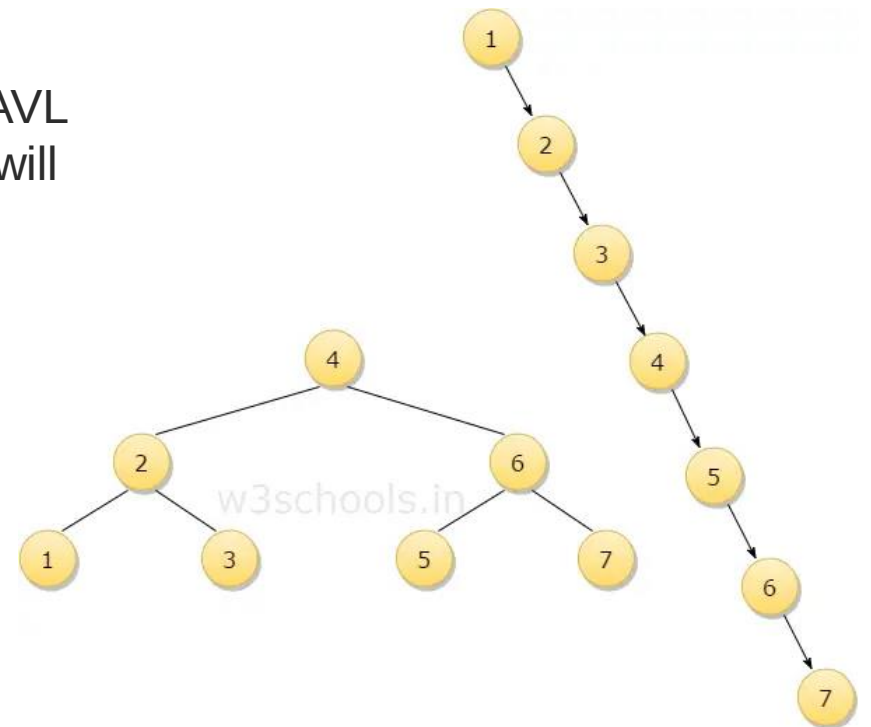
Advantages of AVL Tree

Since AVL trees are height balance trees, operations like insertion and deletion have low time complexity. Let us consider an example:
If you have the following tree having keys 1, 2, 3, 4, 5, 6, 7 and then the binary tree will be like the second figure:

To insert a node with a key Q in the binary tree, the algorithm requires seven comparisons, but if you insert the same key in AVL tree, from the above 1st figure, you can see that the algorithm will require three comparisons.

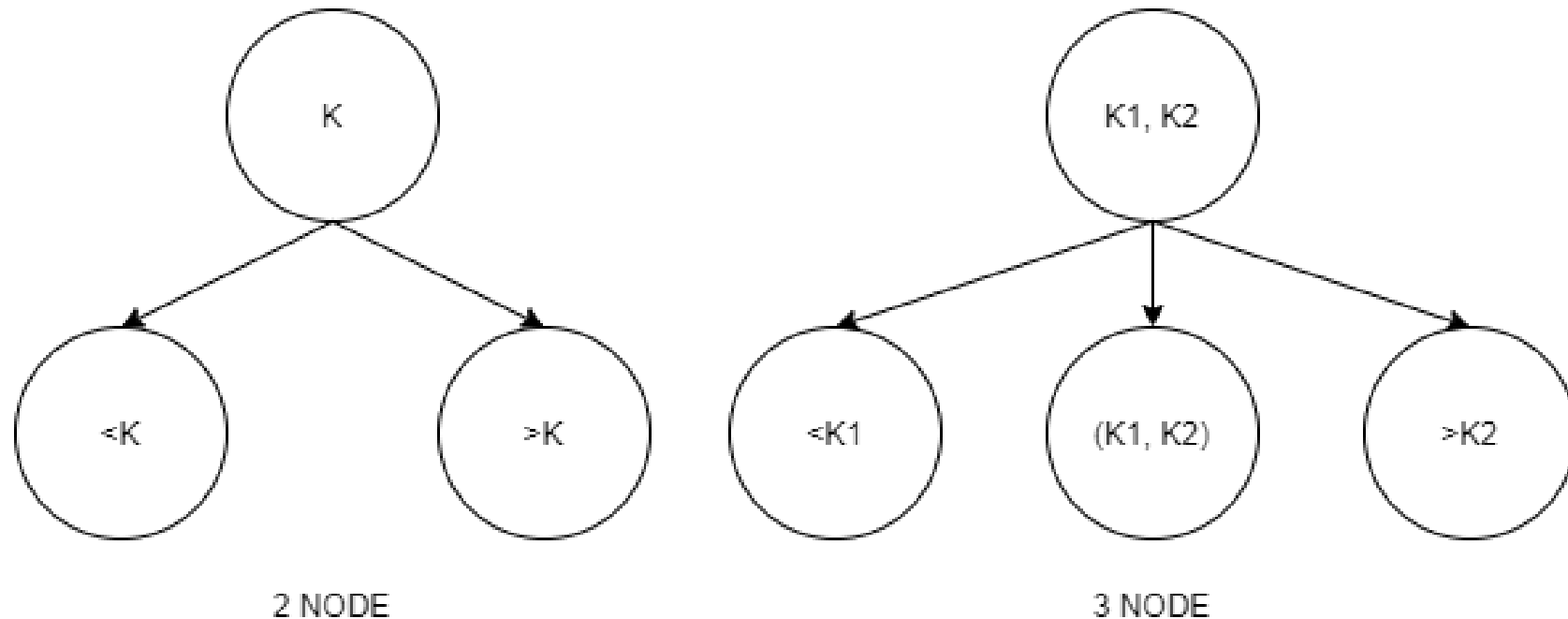
Representation of AVL tree

```
Struct AVLNode
{
    int data;
    struct AVLNode *left, *right;
    int balfactor;
};
```



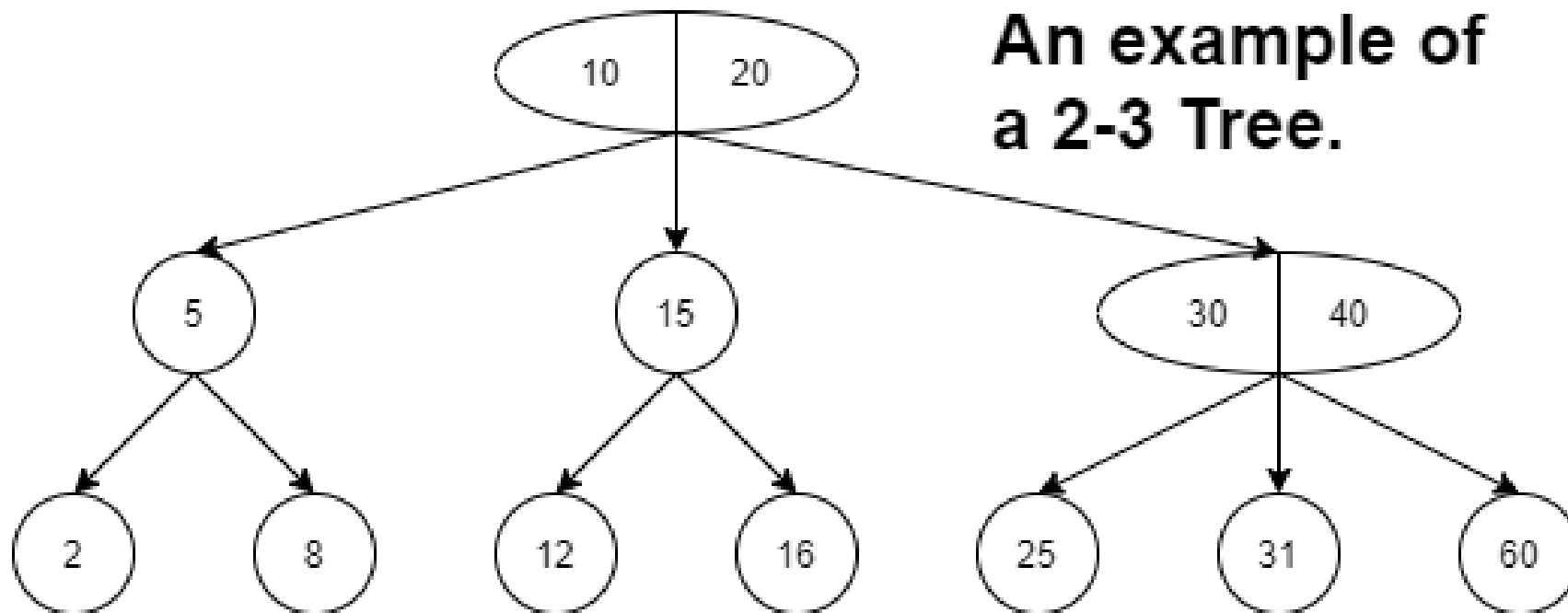
2-3 Tree

A **2-3 Tree** is a tree data structure where every node with children has either two children and one data element or three children and two data elements. A node with 2 children is called a **2-NODE** and a node with 3 children is called a **3-NODE**.



2-3 Tree

Nodes on the outside of the tree i.e. the leaves have no children and have either one or two data elements. All its leaves must be on the same level so that a 2-3 tree is **always height balanced**. A 2-3 tree is a special case of a **B-Tree of order 3**. Below is an example of a 2-3 tree.



2-3 Tree

Properties of 2-3 Trees

A 2-3 tree follows the below mentioned properties.

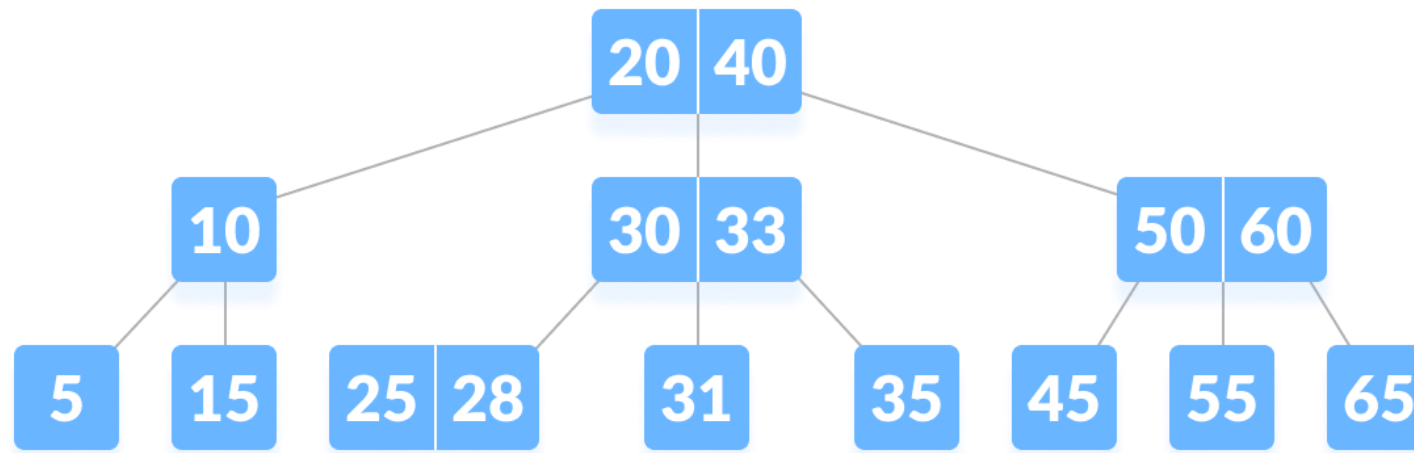
1. Every internal node in the tree is a **2-node** or a **3-node** i.e it has either one value or two values.
2. A node with one value is either a leaf node or has exactly two children. **Values in left sub tree < value in node < values in right sub tree.**
3. A node with two values is either a leaf node or has exactly 3 children. It cannot have 2 children. **Values in left sub tree < first value in node < values in middle sub tree < second value in node < value in right sub tree.**
4. All leaf nodes are at the same level.

You can notice that the previous example given satisfies all the above mentioned properties. It is important to note from the above properties that all data in a 2-3 tree is stored in a sorted manner which makes search operations fast and effective.

B Tree

In search trees like binary search tree, AVL Tree, etc., every node contains only one value (key) and a maximum of two children. But there is a special type of search tree called B-Tree in which a node contains more than one value (key) and more than two children. B-Tree was developed in the year 1972 by **Bayer and McCreight** with the name ***Height Balanced m-way Search Tree***. Later it was named as B-Tree.

B-Tree is a self-balanced search tree in which every node contains multiple keys and has more than two children.



It is also known as a height-balanced m-way tree.

B Tree

Here, the number of keys in a node and number of children for a node depends on the order of B-Tree. Every B-Tree has an order.

B-Tree of Order m has the following properties...

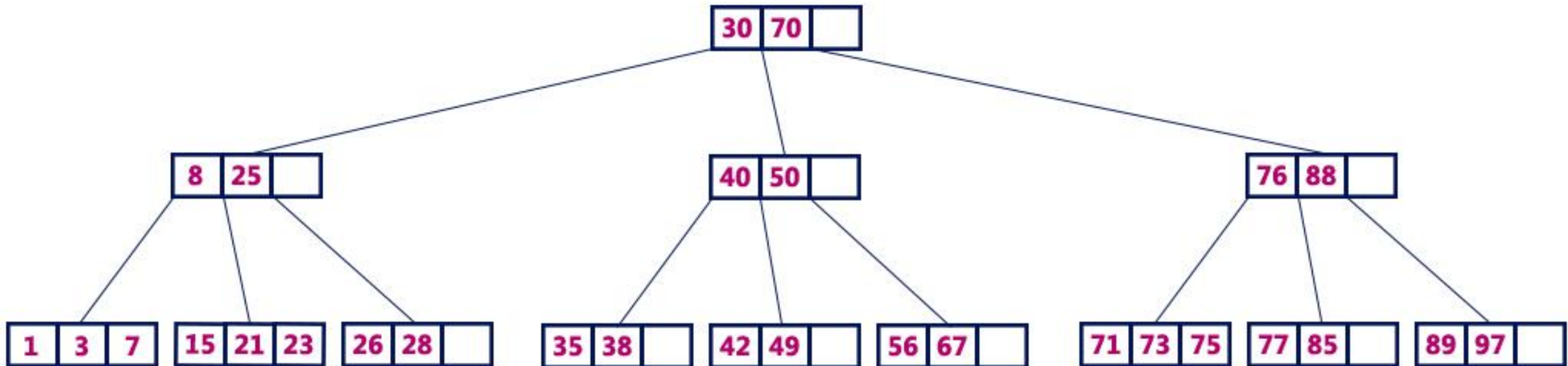
- Property #1** - All **leaf nodes** must be **at same level**.
- Property #2** - All nodes except root must have at least $\lceil m/2 \rceil - 1$ keys and maximum of $m-1$ keys.
- Property #3** - All non leaf nodes except root (i.e. all internal nodes) must have at least $m/2$ children.
- Property #4** - If the root node is a non leaf node, then it must have **atleast 2** children.
- Property #5** - A non leaf node with $n-1$ keys must have n number of children.
- Property #6** - All the **key values in a node** must be in **Ascending Order**.

B Tree

For example, B-Tree of Order 4 contains a maximum of 3 key values in a node and maximum of 4 children for a node.

Example

B-Tree of Order 4



Graph Vs. Tree

The basis of Comparison

Graph

Tree

Definition

Graph is a non-linear data structure.

Tree is a non-linear data structure.

Structure

It is a collection of vertices/nodes and edges.

It is a collection of nodes and edges.

Edges

Each node can have any number of edges.

If there is n nodes then there would be $n-1$ number of edges

Types of Edges

They can be directed or undirected

They are always directed

Root node

There is no unique node called root in graph.

There is a unique node called root(parent) node in trees.

Loop Formation

A cycle can be formed.

There will not be any cycle.

Traversal

For graph traversal, we use Breadth-First Search (BFS), and Depth-First Search (DFS).

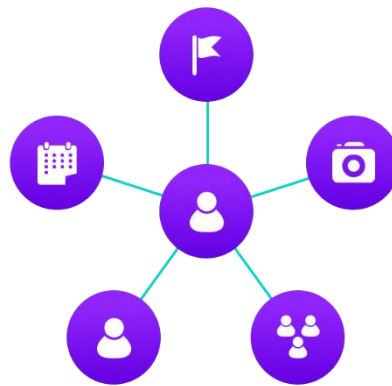
We traverse a tree using in-order, pre-order, or post-order traversal methods.

Graph

A graph data structure is a collection of nodes that have data and are connected to other nodes.

Definition: A graph G can be defined as an ordered set $G(V, E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.

Let's try to understand this through an example. On facebook, everything is a node. That includes User, Photo, Album, Event, Group, Page, Comment, Story, Video, Link, Note...anything that has data is a node. Every relationship is an edge from one node to another. Whether you post a photo, join a group, like a page, etc., a new edge is created for that relationship.

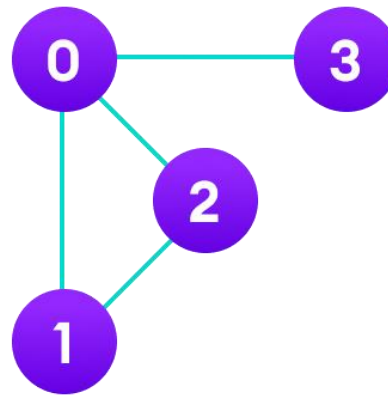


Example of graph data structure

Graph

More precisely, a graph is a data structure (V, E) that consists of

- A collection of vertices V
- A collection of edges E , represented as ordered pairs of vertices (u,v)



Vertices and edges

$$V = \{0, 1, 2, 3\}$$

$$E = \{(0,1), (0,2), (0,3), (1,2)\}$$

$$G = \{V, E\}$$

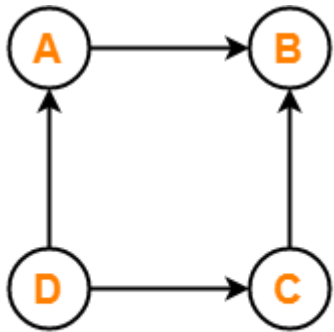
Graph

Graph Terminology

- **Adjacency:** A vertex is said to be adjacent to another vertex if there is an edge connecting them. Vertices 2 and 3 are not adjacent because there is no edge between them.
- **Path:** A sequence of edges that allows you to go from vertex A to vertex B is called a path. 0-1, 1-2 and 0-2 are paths from vertex 0 to vertex 2.

Directed Graph

- A graph in which all the edges are directed is called as a directed graph.
- In other words, all the edges of a directed graph contain some direction.
- Directed graphs are also called as **digraphs**.



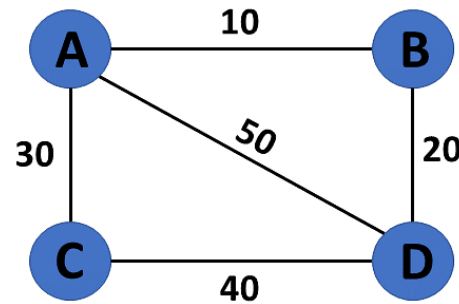
Example of Directed Graph

Here,

- This graph consists of four vertices and four directed edges.
- Since all the edges are directed, therefore it is a directed graph.

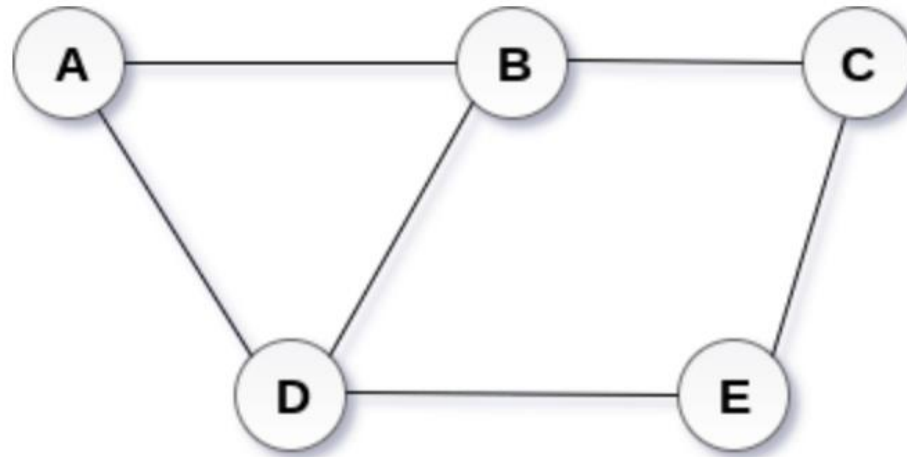
Weighted Graph

A graph $G = (V, E)$ is called a labeled or weighted graph because each edge has a value or weight representing the cost of traversing that edge.



Graph - Types

- **Undirected Graph:** In an undirected graph, edges are not associated with the directions with them.



Undirected Graph

Graph Terminology

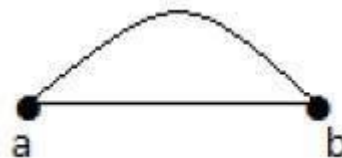
- **Path** : A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U .
- **Closed Path**: A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if $V_0 = V_N$.
- **Simple Path**: If all the nodes of the graph are distinct with an exception $V_0 = V_N$, then such path P is called as closed simple path.

Graph Terminology

- **Cycle**: A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.
- **Connected Graph**: A connected graph is the one in which some path exists between every two vertices (u, v) in V . There are no isolated nodes in connected graph.
- **Complete Graph**: A complete graph is the one in which every node is connected with all other nodes. A complete graph contain $n(n-1)/2$ edges where n is the number of nodes in the graph.
- **Weighted Graph**: In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as $w(e)$ which must be a positive (+) value indicating the cost of traversing the edge.

Graph Terminology

- **Adjacent Nodes**: If two nodes u and v are connected via an edge e , then the nodes u and v are called as neighbors or adjacent nodes.
- **Degree of the Node**: A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.
- **Parallel Edges**: In a graph, if a pair of vertices is connected by more than one edge, then those edges are called parallel edges. In the above graph, 'a' and 'b' are the two vertices which are connected by two edges 'ab' and 'ab' between them. So it is called as a parallel edge.



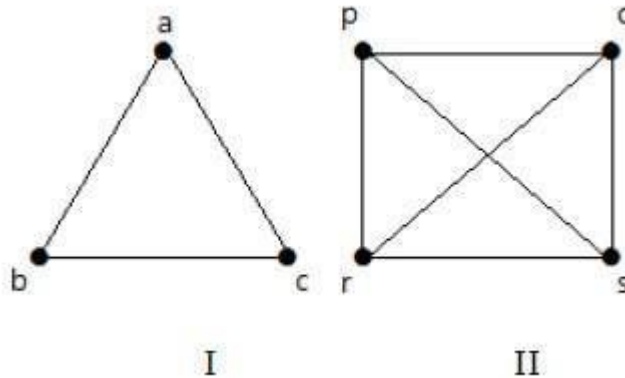
Graph Terminology

- **Simple Graph**
- A graph **with no loops** and no **parallel edges** is called a simple graph.

Graph Terminology

- Complete Graph

- In the graph, a **vertex should have edges with all other vertices**, then it called a complete graph.
- In other words, if a vertex is connected to all other vertices in a graph, then it is called a complete graph.
- Example
- In the following graphs, each vertex in the graph is connected with all the remaining vertices in the graph except by itself.

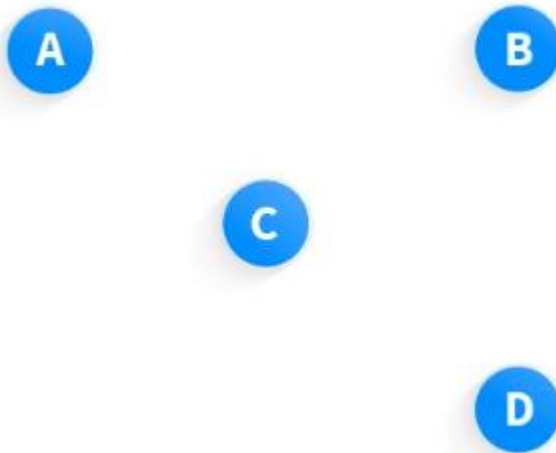


Graph Terminology

Null Graph

A null graph $G=(V,E)$ is one which has only isolated vertices, ie. such a graph does not have any edges connecting the vertices. An example of such a graph would be importing an item from a foreign country to your place when there is no transport connecting the places.

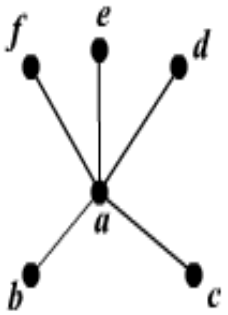
If we consider the 2 locations to be vertices, then due to no route between them, there would be no edges connecting them.



Graph Terminology

- **Degree of a Vertex**

In [graph theory](#), the **degree** of a vertex is the number of edges connecting it. In the example below, vertex *a* has degree 5, and the rest have degree 1. A vertex with degree 1 is called an "end vertex" (you can see why).



Graph Terminology

Pendent Vertex

By using degree of a vertex, we have a two special types of vertices. A vertex with degree one is called a pendent vertex.

Example



Here, in this example, vertex 'a' and vertex 'b' have a connected edge 'ab'. So with respect to the vertex 'a', there is only one edge towards vertex 'b' and similarly with respect to the vertex 'b', there is only one edge towards vertex 'a'. Finally, vertex 'a' and vertex 'b' has degree as one which are also called as the pendent vertex.

Isolated Vertex

A vertex with degree zero is called an isolated vertex.

Example



Here, the vertex 'a' and vertex 'b' has a no connectivity between each other and also to any other vertices. So the degree of both the vertices 'a' and 'b' are zero. These are also called as isolated vertices.

Graph Terminology

- **Degree of a Vertex**

Degree of Vertex in an Undirected Graph

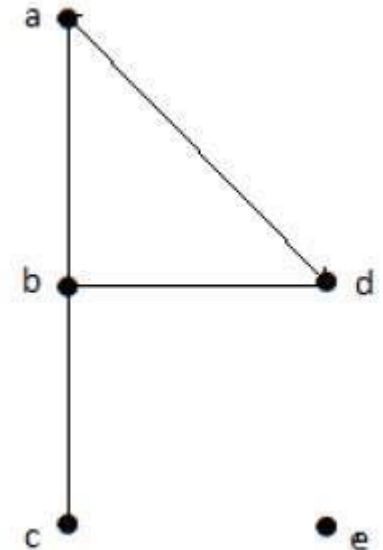
An undirected graph has no directed edges. Consider the following examples.

Example 1

Take a look at the following graph –

In the above Undirected Graph,

- $\deg(a) = 2$, as there are 2 edges meeting at vertex 'a'.
- $\deg(b) = 3$, as there are 3 edges meeting at vertex 'b'.
- $\deg(c) = 1$, as there is 1 edge formed at vertex 'c'
- So 'c' is a **pendent vertex**.
- $\deg(d) = 2$, as there are 2 edges meeting at vertex 'd'.
- $\deg(e) = 0$, as there are 0 edges formed at vertex 'e'.
- So 'e' is an **isolated vertex**.



Graph Terminology

- **Degree of a Vertex**

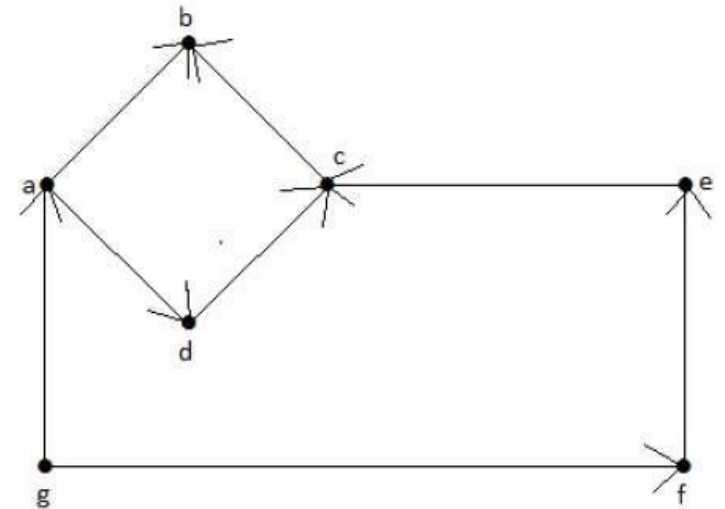
Degree of Vertex in a Directed Graph

In a directed graph, each vertex has an **indegree** and an **outdegree**.

Example 1

Take a look at the following directed graph. Vertex 'a' has two edges, 'ad' and 'ab', which are going outwards. Hence its outdegree is 2. Similarly, there is an edge 'ga', coming towards vertex 'a'. Hence the indegree of 'a' is 1.

Vertex	Indegree	Outdegree
a	1	2
b	2	0
c	2	1
d	1	1
e	1	1
f	1	1
g	0	2



Graph Representation

- **Representing Graphs**
- A graph can be represented using 3 data structures- **adjacency matrix, adjacency list and adjacency set.**
- An **adjacency matrix** can be thought of as a table with rows and columns. The row labels and column labels represent the nodes of a graph. An adjacency matrix is a square matrix where the number of rows, columns and nodes are the same. Each cell of the matrix represents an edge or the relationship between two given nodes. For example, adjacency matrix **A_{ij}** represents the number of links from i to j , given two nodes i and j .

Graph Representation

- **Representing Graphs**

The adjacency matrix for a directed graph is shown in Fig 3. Observe that it is a square matrix in which the number of rows, columns and nodes remain the same (5 in this case). Each row and column correspond to a node or a vertex of a graph. The cells within the matrix represent the connection that exists between nodes. Since, in the given directed graph, no node is connected to itself, all cells lying on the diagonal of the matrix are marked zero. For the rest of the cells, if there exists a directed edge from a given node to another, then the corresponding cell will be marked one else zero.

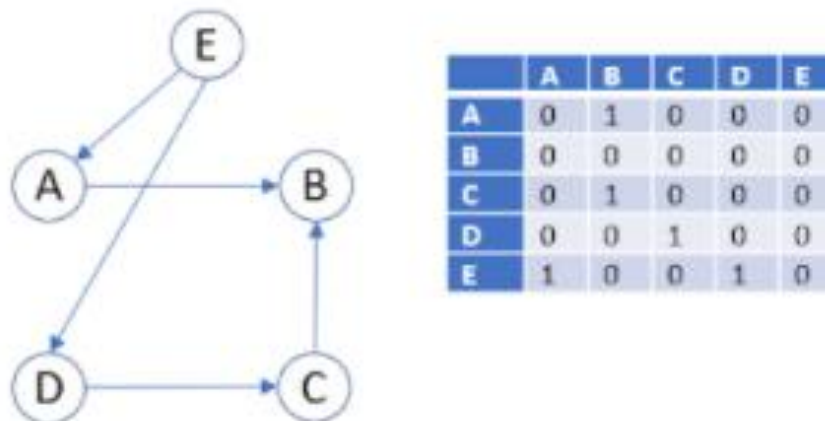


Fig 3: Adjacency Matrix for a directed graph

Graph Representation

- **Representing Graphs**

To understand how an undirected graph can be represented using an adjacency matrix, consider a small undirected graph with five vertices (Fig 4). Here, A is connected to B, but B is connected to A as well. Hence, both the cells i.e., the one with source A destination B and the other one with source B destination A are marked one. This suffices the requirement of an undirected edge. Observe that the second entry is at a mirrored location across the main diagonal.

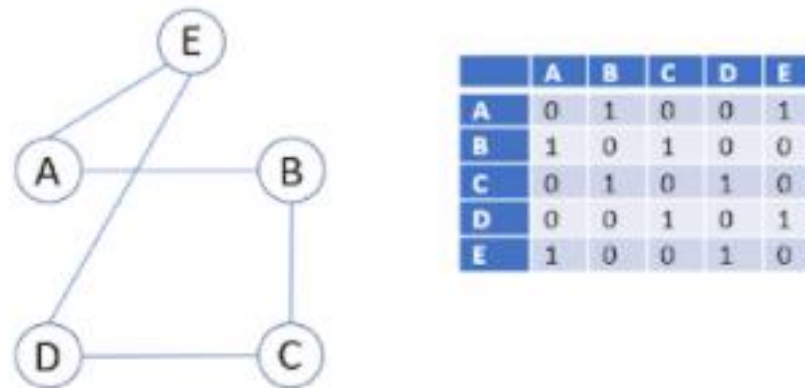


Fig 4: Adjacency Matrix for an undirected graph

Graph Representation

- In **adjacency list** representation of a graph, every vertex is represented as a node object. The node may either contain data or a reference to a linked list. This linked list provides a list of all nodes that are adjacent to the current node. Consider a graph containing an edge connecting node A and node B. Then, the node A will be available in node B's linked list. Fig 6 shows a sample graph of 5 nodes and its corresponding adjacency list.

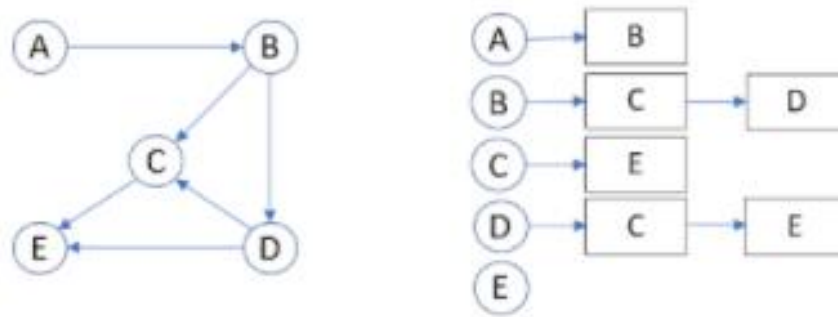


Fig 6: Adjacency list for a directed graph

Graph Representation

- Similarly, adjacency lists for an undirected graph can also be constructed. Fig 7 provides an example of an undirected graph along with its adjacency list for better understanding.

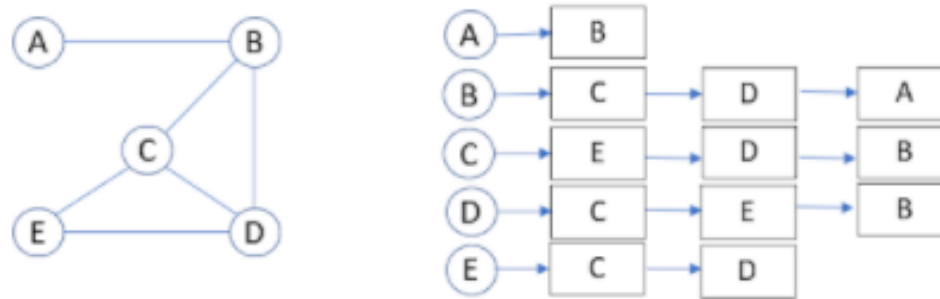


Fig 7: Adjacency list for an undirected graph

Adjacency list enables faster search process in comparison to adjacency matrix.

Graph - Traversals

- Graph traversal is a technique used for searching a vertex in a graph. The graph traversal is also used to decide the order of vertices is visited in the search process. A graph traversal finds the edges to be used in the search process without creating loops. That means using graph traversal we visit all the vertices of the graph without getting into looping path.

There are two graph traversal techniques and they are as follows...

- **DFS (Depth First Search)**
- **BFS (Breadth First Search)**

Graph - Traversals

- **BFS (Breadth First Search)**
- BFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops. We use **Queue data structure** with maximum size of total number of vertices in the graph to implement BFS traversal.

We use the following steps to implement BFS traversal...

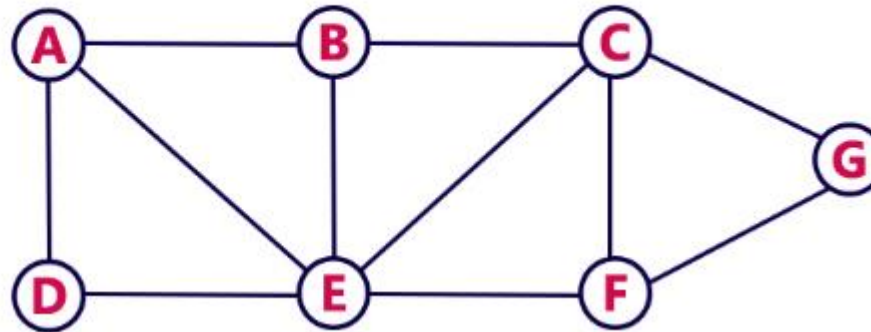
- **Step 1** - Define a Queue of size total number of vertices in the graph.
- **Step 2** - Select any vertex as **starting point** for traversal. Visit that vertex and insert it into the Queue.
- **Step 3** - Visit all the non-visited **adjacent** vertices of the vertex which is at front of the Queue and insert them into the Queue.
- **Step 4** - When there is no new vertex to be visited from the vertex which is at front of the Queue then delete that vertex.
- **Step 5** - Repeat steps 3 and 4 until queue becomes empty.
- **Step 6** - When queue becomes empty, then produce final spanning tree by removing unused edges from the graph

Graph - Traversals

- **BFS (Breadth First Search)**

Example

Consider the following example graph to perform BFS traversal

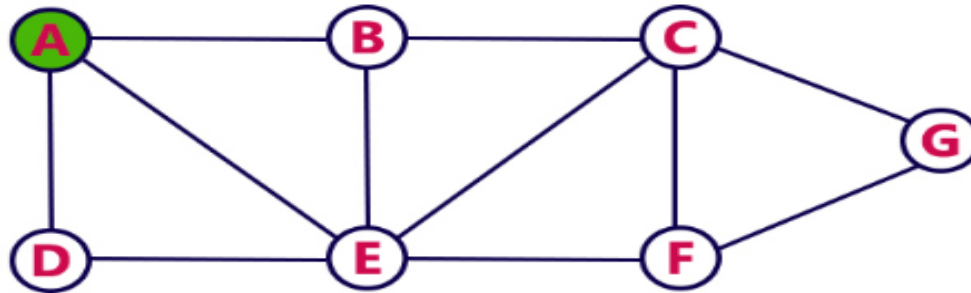


Graph - Traversals

- BFS Example

Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Insert **A** into the Queue.

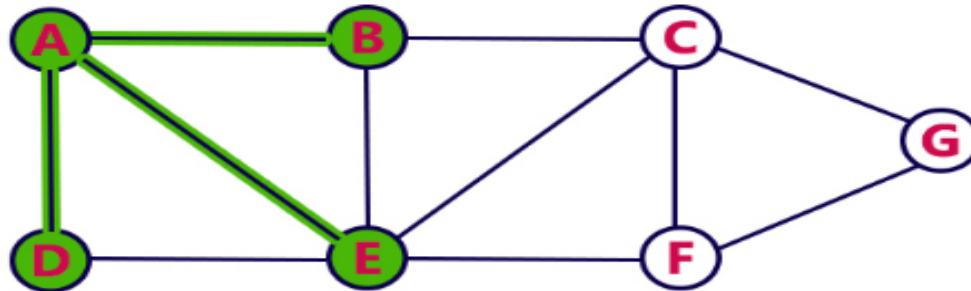


Queue



Step 2:

- Visit all adjacent vertices of **A** which are not visited (**D**, **E**, **B**).
- Insert newly visited vertices into the Queue and delete A from the Queue..



Queue

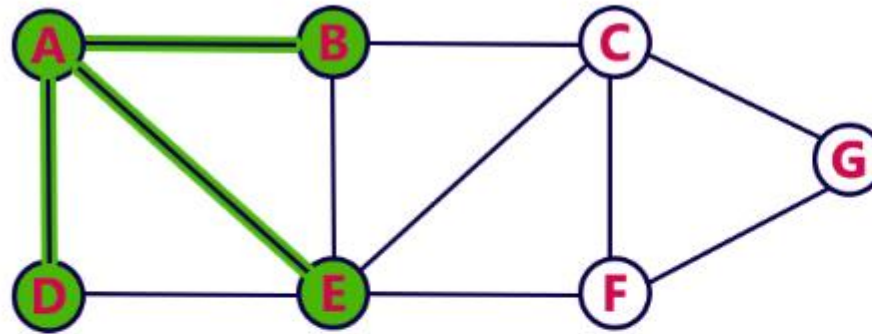


Graph - Traversals

- BFS Example

Step 3:

- Visit all adjacent vertices of **D** which are not visited (there is no vertex).
- Delete D from the Queue.

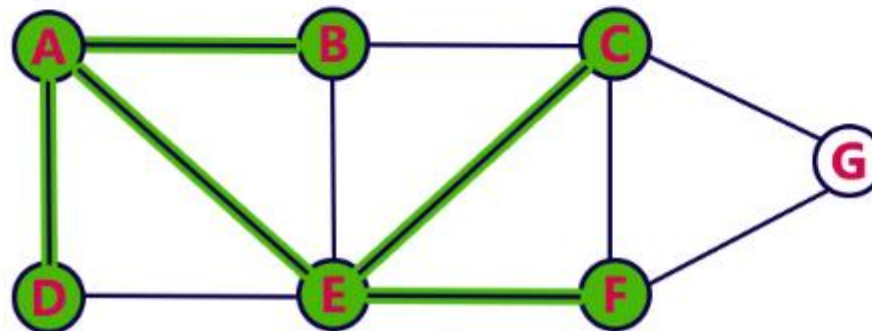


Queue



Step 4:

- Visit all adjacent vertices of **E** which are not visited (**C**, **F**).
- Insert newly visited vertices into the Queue and delete E from the Queue.



Queue

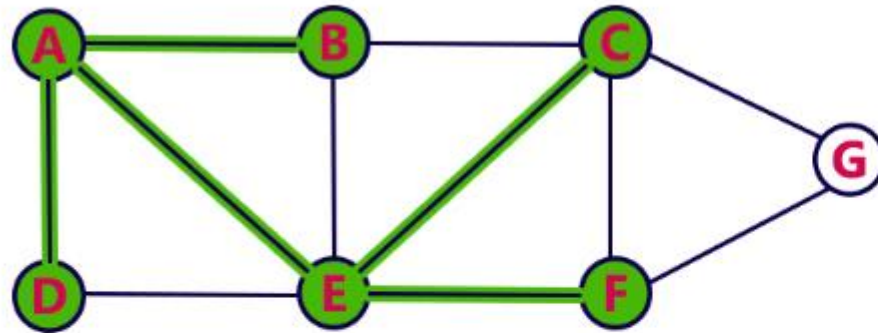


Graph - Traversals

- BFS Example

Step 5:

- Visit all adjacent vertices of **B** which are not visited (**there is no vertex**).
- Delete **B** from the Queue.

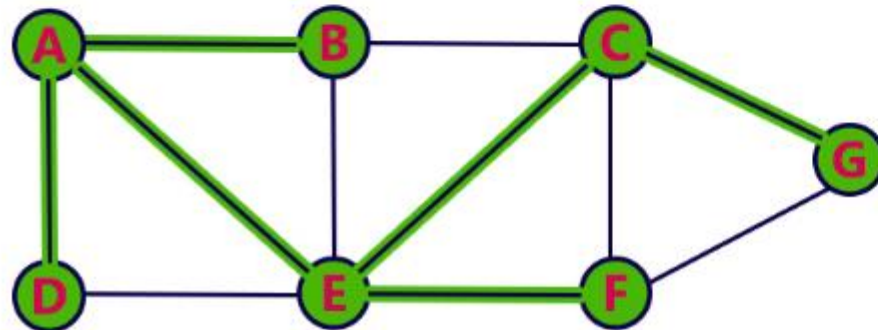


Queue



Step 6:

- Visit all adjacent vertices of **C** which are not visited (**G**).
- Insert newly visited vertex into the Queue and delete **C** from the Queue.



Queue

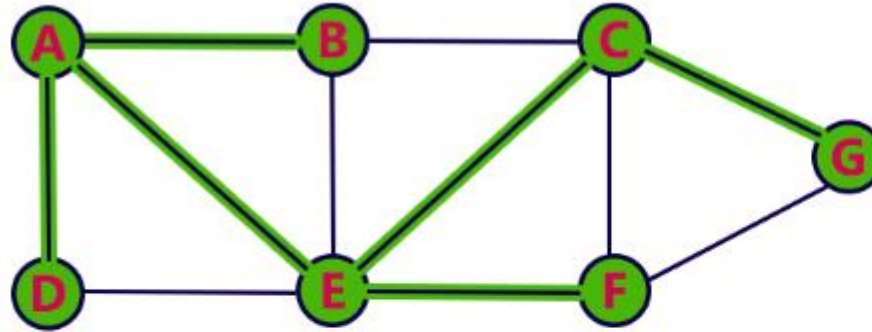


Graph - Traversals

- BFS Example

Step 7:

- Visit all adjacent vertices of **F** which are not visited (**there is no vertex**).
- Delete **F** from the Queue.

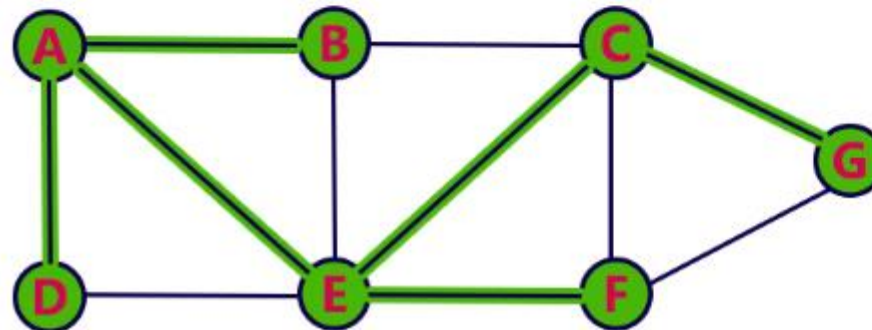


Queue



Step 8:

- Visit all adjacent vertices of **G** which are not visited (**there is no vertex**).
- Delete **G** from the Queue.



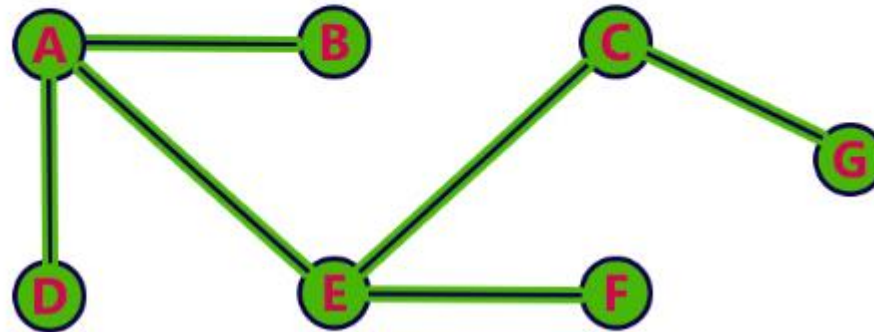
Queue



Graph - Traversals

- BFS Example
- BSF Traversal : ADEBCFG

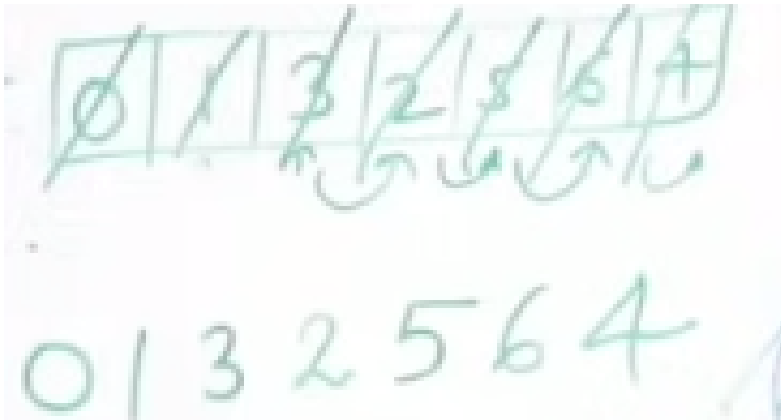
- Queue became Empty. So, stop the BFS process.
- Final result of BFS is a Spanning Tree as shown below...



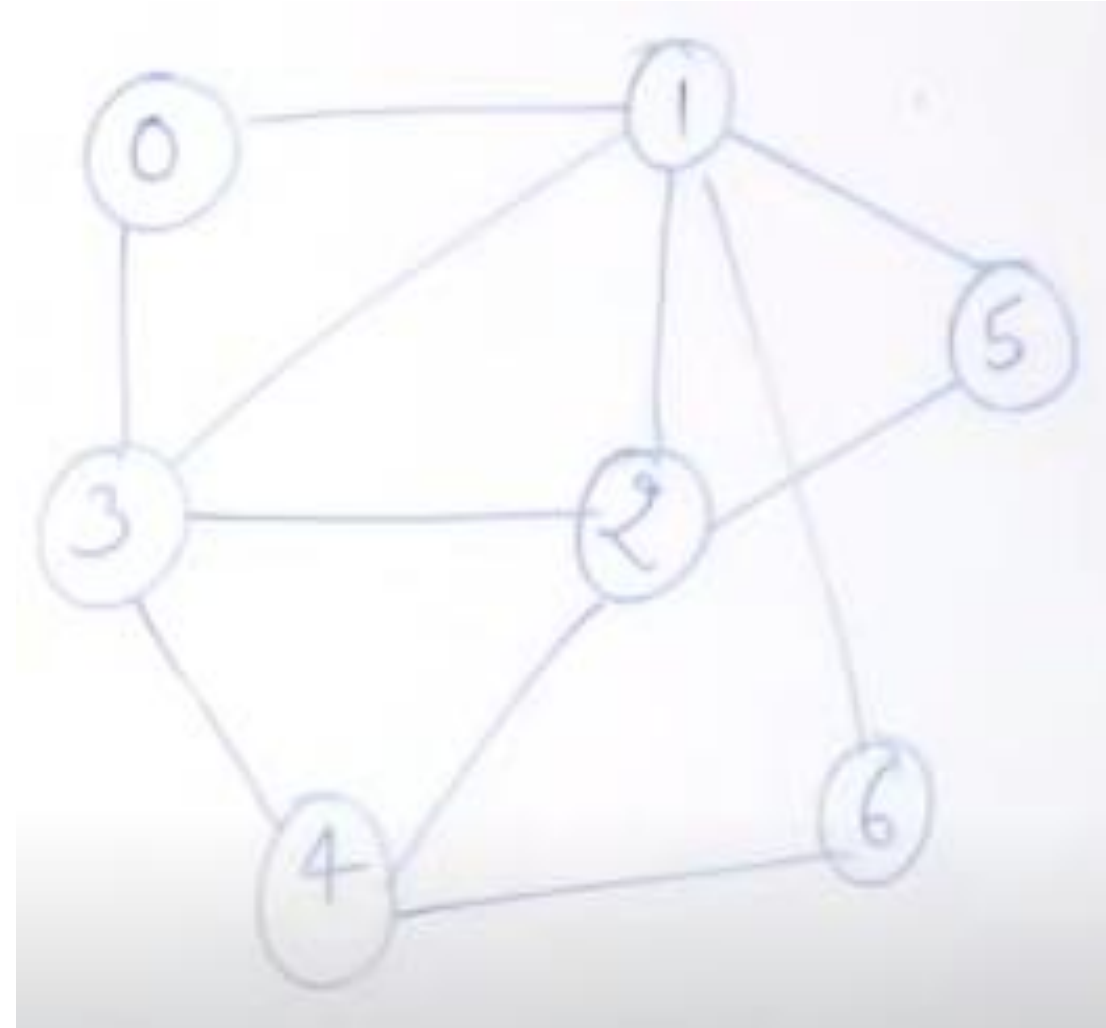
Graph - Traversals

- BFS Example

- Queue :



- Result:



Graph - Traversals

- **DFS (Depth First Search)**
- DFS traversal of a graph produces a **spanning tree** as final result. **Spanning Tree** is a graph without loops. We use **Stack data structure** with maximum size of total number of vertices in the graph to implement DFS traversal.

We use the following steps to implement DFS traversal...

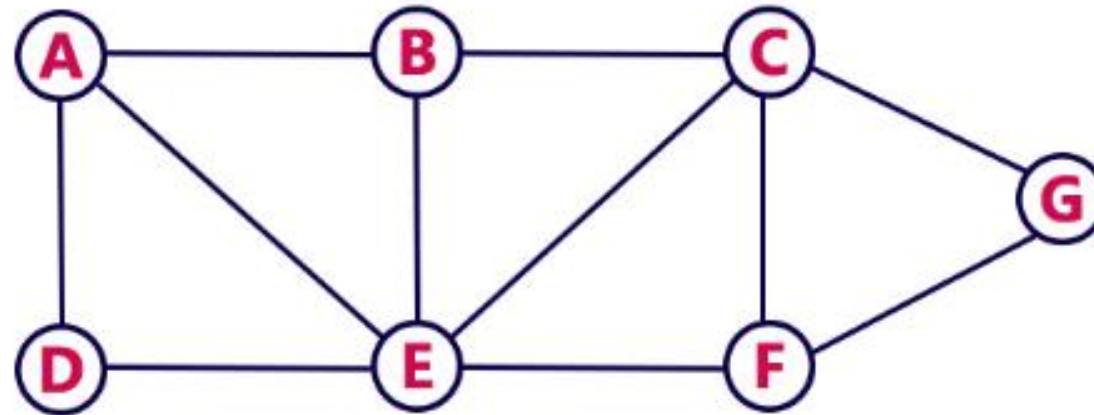
- **Step 1** - Define a Stack of size total number of vertices in the graph.
- **Step 2** - Select any vertex as **starting point** for traversal. Visit that vertex and push it on to the Stack.
- **Step 3** - Visit any one of the non-visited **adjacent** vertices of a vertex which is at the top of stack and push it on to the stack.
- **Step 4** - Repeat step 3 until there is no new vertex to be visited from the vertex which is at the top of the stack.
- **Step 5** - When there is no new vertex to visit then use **back tracking** and pop one vertex from the stack.
- **Step 6** - Repeat steps 3, 4 and 5 until stack becomes Empty.
- **Step 7** - When stack becomes Empty, then produce final spanning tree by removing unused edges from the graph

Back tracking is coming back to the vertex from which we reached the current vertex.

Graph - Traversals

Example

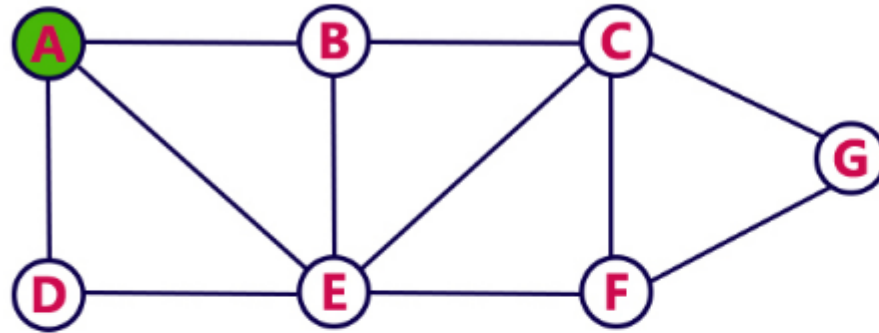
Consider the following example graph to perform DFS traversal



Graph - Traversals

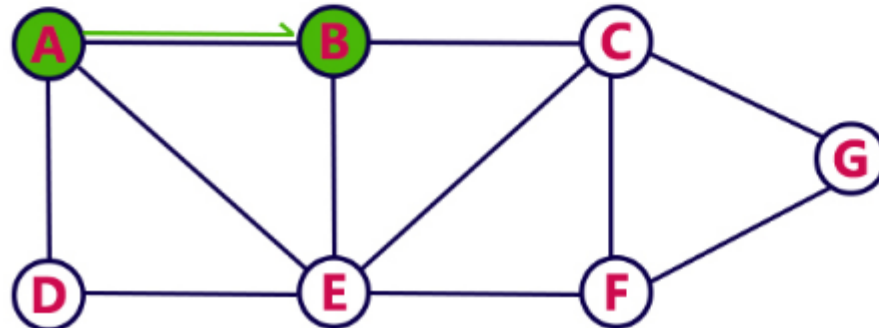
Step 1:

- Select the vertex **A** as starting point (visit **A**).
- Push **A** on to the Stack.



Step 2:

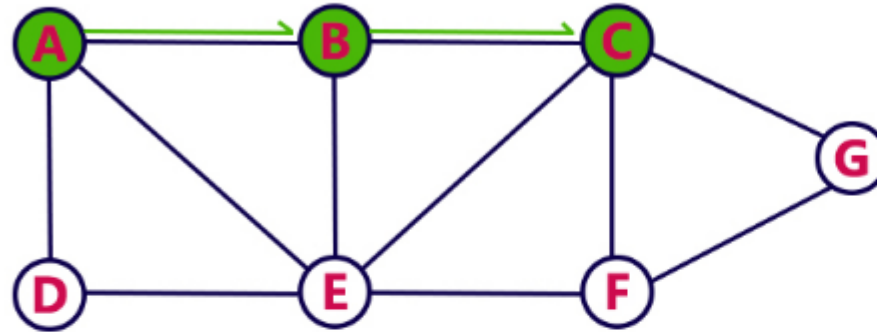
- Visit any adjacent vertex of **A** which is not visited (**B**).
- Push newly visited vertex B on to the Stack.



Graph - Traversals

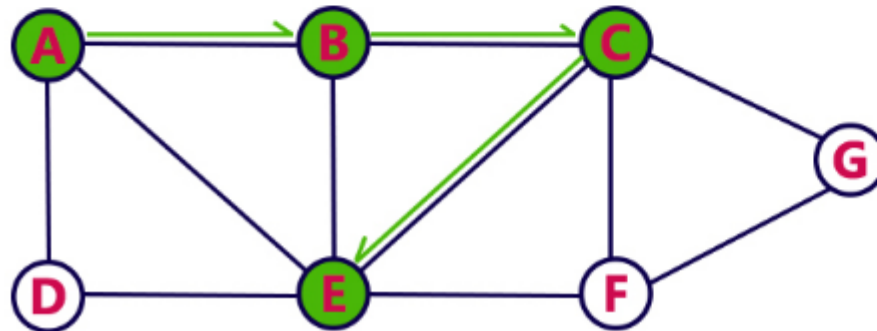
Step 3:

- Visit any adjacent vertex of **B** which is not visited (**C**).
- Push C on to the Stack.



Step 4:

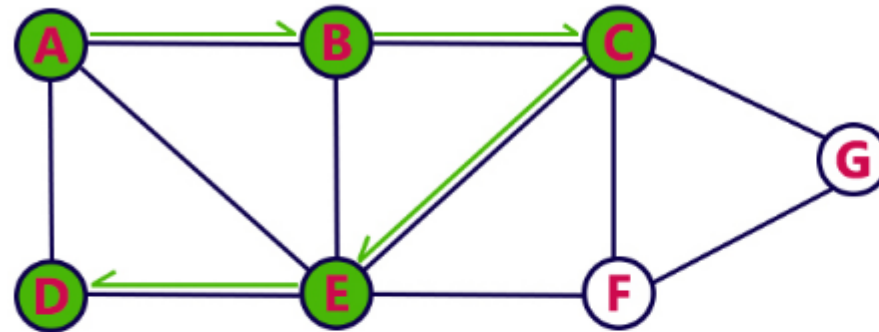
- Visit any adjacent vertex of **C** which is not visited (**E**).
- Push E on to the Stack



Graph - Traversals

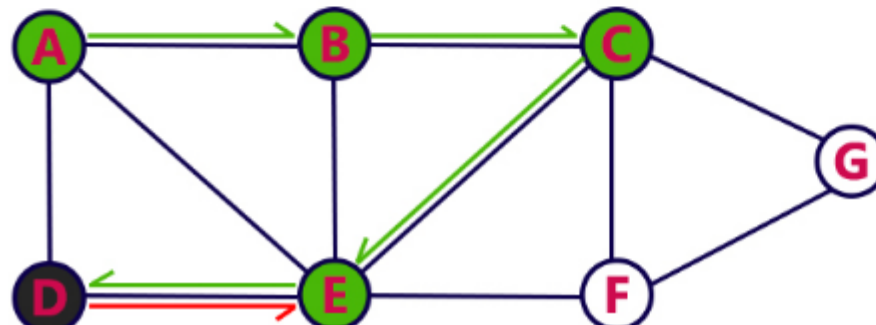
Step 5:

- Visit any adjacent vertex of **E** which is not visited (**D**).
- Push D on to the Stack



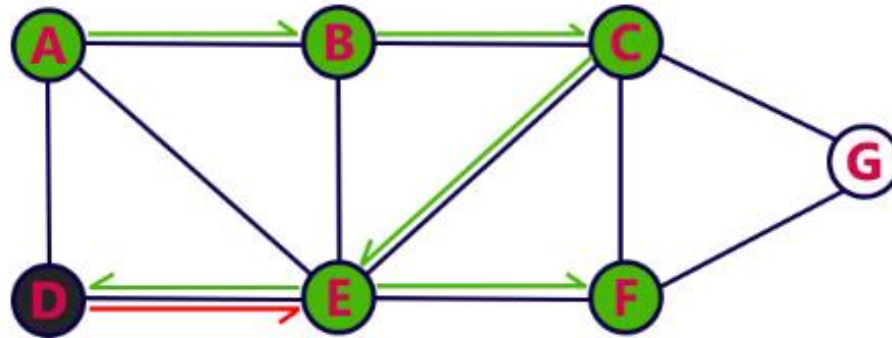
Step 6:

- There is no new vertex to be visited from D. So use back track.
- Pop D from the Stack.



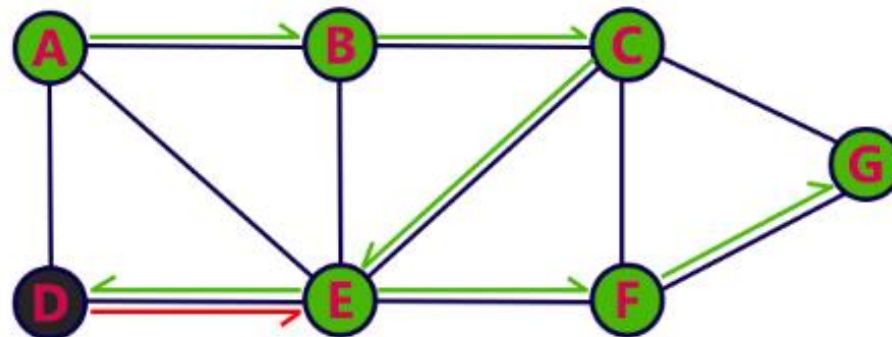
Graph - Traversals

- Visit any adjacent vertex of **E** which is not visited (**F**).
- Push **F** on to the Stack.



Step 8:

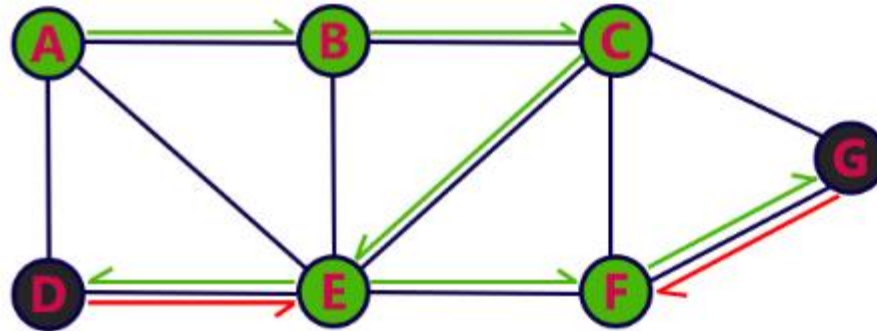
- Visit any adjacent vertex of **F** which is not visited (**G**).
- Push **G** on to the Stack.



Graph - Traversals

Step 9:

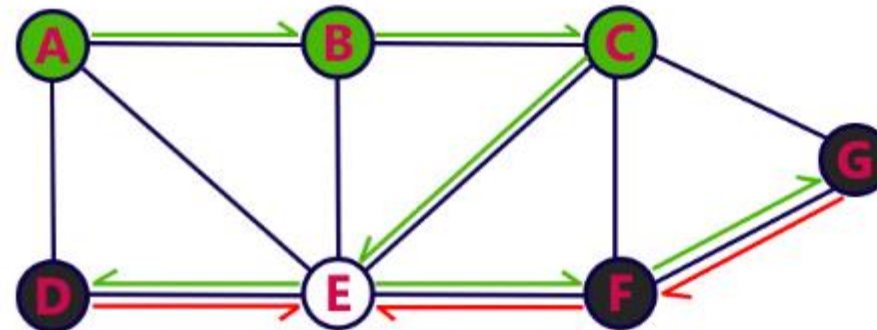
- There is no new vertex to be visited from G. So use back track.
- Pop G from the Stack.



Stack

Step 10:

- There is no new vertex to be visited from F. So use back track.
- Pop F from the Stack.

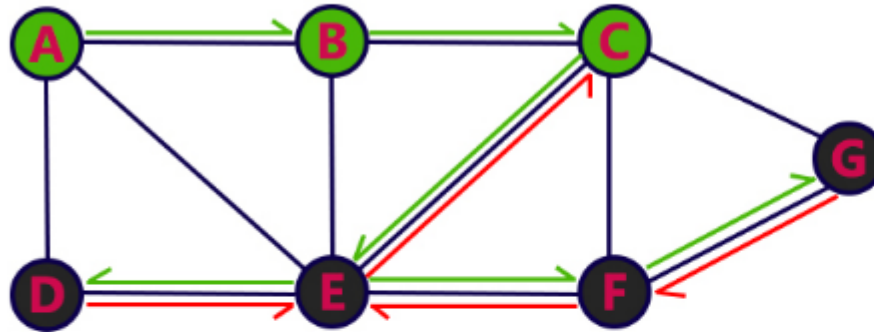


Stack

Graph - Traversals

Step 11:

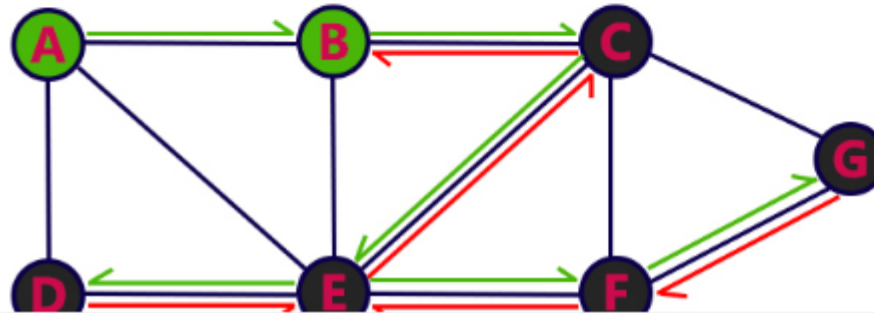
- There is no new vertex to be visited from E. So use back track.
- Pop E from the Stack.



Stack

Step 12:

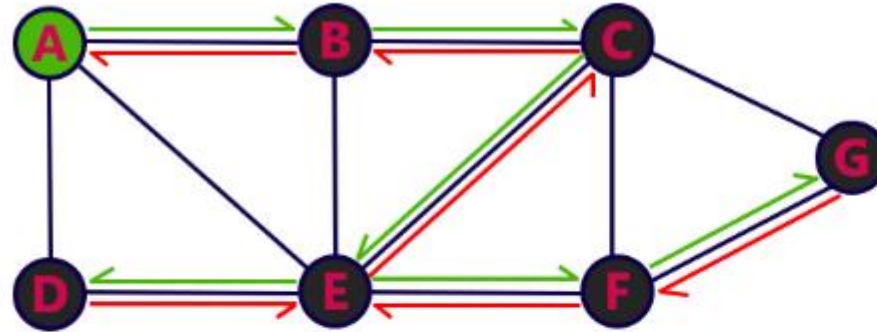
- There is no new vertex to be visited from C. So use back track.
- Pop C from the Stack.



B
A

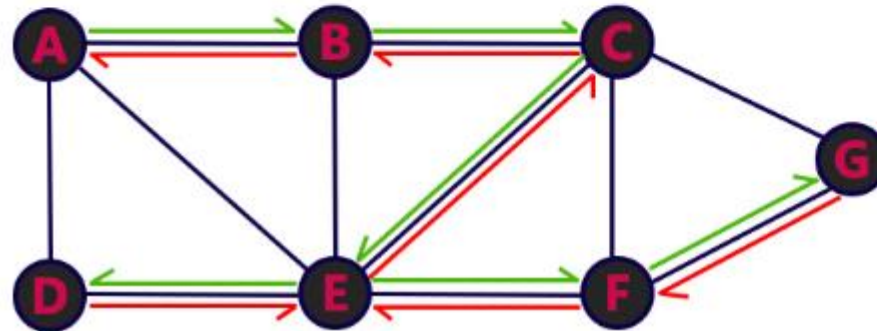
Graph - Traversals

- There is no new vertex to be visited from B. So use back track.
- Pop B from the Stack.



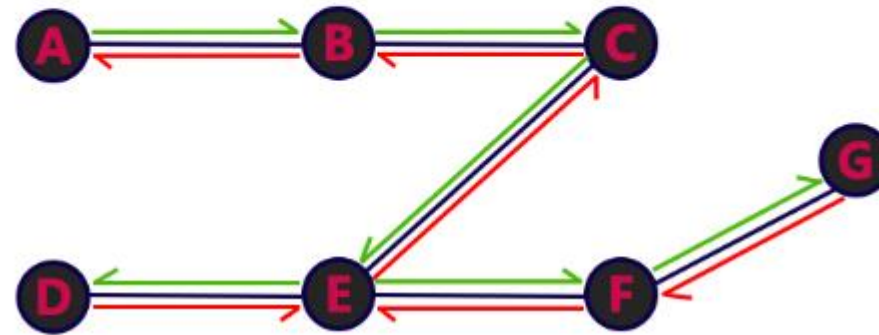
Step 14:

- There is no new vertex to be visited from A. So use back track.
- Pop A from the Stack.

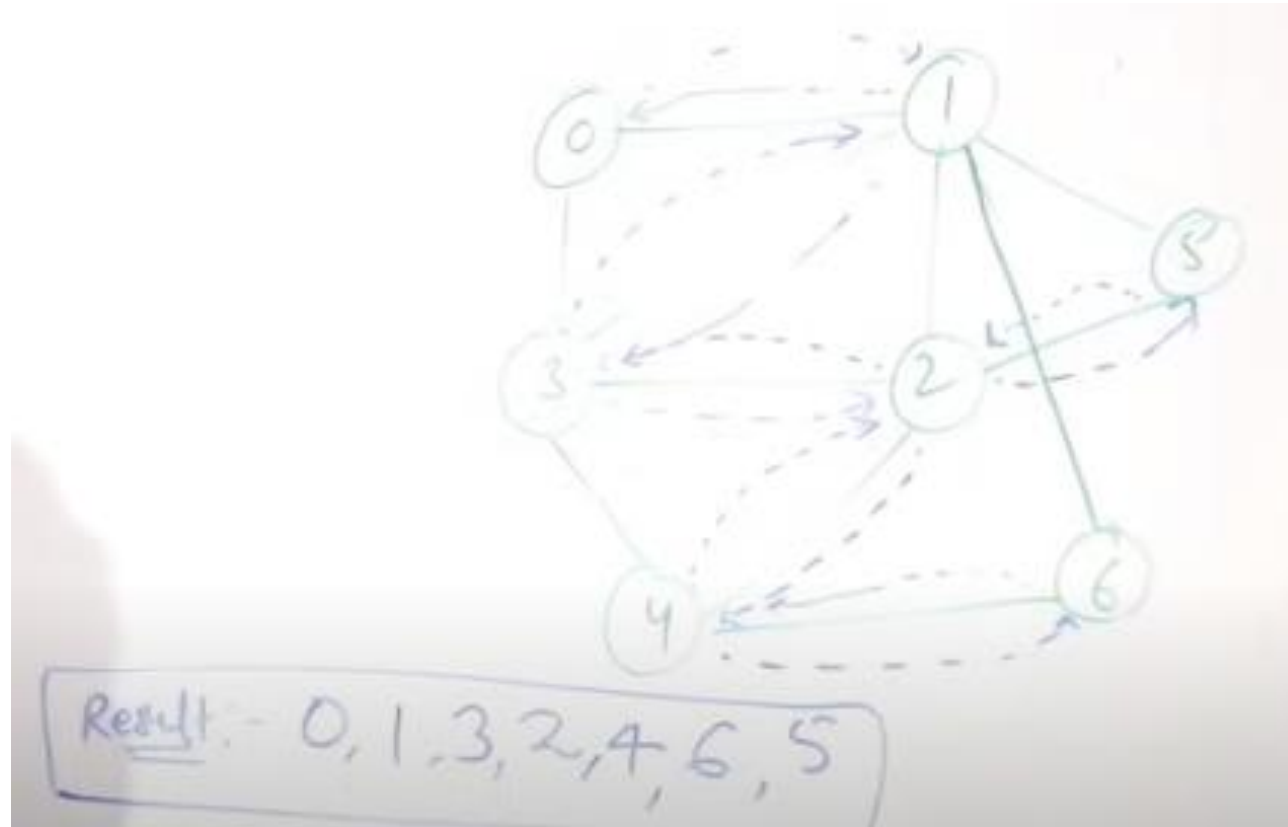


Graph - Traversals

- Stack became Empty. So stop DFS Traversal.
- Final result of DFS traversal is following spanning tree.



Graph - Traversals



Graph - Traversals

BFS: Use when you need the shortest path or level-order traversal.

1. Find the Shortest Path (BFS):

- **Scenario:** You need the shortest path between two points (e.g., from S to E in a maze).
- **What BFS does:** It explores all neighboring points level by level, ensuring that when it reaches the destination, it's via the shortest path.
- **Why BFS:** It guarantees the shortest path, as it explores all nodes at a given distance before moving on to nodes at the next level.

2. Level-Order Traversal (BFS):

- **Scenario:** You want to visit nodes level by level (e.g., in a tree structure).
- **What BFS does:** It visits all nodes at the current level before moving to the next level.
- **Why BFS:** It's perfect for printing or processing nodes in a tree in level-order (top to bottom, left to right).

3. Finding Connections (BFS):

- **Scenario:** You want to check if two nodes are connected (e.g., if two people are friends in a social network).
- **What BFS does:** It explores all possible connections starting from one point, level by level.
- **Why BFS:** It's great for finding connections and shortest paths in unweighted graphs (like checking if someone is connected to another person through friends).

Graph - Traversals

DFS: Use when you need to explore deeply, find all paths, or sort tasks with dependencies.

1. Explore Deeply (DFS):

- **Scenario:** Solving a maze.
- **What DFS does:** It goes as deep as possible into one path, exploring until it reaches a dead-end, then backtracks and tries another path.
- **Why DFS:** You want to explore every possible route in depth before backtracking to explore other options.

2. Find All Paths (DFS):

- **Scenario:** Finding all possible routes in a maze.
- **What DFS does:** It explores every single possible path from start to end, even if it takes a longer route.
- **Why DFS:** You want to find all possible solutions or paths, not just the shortest one.

3. Sort Tasks with Dependencies (DFS):

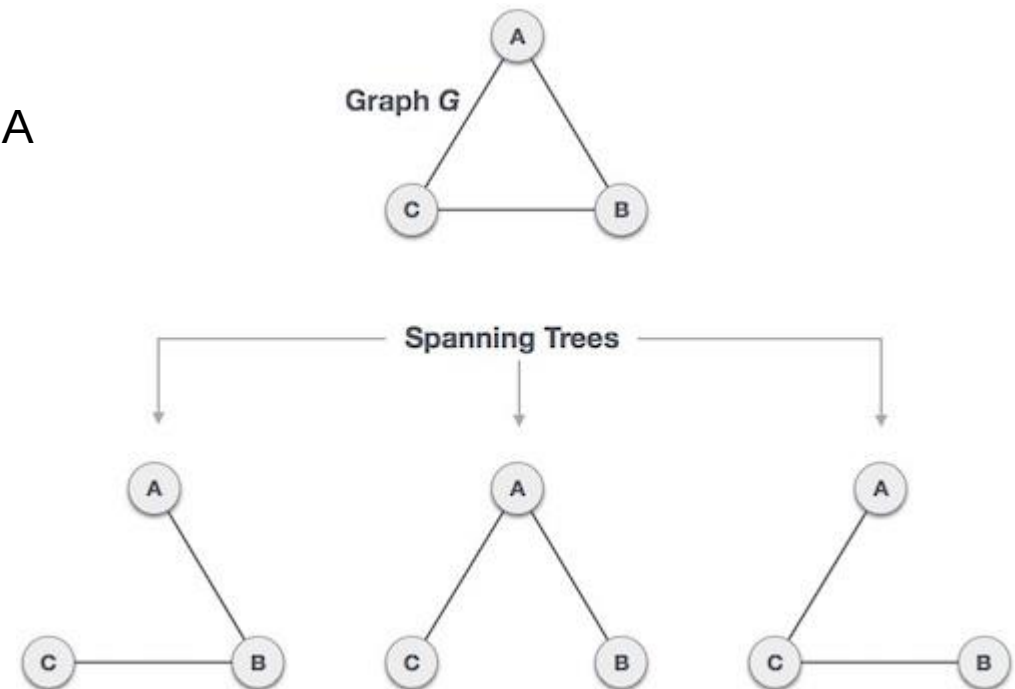
- **Scenario:** Scheduling tasks where some tasks depend on others.
- **What DFS does:** It ensures that each task is done in the correct order, considering dependencies.
- **Why DFS:** You want to respect task dependencies and find the right order to complete the tasks.

Spanning Tree

A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..

By this definition, we can draw a conclusion that every connected and undirected Graph G has at least one spanning tree. A disconnected graph does not have any spanning tree, as it cannot be spanned to all its vertices.

We found three spanning trees off one complete graph. A complete undirected graph can have maximum n^{n-2} number of spanning trees, where n is the number of nodes. In the above addressed example, n is **3**, hence $3^{3-2} = 3$ spanning trees are possible.



Spanning Tree

General Properties of Spanning Tree

We now understand that one graph can have more than one spanning tree. Following are a few properties of the spanning tree connected to graph G –

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G , have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

Spanning tree is basically used to find a minimum path to connect all nodes in a graph.

One example is **Computer Network Routing Protocol**

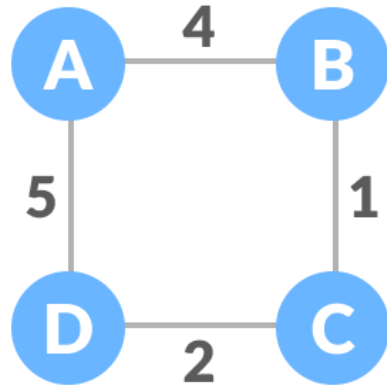
Minimum Spanning Tree

A minimum spanning tree is a spanning tree in which the sum of the weight of the edges is as minimum as possible.

Example of a Spanning Tree

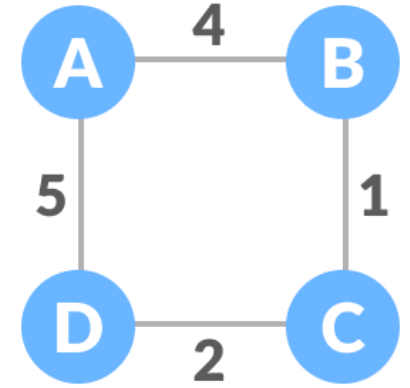
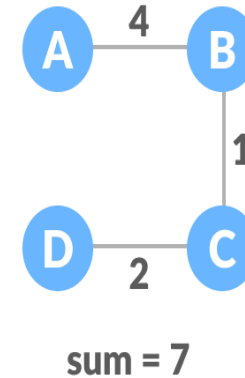
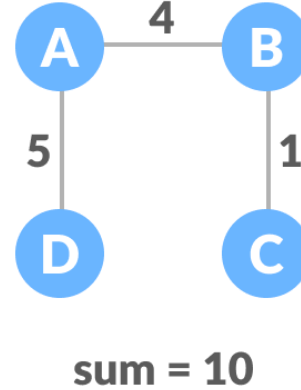
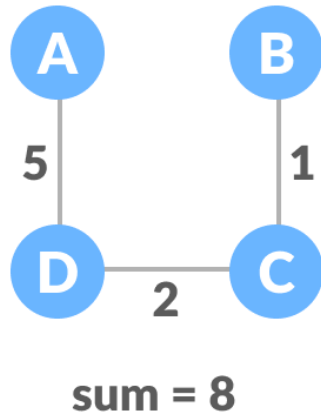
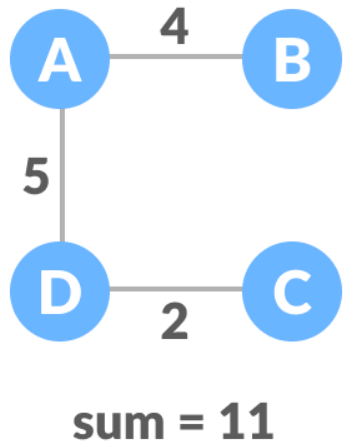
Let's understand the above definition with the help of the example below.

The initial graph is:

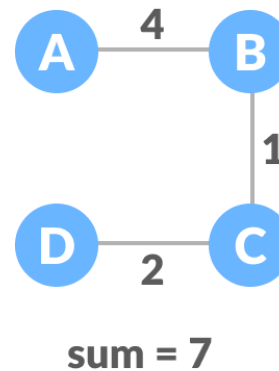


Minimum Spanning Tree

The possible spanning trees from the above graph are:



The minimum spanning tree from the above spanning trees is:



Minimum Spanning Tree

The minimum spanning tree from a graph is found using the following algorithms:

1. Prim's Algorithm
2. Kruskal's Algorithm

Minimum Spanning tree Applications

- To find paths in the map
- To design networks like telecommunication networks, water supply networks, and electrical grids.

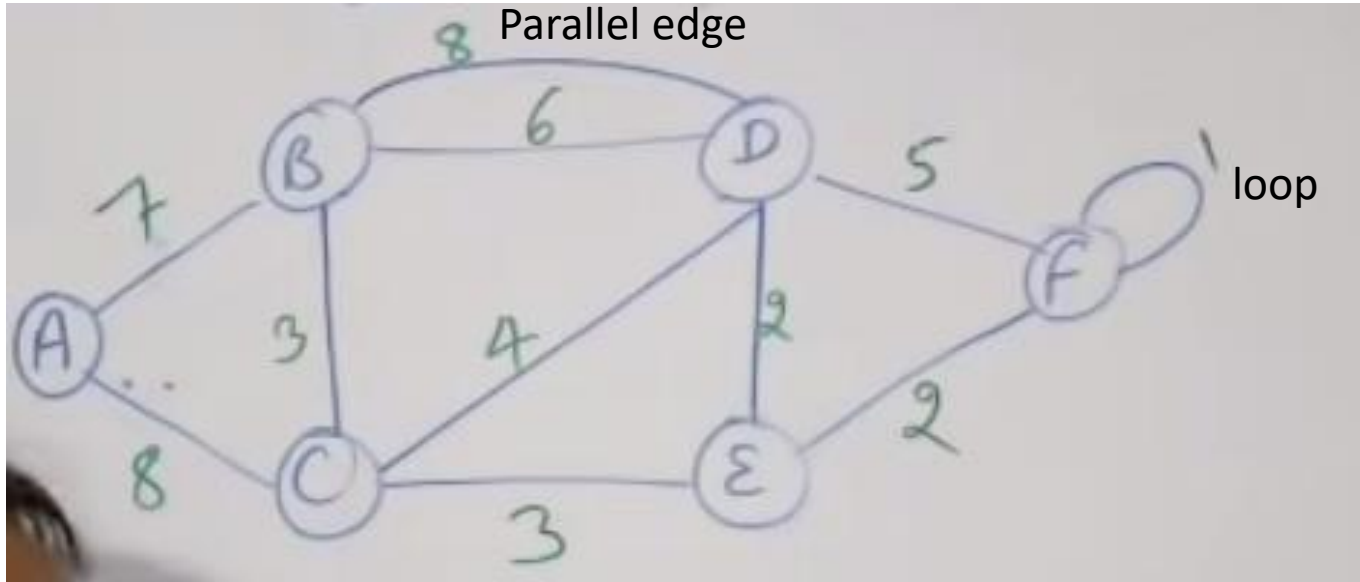
Prim's Minimum Spanning Tree Algorithm

- It is used for finding the Minimum Spanning Tree (MST) of a given graph.
- To apply Prim's algorithm, the given graph must be weighted, connected and undirected.

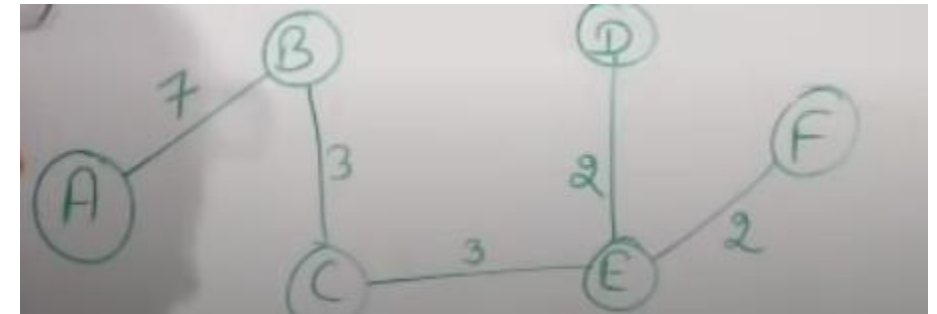
First remove loop and parallel edges if any.

- Step 1: Randomly choose any vertex. The vertex connecting to the edge having least weight is usually selected.
- Step 2: Find all the edges that connect the tree to new vertices. Find the least weight edge among those edges and include it in the existing tree. If including that edge creates a cycle, then reject that edge and look for the next least weight edge.
- Step 3: Keep repeating step-02 until all the vertices are included and Minimum Spanning Tree (MST) is obtained.

Prim's Minimum Spanning Tree Algorithm



$G(V,E)$



MST

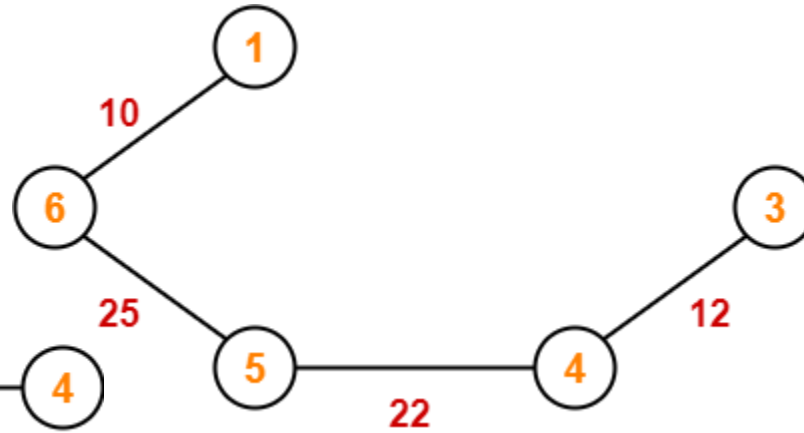
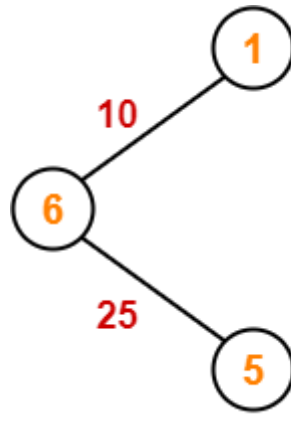
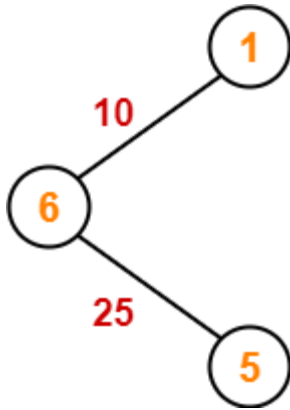
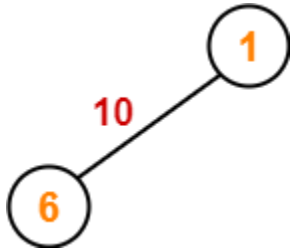
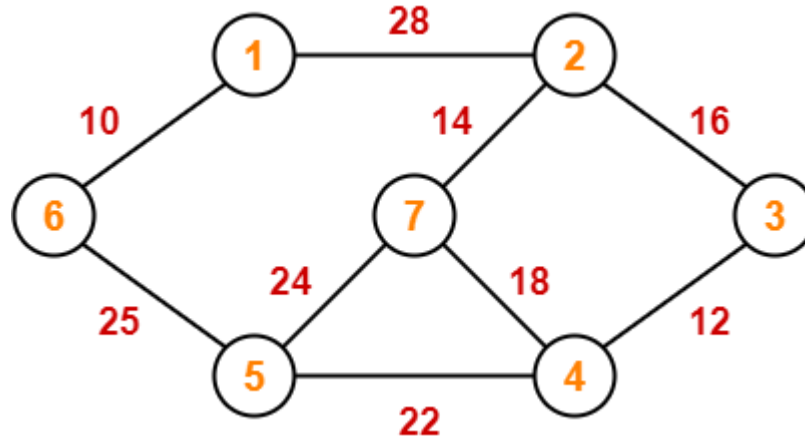
$G'(V',E')$

No of vertices in MST, $v' = v$ (no of vertices in original graph)

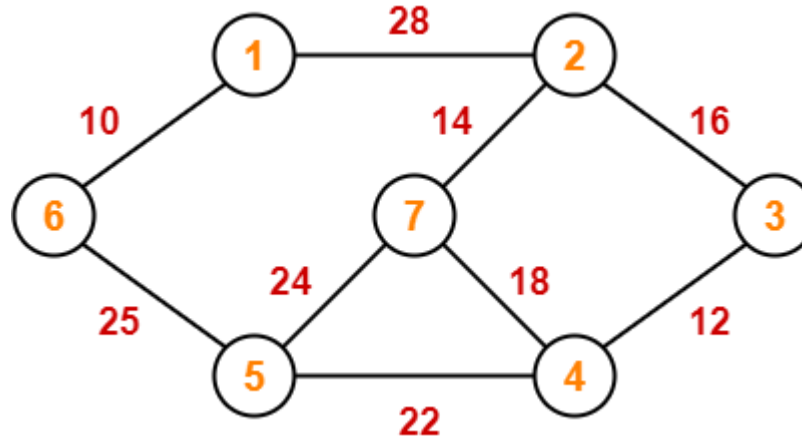
No of edges E' is the subset of E

No of edges in MST, $E' = |V| - 1$, $6 - 1 = 5$

Prim's Algorithm - Examples



Prim's Algorithm - Examples



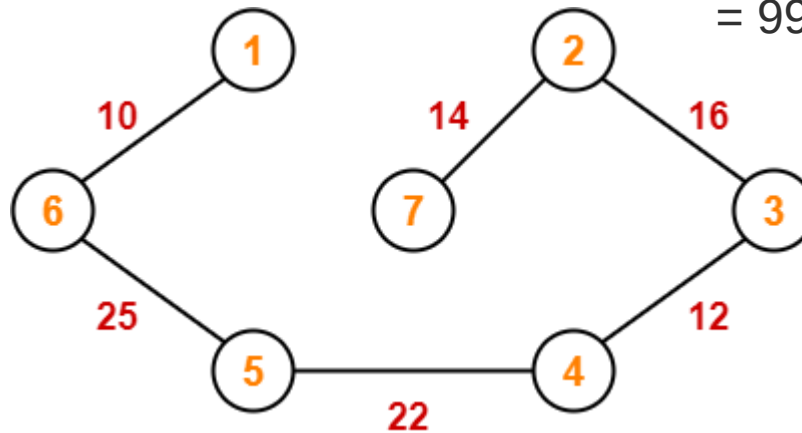
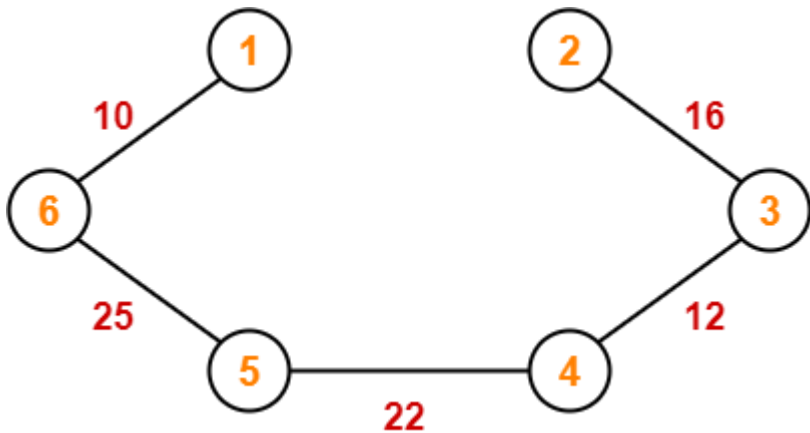
Since all the vertices have been included in the MST, so we stop.

Now, Cost of Minimum Spanning Tree

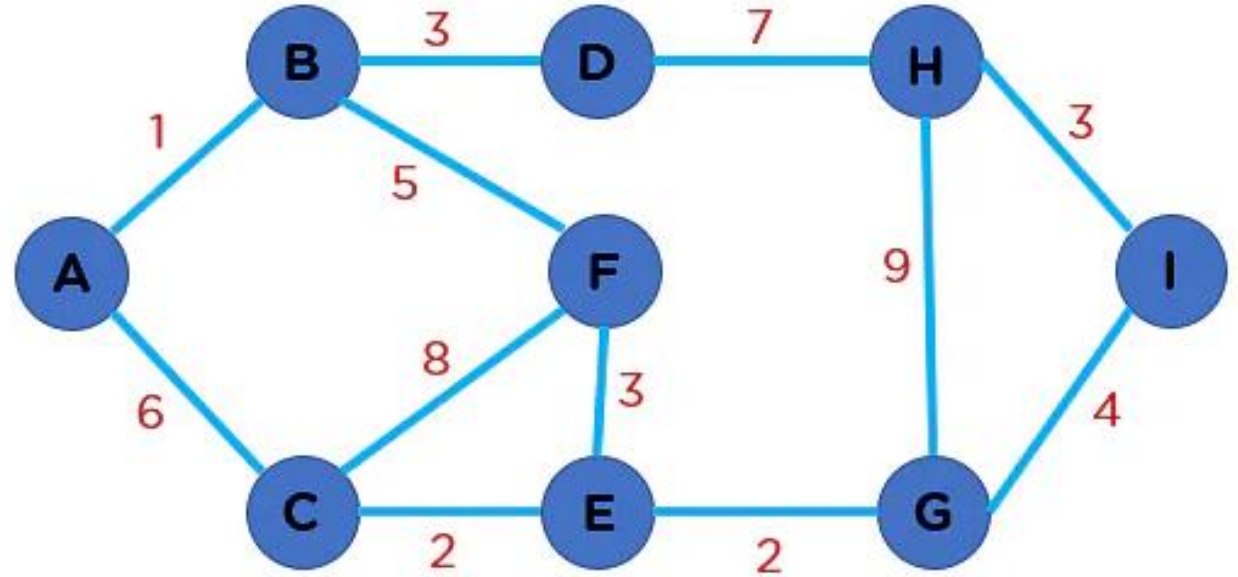
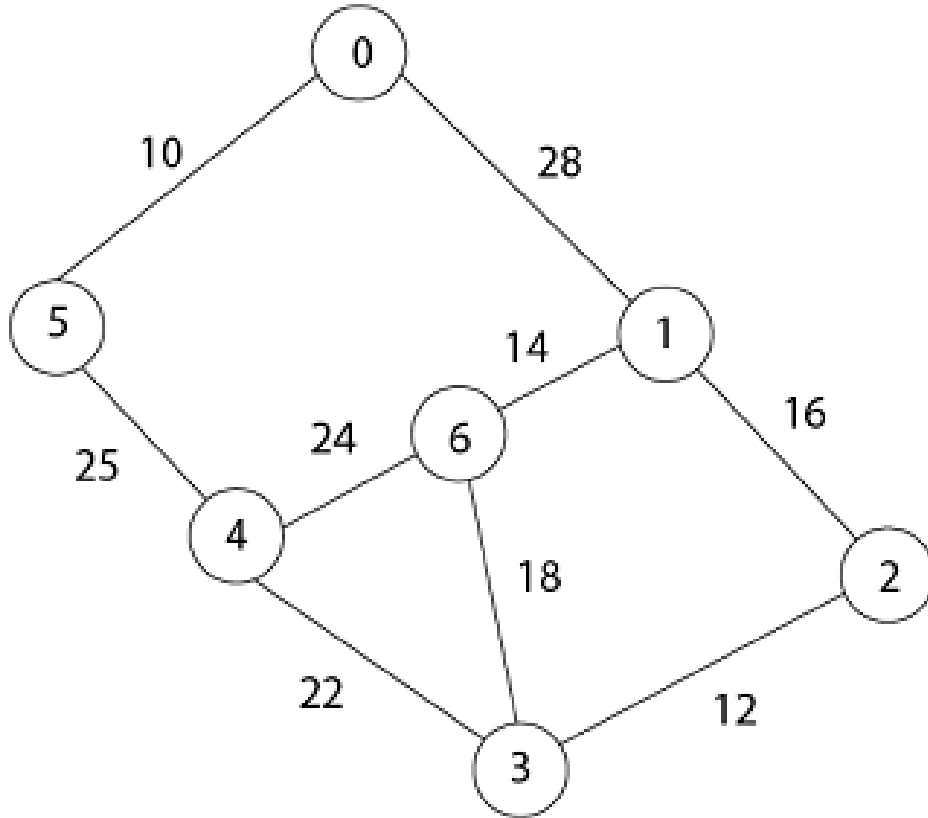
= Sum of all edge weights

= $10 + 25 + 22 + 12 + 16 + 14$

= 99 units



Prim's Algorithm - Examples



Kruskal's algorithm

Step 1: Sort all edges in increasing order of their edge weights.

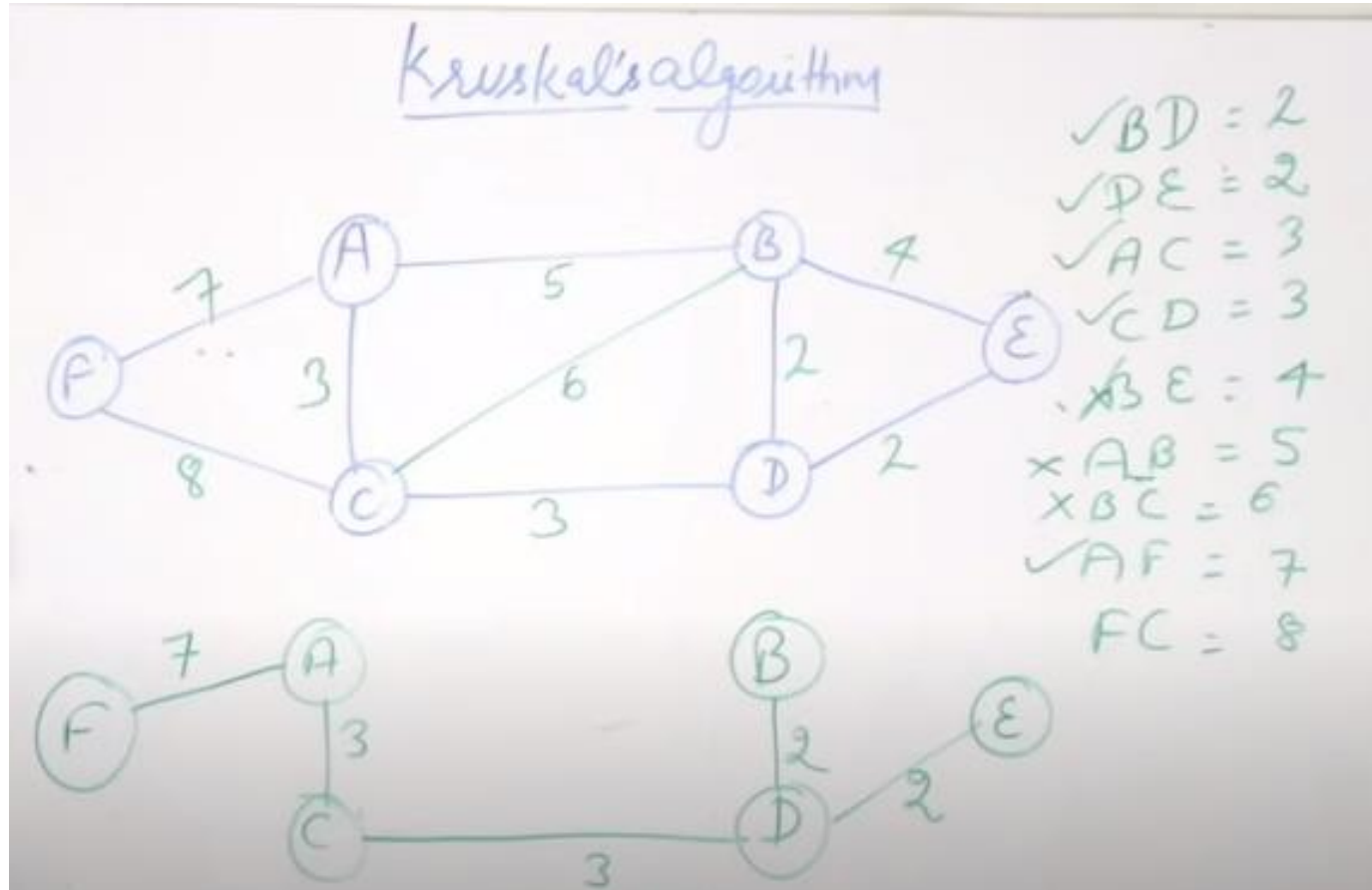
Step 2: Pick the smallest edge.

Step 3: Check if the new edge creates a cycle or loop in a spanning tree.

Step 4: If it doesn't form the cycle, then include that edge in MST. Otherwise, discard it.

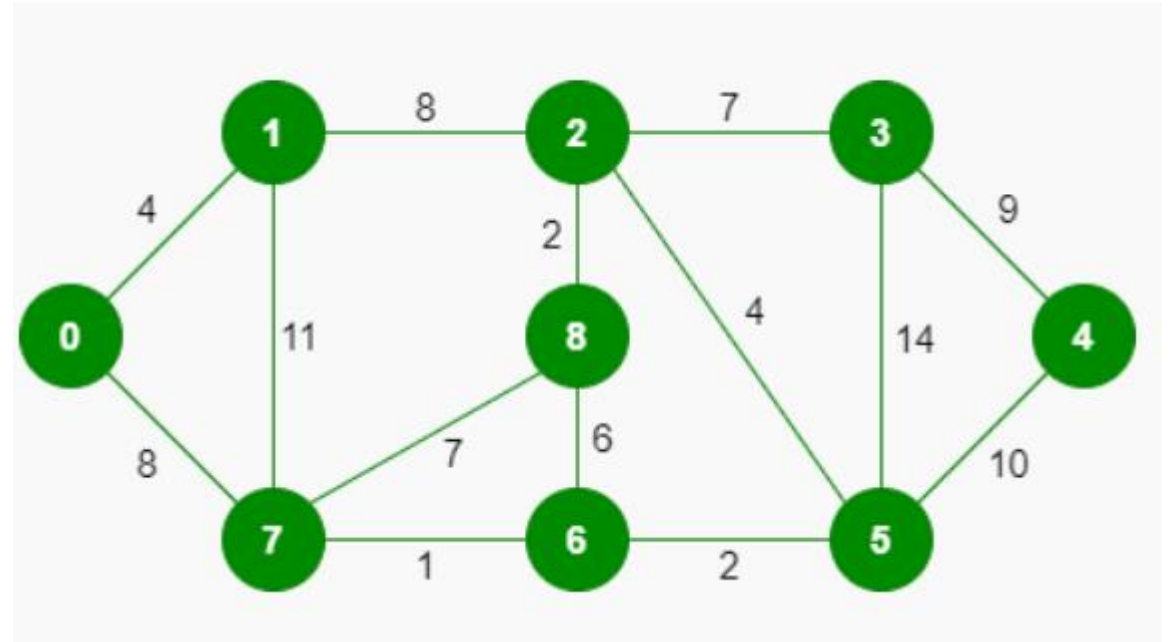
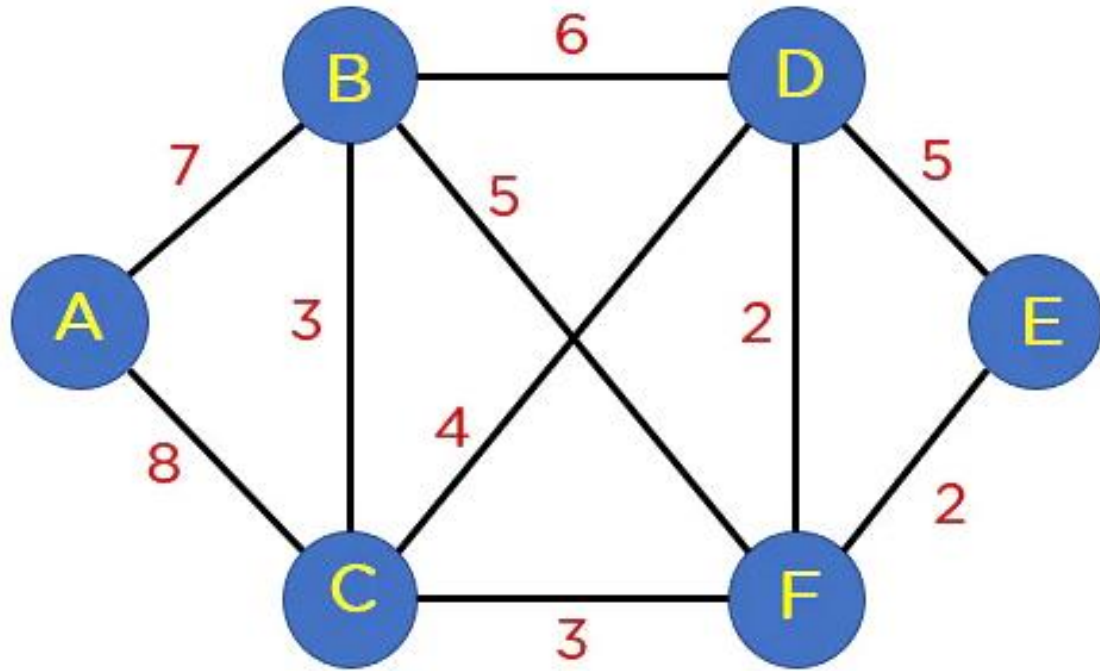
Step 5: Repeat from step 2 until it includes $|V| - 1$ edges in MST.

Kruskal's algorithm - Examples

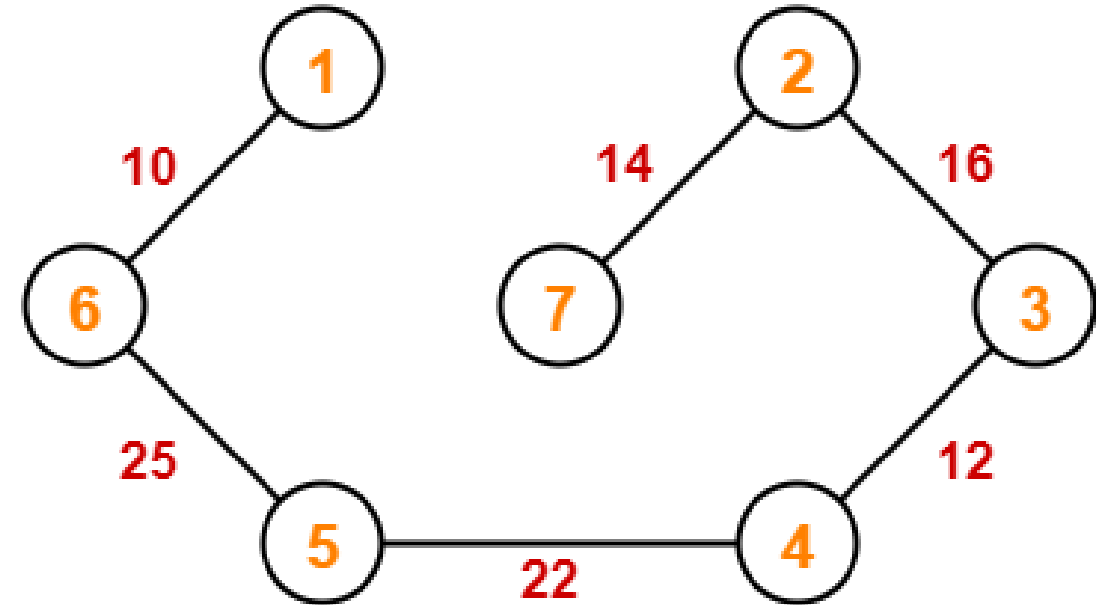
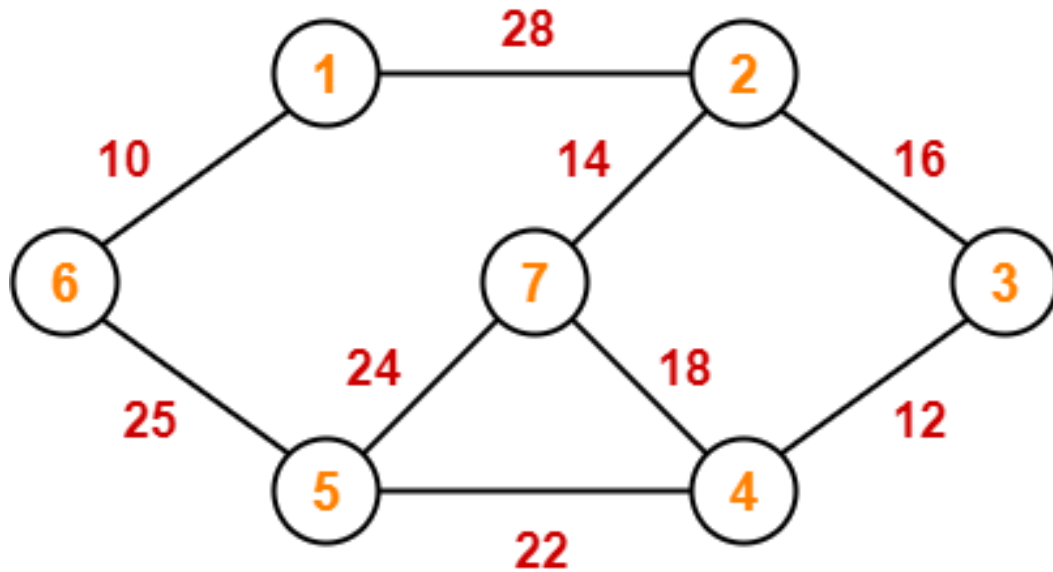


MST should not have cycle and no of vertices are same as the original graph and no of edges are no of vertices -1

Kruskal's algorithm - Examples



Kruskal's algorithm - Examples



Dijkstra's Algorithm

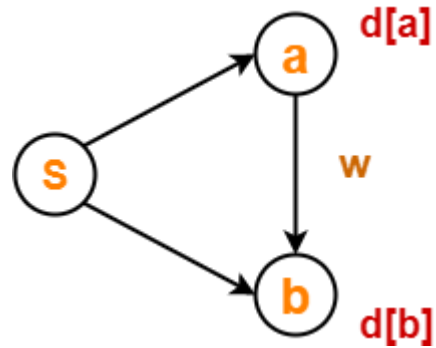
- Dijkstra algorithm is a single-source shortest path algorithm.
- With Dijkstra's Algorithm, you can find the shortest path between nodes in a graph. Particularly, you can **find the shortest path from a node (called the "source node") to all other nodes in the graph**, producing a shortest-path tree.
- This algorithm is used in GPS devices to find the shortest path between the current location and the destination.

Dijkstra's Algorithm - Basics

- Dijkstra's Algorithm basically starts at the node that you choose (the source node) and it analyzes the graph to find the shortest path between that node and all the other nodes in the graph.
- The algorithm keeps track of the currently known shortest distance from each node to the source node and it updates these values if it finds a shorter path.
- Once the algorithm has found the shortest path between the source node and another node, that node is marked as "visited" and added to the path.
- The process continues until all the nodes in the graph have been added to the path. This way, we have a path that connects the source node to all other nodes following the shortest path possible to reach each node.

Dijkstra's Algorithm - Basics

- Consider the edge (a,b) in the following graph-
- Update the node value

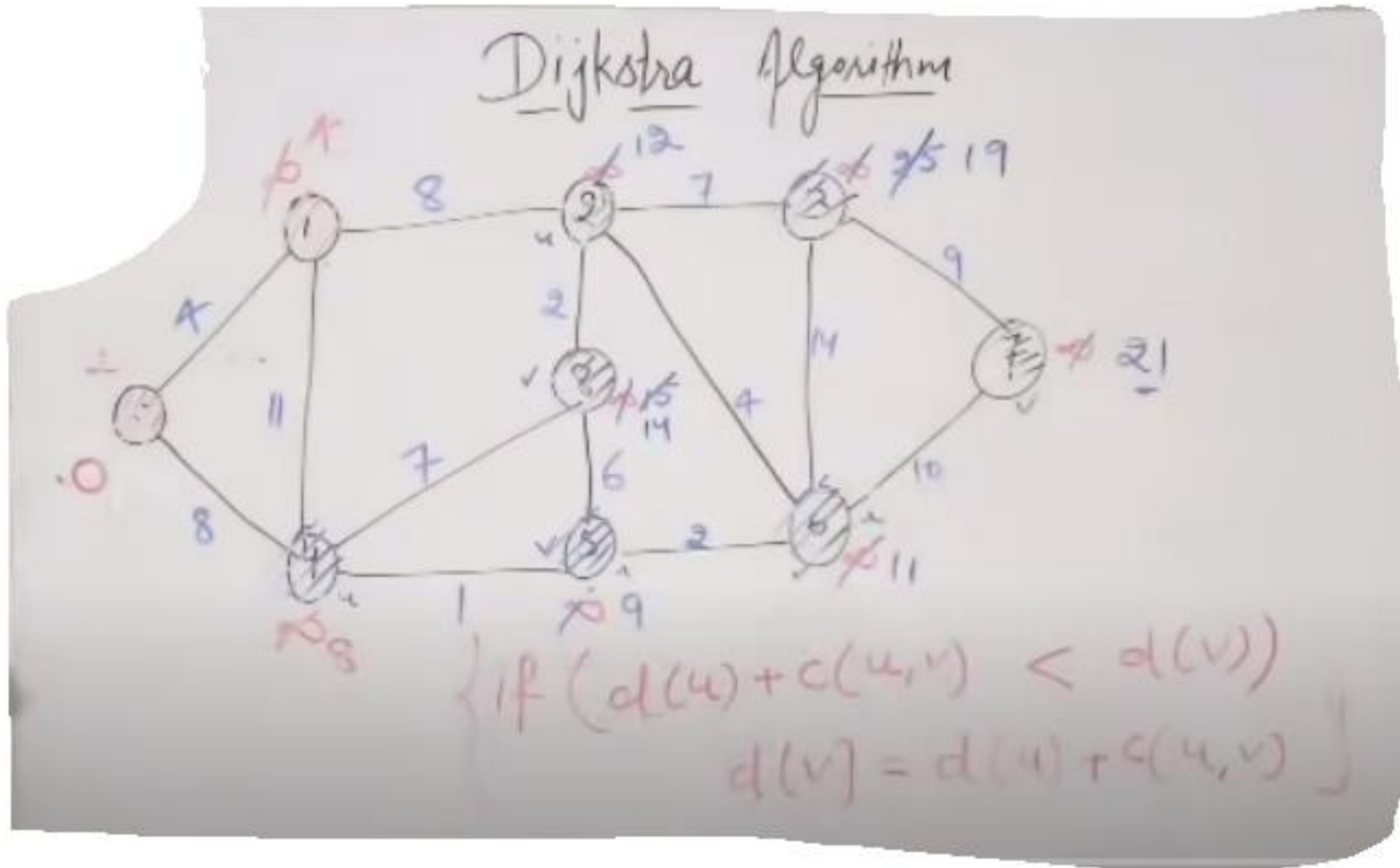


Here, $d[a]$ and $d[b]$ denotes the shortest path estimate for vertices **a** and **b** respectively from the source vertex '**S**'.

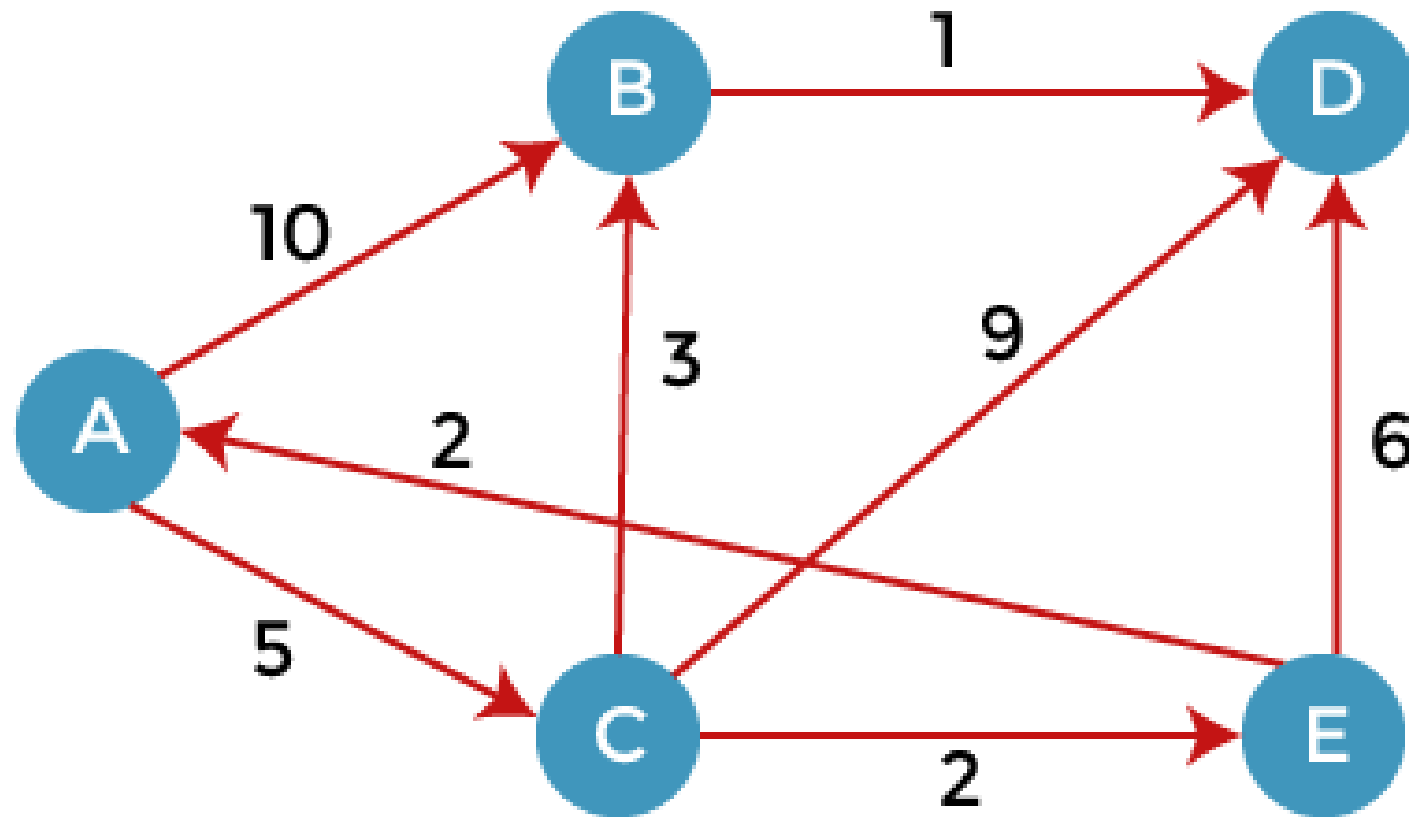
Now,

If $d[a] + w < d[b]$
then $d[b] = d[a] + w$

Dijkstra's Algorithm



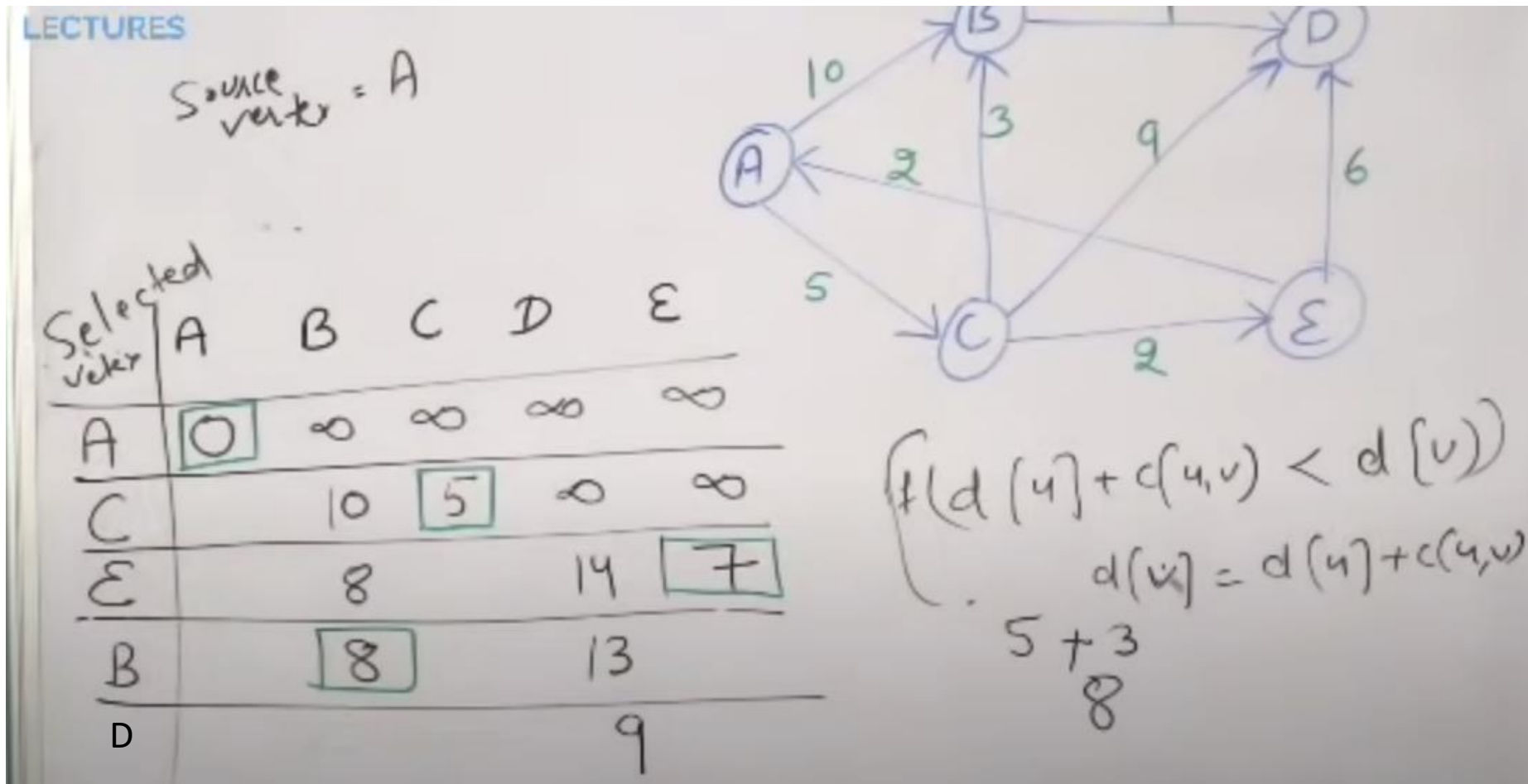
Dijkstra's Algorithm



Dijkstra's Algorithm

	A	B	C	D	E
A	0	∞	∞	∞	∞
C		10	5	∞	∞
E		8		14	7
B		8		13	
D				9	

Dijkstra's Algorithm - example



A to D
Path - DBCA
 [ACBD] = 9

Dijkstra's Algorithm - example

